# V9968

# MSX-VIDEO

## TECHNICAL DATA BOOK

V9968 TECHNICAL DATA BOOK

2026.10

# 目次


| 項目 | ページ |
|---|---|
| 第 1 章 はじめに | 3 |
| 第 2 章 I/O ポートとアクセス方法 | 6 |
| 第 3 章 コントロールレジスタ詳説 | 12 |
| 第 4 章 ステータスレジスタ詳説 | 34 |
| 第 5 章 スクリーンモード | 39 |
| 第 6 章 スプライト | 56 |
| 第 7 章 カラーパレットと色表現 | 71 |
| 第 8 章 VDP コマンド | 78 |
| 第 9 章 Geo3D | 108 |
| 第 10 章 割り込み | 120 |
| 第 11 章 VRAM アクセスとタイミング | 127 |
| 第 12 章 互換性と移行の注意 | 131 |
| 付録 A コントロールレジスター覧 | 135 |
| 付録 B ステータスレジスター覧 | 155 |
| 付録 C スクリーンモード一覧 | 159 |
| 付録 D VDP コマンドコード一覧 | 176 |
| 付録 E I/O ポート一覧 | 177 |
| 索引 | 179 |

# 第 1 章 はじめに

## 1.1 本書について

本書は、MSX2++ 向けに開発された Video Display Processor（VDP）「V9968」のプログラミングマニュアルである。V9968 を使って画面表示を行う MSX 向けソフトウェアの開発者を対象に、V9968 の全機能を解説する。

まず第 2 章で CPU から V9968 にアクセスするための基本手順を説明し、以降の章で各機能を詳しく説明する。レジスタのビット単位の仕様だけを手早く確認したい場合は、第 3 章・第 4 章および付録 A を参照されたい。

## 1.2 表記規則

本書では次の表記を用いる。

- コントロールレジスタは **R#n**（例：R#7）、ステータスレジスタは **S#n**（例：S#0）と表記する。
- 16 進数は末尾に h を付けて表記する（例：98h、3FFFh）。先頭が英字になる場合は 0 を前置する（例：0FFh）。
- ビットは bit7 を最上位、bit0 を最下位とする。ビット範囲は [7:0] のように表記する。
- VRAM のアドレス信号は A17〜A0 と表記する。
- 機能の由来を示すため、V9958 から搭載された機能には「9958 Compatible」と付記することがある。同様に、V9938 から搭載された機能はその旨を記す。
- 「互換モード」「拡張モード」は、R#21 の V58 ビットで切り替わる V9968 の動作モードを指す（1.4 節）。

## 1.3 V9968 の概要

V9968 は、MSX2++ 向けに開発された Video Display Processor（VDP）である。

MSX シリーズの VDP は、初代 MSX の TMS9918、MSX2 の V9938、MSX2+ および MSX turboR の V9958 と受け継がれてきた。V9968 はこの系譜の後継にあたり、TMS9918 / V9938 / V9958 との互換性を持ちつつ、新たな機能を搭載している。従来の VDP 向けに書かれたソフトウェアの多くは、原則としてそのまま V9968 でも動作する。

一方で、V9968 には従来の VDP にはなかった拡張機能も搭載されている。本書では互換機能と拡張機能を区別して説明するので、拡張機能を使う場合は該当する章をよく確認されたい。

## 1.4 互換性と「9958 Compatible」モード

V9968 には、動作モードを切り替える **V58 ビット**（R#21 の bit0）がある。

- **互換モード（V58 = 1）**：リセット時の状態。V9958 と同等の動作をするモードである。このモードでは、テーブルベースアドレスの A17 が 0 として扱われ、新設の VDP コマンド（1.5 節）は使えない。
- **拡張モード（V58 = 0）**：V9968 の拡張機能が有効になるモードである。VRAM 256KB（18 ビットアドレス）が使えるようになり、新設コマンドや拡張パレットなどが有効になる。

なお、V58 ビットを持つ R#21 と、その他の拡張レジスタである R#20 は、誤操作防止のためのロック機構（LockExtReg）で保護されており、リセット直後は書き込みが無視される。ロックの解除方法は第 2 章で説明する。

本書では、V9958 互換モードで従来通り動作する機能を「9958 Compatible」と呼ぶことがある（1.2 節）。

## 1.5 機能概要

### 画面モード

V9968 の画面モードは V9958 と同等であり、新たな画面モードは存在しない。Text1（SCREEN0 WIDTH40）、Text2（SCREEN0 WIDTH80）、Multicolor（SCREEN3）、Graphic1〜Graphic7（SCREEN1、2、4〜8）、Graphic7-YAE（SCREEN10、11）、Graphic7-YJK（SCREEN12）の各モードを備える。各モードの設定方法と VRAM の使い方、細かい挙動については第 5 章で説明する。

### スプライト

スプライトは、従来互換の Sprite mode1、Sprite mode2 に加え、V9968 で新設された **Sprite mode3** を備える。

Sprite mode3 では、ドットバイドットで色付けができ、拡大・縮小、左右反転、上下反転が可能である。表示枚数は最大 64 枚、1 ラインあたり 16 枚まで表示できる。詳細は第 6 章で説明する。

### カラーパレット

カラーパレットは 256 エントリを持ち、各色を RGB 各 5 ビットで保持する。従来形式（RGB 各 3 ビット、16 色）での書き込みに加え、拡張パレットモード（EPAL）では 5 ビット値を直接書き込める。詳細は第 7 章で説明する。

### VDP コマンド

VDP コマンドは、V9958 までの 13 種類のコマンドに加え、拡張モードでのみ使える **LRMM、LFMC、LFMM** の 3 種類が新設された。また、回転パラメータ（R#47～R#50）やクリッピング用のウィンドウ指定（R#51～R#58）など、コマンド関連レジスタも拡張されている。詳細は第 8 章で説明する。

### Geo3D

V9968 には 3D 描画エンジン「Geo3D」が搭載されている。詳細は第 9 章で説明する。

### 割り込み

垂直同期割り込み（フレーム割り込み）とライン割り込みに加え、VDP コマンドの完了を通知するコマンド完了割り込み（CEIE、R#20）が用意されている。詳細は第 10 章で説明する。

### VRAM

VRAM は、拡張モードでは 256KB（アドレス A17～A0 の 18 ビット）までアクセスできる。互換モードでは従来互換のアドレス範囲で動作する。アクセス方法は第 2 章、アクセスのタイミングや制約は第 11 章で説明する。

# 第 2 章 I/O ポートとアクセス方法

V9968 へのアクセスは、すべて I/O ポート経由で行う。本章では、ポートの割り当てと、VRAM・コントロールレジスタ・パレット・ステータスレジスタへの基本的なアクセス手順を説明する。各レジスタのビット仕様は第 3 章・第 4 章を参照されたい。

## 2.1 ポート割り当て

V9968 のベースアドレスは、BIOS ワークエリアの 0006h / 0007h に格納された値である（通常は 98h）。ベースアドレスを基準として、次の 5 つのポートが割り当てられている。

| ポート | 書き込み | 読み出し |
|---|---|---|
| Port#0（base+0） | VRAM データ書き込み | VRAM データ読み出し |
| Port#1（base+1） | 2 バイト組プロトコル（2.3 節） | R#15 が指すステータスレジスタ |
| Port#2（base+2） | パレットデータ書き込み | FFh を返す（機能なし） |
| Port#3（base+3） | R#17 ポインタ経由の間接レジスタ書き込み | FFh を返す（機能なし） |
| Port#4（base+4） | 割り込みフラグクリア＋LockExtReg 設定 | 割り込み状態 |

以下、各ポートの使い方を順に説明する。

なお、V9968 が 2 台目の VDP（2nd VDP）として接続される構成では、ベースアドレスは 88h になる。この場合、ワークエリアの 0006h / 0007h は内蔵 VDP の値（98h）のままであるため、プログラムは 88h を直接指定してアクセスすること。また、2nd VDP の垂直同期割り込みの扱いは内蔵 VDP と異なる（10.8 節の注意を参照）。

## 2.2 VRAM へのアクセス

VRAM の読み書きは Port#0 で行う。アクセス先のアドレスは、事前に Port#1 の 2 バイト組プロトコル（2.3 節）で設定する。

V9968 の VRAM アドレスは 18 ビット（A17〜A0）で、アドレスレジスタは読み出し・書き込み共通である。アドレスの下位

14ビット（A13〜A0）は 2 バイト組プロトコルで設定し、上位の A17〜A14 は R#14 で別途指定する。

### アドレス設定後の動作

- 書き込み用にアドレスを設定した場合、Port#0 に書いたデータがそのアドレスに書き込まれ、アドレスがインクリメントされる。ただし、アドレス設定直後の最初のアクセスではインクリメントされない。
- 読み出し用にアドレスを設定した場合、Port#0 の最初の読み出しで、設定したアドレスのデータが返る。読み出しアドレスは、VRAM からの応答時にインクリメントされる。
- 2 バイト組の 2 バイト目の bit6 が、読み出し用（0）か書き込み用（1）かの区別になる（2.3 節）。

### アドレスのラップアラウンド

アドレスをインクリメントした際の桁上がり（ラップアラウンド）の範囲は、動作条件によって次のように変わる。

| 条件 | ラップ範囲 |
|---|---|
| 画面モード {M5, M4} が 00 の系統（Graphic1 / Text1 / Multicolor / Graphic2）、または R#8 の VR = 0 のとき | 14ビット（A13〜A0 のみ進む） |
| 上記以外で、互換モード（V58 = 1）のとき | 17ビット |
| 上記以外で、拡張モード（V58 = 0）のとき | 18ビット |

14ビットでラップする系統では、たとえば Text1 モードで 3FFFh の次は 0000h に戻る。これは TMS9918 搭載機との互換性のために設けられた意図的な仕様である。R#14 で A17〜A14 を指定すれば 4000h 以降の領域にもアクセスできるが、bit13 から bit14 への桁上がりが遮断されているため、7FFFh の次は 4000h に戻る点に注意すること。

また、R#14 に設定する A17 は、R#8 の VR ビットが 0 のときは 0 固定として扱われ、互換モード（V58 = 1）のときもマスクされる。

### コマンド実行中のアクセス制限

R#45 の MXC ビットが 1 の間は、CPU からの VRAM アクセスはマスクされる。この間、VRAM への書き込みは実行されず、読み出しは FFh を返す。MXC については第 8 章を参照されたい。

## 2.3 Port#1 の 2 バイト組プロトコル

Port#1 への書き込みは、常に 2 バイトを 1 組として扱う。1 バイト目に値（データまたはアドレス下位）を書き、2 バイト目の bit7 / bit6 でその意味を指定する。

| 2 バイト目の bit7 | 2 バイト目の bit6 | 意味 |
|---|---|---|
| 1 | — | コントロールレジスタ書き込み。bit5〜bit0 で指定したレジスタ（R#0〜R#63）に、1 バイト目の値を書き込む |
| 0 | 1 | VRAM 書き込み用アドレス設定。bit5〜bit0 が A13〜A8 となる |
| 0 | 0 | VRAM 読み出し用アドレス設定。bit5〜bit0 が A13〜A8 となる |

どちらの場合も、1 バイト目は VRAM アドレスの下位 8 ビット（A7〜A0）、あるいはレジスタへの書き込みデータである。

### 組の破棄（アボート）

2 バイト組の 1 バイト目を書いた後、2 バイト目を書く前に Port#0 へアクセスしたり、いずれかのポートを読み出したりすると、書きかけの組は破棄される。1 バイト目だけを書いた状態で放置しないこと。

なお、Port#3 への書き込み（2.4 節）はこの 2 バイト組の状態に影響しない。

## 2.4 コントロールレジスタへの書き込み

コントロールレジスタはすべて書き込み専用であり、直接指定と間接指定の 2 つの方法で書き込める。

### 直接指定（Port#1）

2.3 節のプロトコルで、1 バイト目に値を、2 バイト目に 80h ＋レジスタ番号を書く方法である。任意のレジスタを 1 回の組で書き換えられる。

### 間接指定（Port#3）

R#17（コントロールレジスタポインタ）が指すレジスタへ、Port#3 に値を書くことで書き込む方法である。連続したレジスタへ順に値を流し込む用途に向いている。

- R#17 の bit5〜bit0 がポインタ、bit7 が AII（オートインクリメント禁止）である。
- AII = 0 のとき、Port#3 へ 1 回書くごとにポインタがインクリメントされる。AII = 1 のときはポインタは固定される。
- R#17 自体を設定できるのは直接指定（Port#1 経由）のみである。Port#3 経由でポインタが 17 を指していても、その値は無視される（ただし AII = 0 ならポインタだけはインクリメントする）。

## 2.5 パレットへのアクセス

パレットデータの書き込みは Port#2 で行う。書き込み先のパレット番号は、あらかじめ R#16（カラーパレットポインタ）で指定する。R#16 を書くと、パレット書き込みのフェーズ（何バイト目か）は 0 に戻る。

書き込み形式は、R#20 の EPAL ビットによって 2 通りある。

### 通常形式（EPAL = 0）

R#16 の bit3〜bit0 で番号（0〜15）を指定し、Port#2 に 2 バイトを書く。

| バイト | ビット構成 |
|---|---|
| 1 バイト目 | [0][R2][R1][R0][0][B2][B1][B0] |
| 2 バイト目 | [0][0][0][0][0][G2][G1][G0] |

RGB 各 3 ビットの値は、内部では上位ビットを下位にも複製する形で 5 ビットに拡張されて格納される。1 組書き込むごとにパレット番号は [3:0] の範囲で自動インクリメントされる（上位 4 ビットは 0 固定）。

### 拡張形式（EPAL = 1）

R#16 の bit7〜bit0 で番号（0〜255）を指定し、Port#2 に 3 バイトを書く。各バイトは [0][0][0][値 4][値 3][値 2][値 1][値 0] の 5 ビット値で、順に R、G、B を表す。1 組書き込むごとにパレット番号は 8 ビットで自動インクリメントされる。

パレット RAM は 256 エントリ × RGB 各 5 ビットの構成である。色表現の詳細は第 7 章を参照されたい。

なお、リセット時に初期化されるのは先頭の 16 エントリのみであり、V9958 互換の初期カラーに初期化される。16 番以降のエントリの初期値は不定である。

## 2.6 ステータスレジスタの読み出し

ステータスレジスタは読み出し専用である。R#15（ステータスレジスタポインタ）に番号を指定し、Port#1 を読むことで、そのステータスレジスタ S#0〜S#9 の値が得られる。リセット時のポインタは 0 である。S#10〜S#15 を指定した場合は FFh が返る。

ステータスレジスタの中には、読み出すことでフラグ等がクリアされるものがある。主なものは次の通りである。

| レジスタ | 読み出し時の副作用 |
|---|---|
| S#0 | 割り込みフラグ F、スプライト衝突 C、overmap フラグ 5S がクリアされる |
| S#1 | ライン割り込みフラグ FH がクリアされる |
| S#5 | スプライト衝突座標（X / Y）がクリアされる |
| S#9 | S#2 の BD がクリアされる |

各ステータスレジスタのビット仕様と副作用の詳細は第 4 章を参照されたい。

## 2.7 割り込み制御ポート（Port#4）

Port#4 は、割り込み要因の確認とクリア、および拡張レジスタのロック（LockExtReg）を扱うポートである。

### 読み出し

読み出した値のビット構成は次の通りである。

| bit | 名称 | 意味 |
|---|---|---|
| bit7 | LockExtReg | 拡張レジスタのロック状態（リセット時 1） |
| bit6〜bit3 | — | 0 |
| bit2 | CI | コマンド完了割り込みの要因が発生中なら 1 |
| bit1 | LI | ライン割り込みの要因が発生中なら 1 |

| bit | 名称 | 意味 |
|---|---|---|
| bit0 | FI | フレーム割り込みの要因が発生中なら 1 |

### 書き込み

- bit2 / bit1 / bit0 に 1 を書くと、対応する割り込みフラグ（コマンド完了 / ライン / フレーム）がクリアされる。0 を書いても何も起きない。
- bit7 は LockExtReg そのものへの書き込みである。

## 拡張レジスタのロック（LockExtReg）

LockExtReg はリセット時に 1（ロック状態）である。ロック中は、拡張レジスタである **R#20 と R#21 への書き込みが無視される**。

拡張機能を使う場合は、次の手順でロックを解除してから設定を行う。

1. Port#4 に bit7 = 0 の値を書く（LockExtReg が 0 になり、ロックが解除される）。
2. R#21 の V58 ビットを 0 にして拡張モードに切り替えるなど、R#20 / R#21 を設定する。

# 第 3 章 コントロールレジスタ詳説

本章では、V9968 のコントロールレジスタ R#0〜R#27 と、VDP コマンド用のレジスタ R#32〜R#58 について、1 レジスタずつ詳しく説明します。

コントロールレジスタはすべて**書き込み専用**です。読み出す手段はありません（Port#1 の読み出しは、次章で説明するステータスレジスタの読み出しになります）。各レジスタへのアクセス方法（直接指定／間接指定）については第 2 章を参照してください。

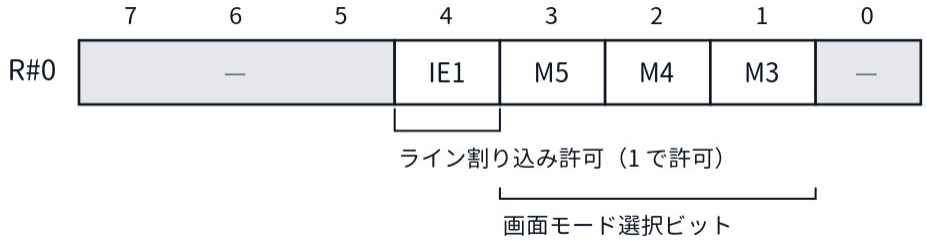
なお、本章を通じて重要になる 2 つの前提があります。

- **V58 ビット（R#21 bit0）**：リセット時は 1 で、この状態が **V9958 互換モード**です。0 にすると**拡張モード**になり、新コマンド（LRMM / LFMC / LFMM）が有効になるほか、VRAM が 256KB モードになります。本章では「互換モード（V58=1）」「拡張モード（V58=0）」と呼び分けます。
- **LockExtReg（Port#4 bit7）**：リセット時は 1 で、この間は **R#20 と R#21 への書き込みが無視されます**。R#20 / R#21 を変更するには、先に Port#4 で LockExtReg を 0 にしてロックを解除してください（Port#4 については第 2 章を参照）。

各レジスタの見出しでは、名称のほかにアクセス属性とリセット値を示します。「未使用」としたビットは、値を書き込んでもレジスタに保持されず、動作に影響しません。

## 3.1 モード設定レジスタ

### R#0 Mode0



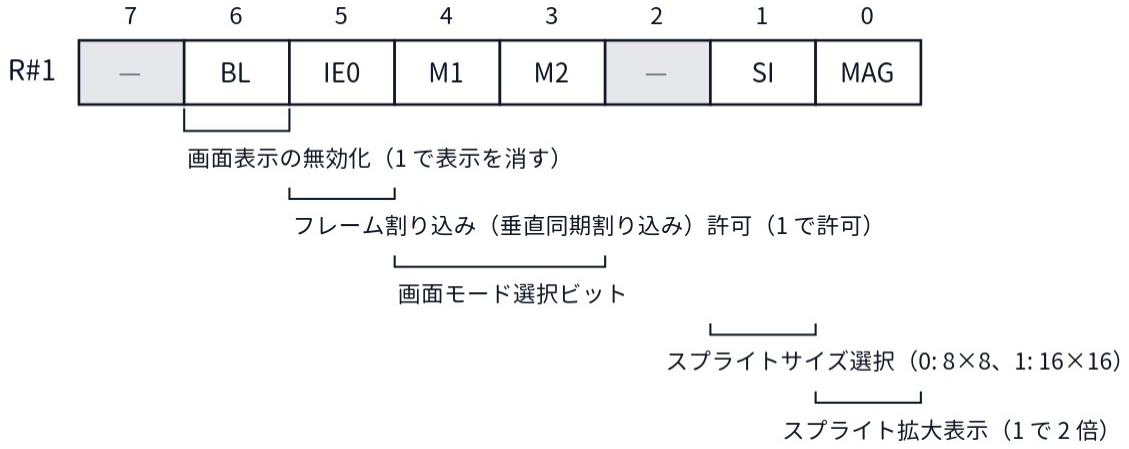

- アクセス: 書き込み専用
- リセット値: M5=M4=M3=0、IE1=0

画面モードは、R#0 の M5〜M3 と R#1 の M2、M1 の計 5 ビットで決まります。組み合わせと画面モードの対応は R#1

の節の表を参照してください。

副作用として、R#0 に書き込むとライン割り込みフラグがクリアされます。

## R#1 Mode1



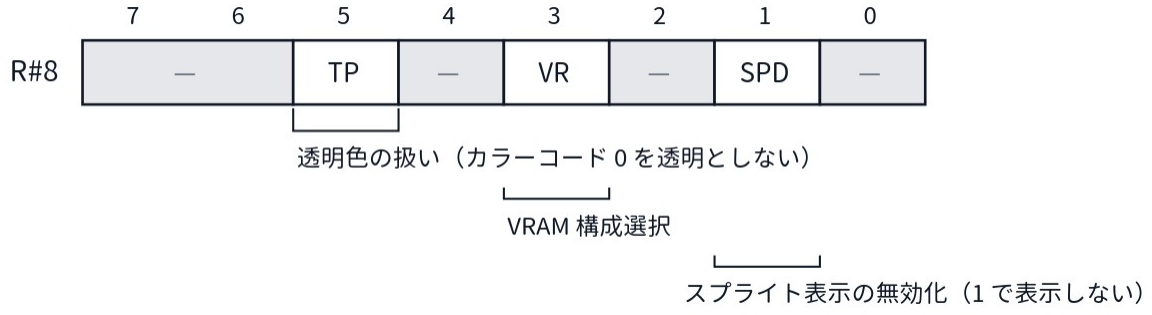

- アクセス: 書き込み専用
- リセット値: BL=0、IE0=0、SI=0、MAG=0

画面モードの選択ビット {M5, M4, M3, M2, M1} と画面モードの対応は次の通りです（各モードの詳細は第 5 章を参照）。

| M5 M4 M3 M2 M1 | 画面モード |
|---|---|
| 00000 | Graphic1（SCREEN1） |
| 00001 / 00101 | Text1（SCREEN0 WIDTH40） ※M3 はドントケア |
| 00010 / 00110 | Multicolor（SCREEN3） ※M3 はドントケア |
| 00100 | Graphic2（SCREEN2） |
| 01000 | Graphic3（SCREEN4） |
| 01100 | Graphic4（SCREEN5） |
| 10000 | Graphic5（SCREEN6） |
| 10100 | Graphic6（SCREEN7） |

| M5 M4 M3 M2 M1 | 画面モード |
|---|---|
| | |
| 11100 | Graphic7（SCREEN8 / 10 / 11 / 12） |
| 01001 | Text2（SCREEN0 WIDTH80） |

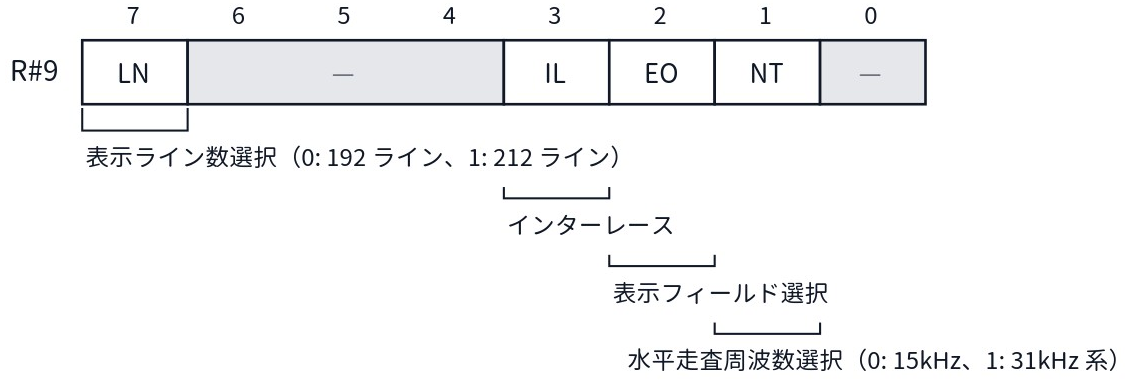
## R#8 Mode2




- アクセス: 書き込み専用
- リセット値: TP=0、VR=1、SPD=0

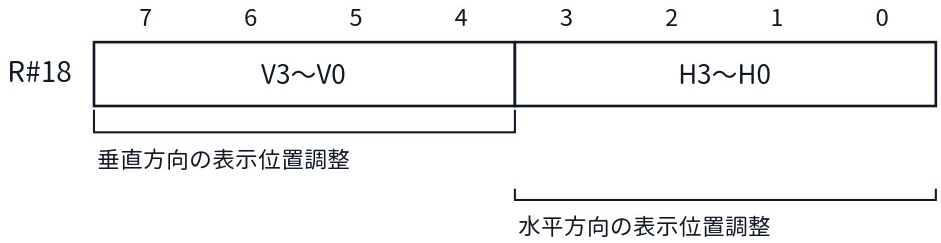
VR は VRAM アクセスに関わります。VR=0 のとき、R#14 で指定する VRAM アドレスの上位（A17～A14）は 0 に固定され、アドレスのインクリメントも 14 ビット範囲でラップします（第 2 章も参照）。

## R#9 Mode3




- アクセス: 書き込み専用
- リセット値: すべて 0

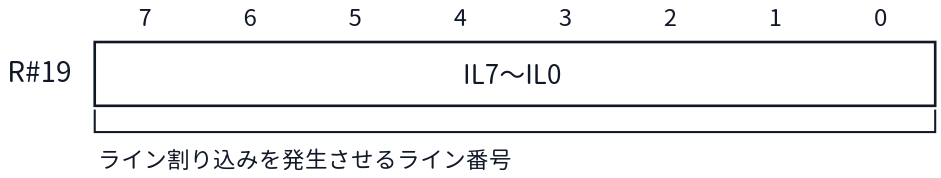
## R#18 Display Adjust




- アクセス: 書き込み専用
- リセット値: 0

画面全体の表示位置を微調整します。値の解釈（符号と方向）は表示系の仕様に従います。

## R#19 Interrupt Line




- アクセス: 書き込み専用
- リセット値: 0

表示ラインがこの番号に達すると、ライン割り込みフラグが立ちます（R#0 の IE1=1 のとき実際に割り込みが発生します）。

副作用として、R#19 に書き込むとライン割り込みフラグがクリアされます（R#0 への書き込みでもクリアされます）。

なお、R#19 のライン番号は通常、垂直スクロール（R#23）のオフセットの影響を受けます。R#20 の ILNS=1 にすると、R#23 に影響されず、画面の一番上を 0 とする絶対位置として解釈されます（R#20 の節、第 10 章参照）。

## R#20 Mode5【V9968 追加】




- アクセス: 書き込み専用（ただし LockExtReg=1 の間は書き込みが無視される）
- リセット値: すべて 0

V9968 で追加されたモードレジスタです。EPAL は Port#2 によるパレット書き込みの形式を切り替えます（第 2 章・第 7 章参照）。CEIE を使うと、VDP コマンドの完了を割り込みで知ることができます。

S16 は、スプライトの 1 ライン表示数を拡張するビットです（初期値 0）。Sprite mode3 では常に 1 ライン 16 枚まで表示されるため、S16 は効果を持ちません。Sprite mode2 では、S16=0 のとき従来互換の 8 枚まで、S16=1 のとき 16 枚まで表示できます。Sprite mode1 では、S16=0 のとき従来互換の 4 枚まで、S16=1 のとき 16 枚まで表示できます（詳細は第 6 章を参照）。

HS は基板側の入力信号（force_highspeed）と OR されるため、HS=0 でも基板側の条件によって高速モードになることがあります。

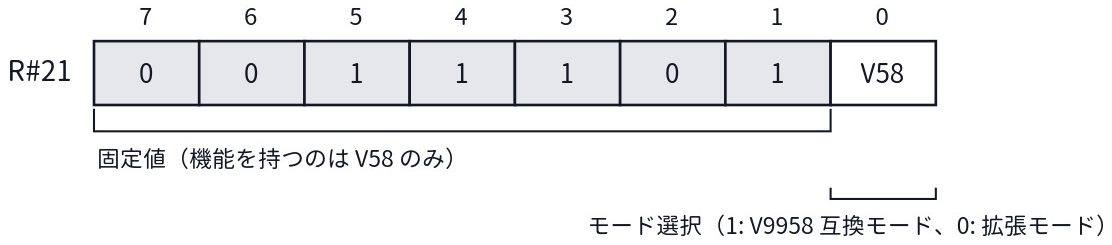
FIL はフラットインターレースモードです。従来のインターレースモードでは、page0 と page1（または page2 と page3）を垂直方向に半画素ずらして交互に表示しますが、2 つのページは別画面のままなので、ページをまたいで線を引いてもつながりません。FIL=1 にすると、page0 と page1 を VRAM 上で上下につなげた形で表示できるようになり、1 枚の縦に長い画面として扱えます。VDP コマンドも、この結合した高解像度の画面として使えます（第 5 章・第 8 章参照）。

ILNS はライン割り込みのライン番号の解釈を切り替えます。通常、ライン割り込み（R#19）のライン番号は垂直スクロール（R#23）のオフセットの影響を受けます。ILNS=1 にすると、R#19 のライン番号は R#23 の値に影響されず、画面の

一番上（R#19=0）を基準とする絶対位置として解釈されます。垂直スクロール（R#23）を動かしながら、画面上の決まった位置で割り込みたい場合に使います（第 10 章参照）。

SVNS はスプライトの表示位置と垂直スクロール（R#23）の関係を切り替えます。通常、スプライトは背景と一緒に R#23 による垂直スクロールの影響を受けます。SVNS=1 にすると、スプライトは R#23 の指定に影響を受けなくなり、スプライトの Y 座標は画面上の絶対位置を指すようになります（Y=0 で画面の一番上に表示されます）。背景だけをスクロールさせてスプライトを画面に固定したい場合に使います（第 6 章参照）。

### R#21 Mode6【V9968 追加】




- アクセス: 書き込み専用（ただし LockExtReg=1 の間は書き込みが無視される）
- リセット値: V58=1

bit7～bit1 は、現在のビットアサインでは上記の固定値です。機能を持つのは bit0 の V58 のみです。

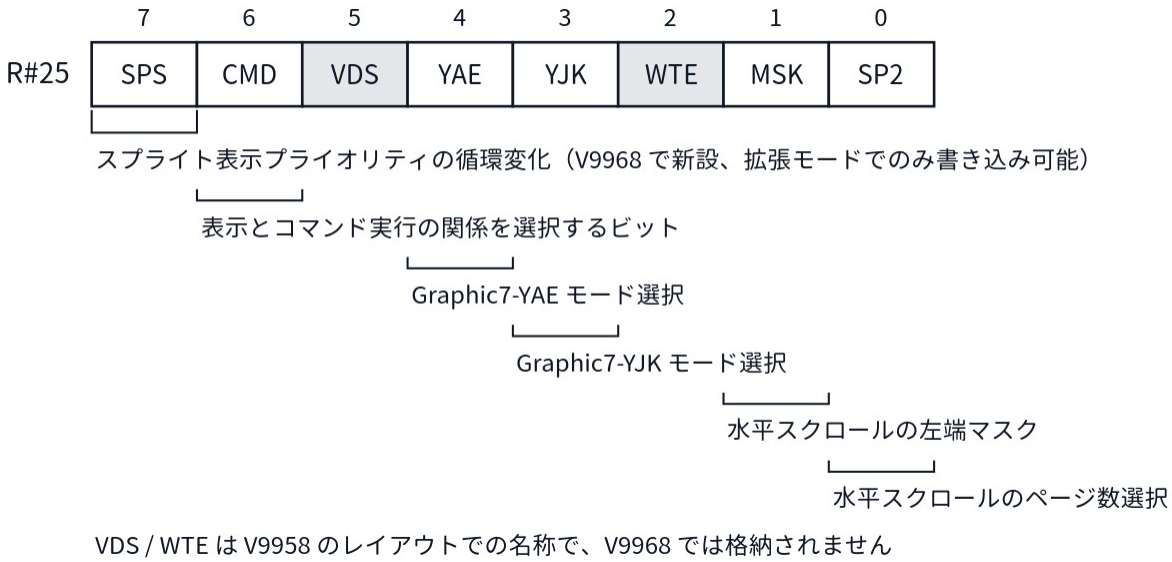
V58 は V9968 の動作モード全体を切り替えるビットです。

- **V58=1（互換モード）**：新コマンド（LRMM / LFMC / LFMM）は無効です。R#2 / R#4 / R#6 / R#10 / R#11 に書いたアドレスの A17 は 0 として保持されます。
- **V58=0（拡張モード）**：新コマンドが有効になり、VRAM は 256KB モードになります。

副作用として、R#21 に 1 を書き込む（互換モードに戻す）と、5 つのテーブルベースアドレス（パターンネーム／パターンジェネレータ／スプライトパターンジェネレータ／カラー／スプライトアトリビュート）の bit17 がすべて 0 にクリアされます。

なお、旧資料に「R#20 の V58」と記した箇所がありますが、V58 は本レジスタ R#21 の bit0 です。

**R#25 Mode4**




- アクセス: 書き込み専用
- リセット値: すべて 0

Graphic7（SCREEN8 相当）で YJK / YAE を使う場合は、画面モード選択（R#0 / R#1）に加えて本レジスタの YJK / YAE ビットで色表現形式を選択します（第 5 章参照）。

水平スクロールの量は R#26 / R#27 で指定します（各レジスタの節を参照）。本レジスタの MSK と SP2 は、その見え方を制御します。

- **MSK**：0 のときは左側 8 ドットはマスクされません（初期値）。1 のときは左側 8 ドットをマスクし、ボーダーカラーを表示します。R#27 の値が 0 の場合は、マスクする必要はありません（GRAPHIC 5、6 では 16 ドットのマスクになります）。
- **SP2**：0 のときは水平方向画面サイズを 1 ページとします（初期値）。スクロールは 1 ページ内で行われ、画面上では左側の隠れた部分が右側に現れます。1 のときは水平方向画面サイズを 2 ページとし、スクロールは 2 ページで行われ、スクロールによって裏のページが現れます。

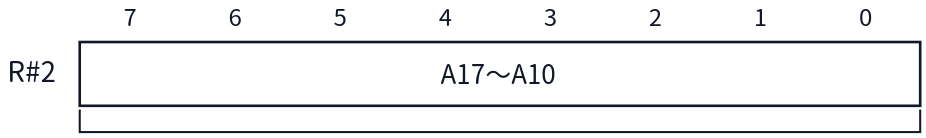
- **SPS**：V9968 で新設されたビットです。拡張モード（V58=0）のときにのみ書き込むことができ、互換モード（V58=1）では書き込みが無視されます。1 にすると、スプライトの表示プライオリティが循環的に変化するようになります。Sprite mode1 / 2 / 3 のすべてで利用できます（第 6 章参照）。

## 3.2 テーブルアドレスレジスタ

以下のレジスタは、VRAM 内の各テーブルのベースアドレスを指定します。いずれも書き込み専用で、リセット値は 0 で

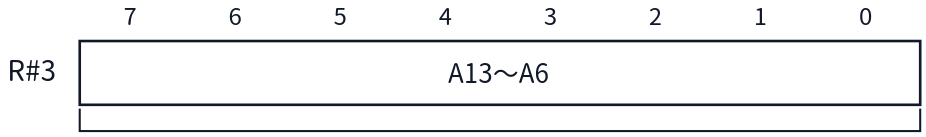
す。互換モード（V58=1）では、A17 を含むレジスタに書いた A17 は 0 として保持されます。

### R#2 Pattern Name Table Address




パターンネームテーブルのベースアドレス [17:10]

### R#3 Color Table Address (L)




カラーテーブルのベースアドレス [13:6]

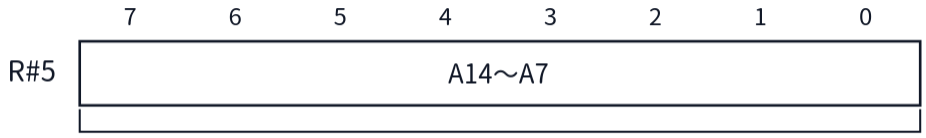
カラーテーブルのベースアドレスは、本レジスタ（下位）と R#10（上位）で構成されます。

### R#4 Pattern Generator Table Address




パターンジェネレータテーブルのベースアドレス [17:11]

### R#5 Sprite Attribute Table Address (L)

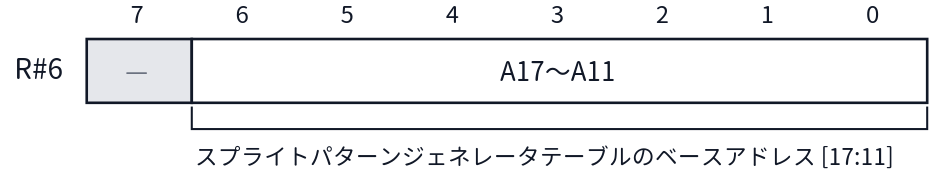



スプライトアトリビュートテーブルのベースアドレス [14:7]

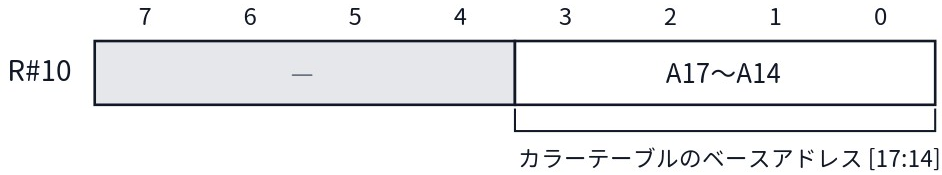
スプライトアトリビュートテーブルのベースアドレスは、本レジスタ（下位）と R#11（上位）で構成されます。

本レジスタの bit1 / bit0（ベースアドレスの [8:7]）は、Sprite mode2 / 3 におけるアトリビュートアドレス生成の際に、プレーン番号の上位ビットとの AND マスクとしても使われます。アトリビュートテーブルを小さく見せたい場合のマスク機能です（第 6 章参照）。

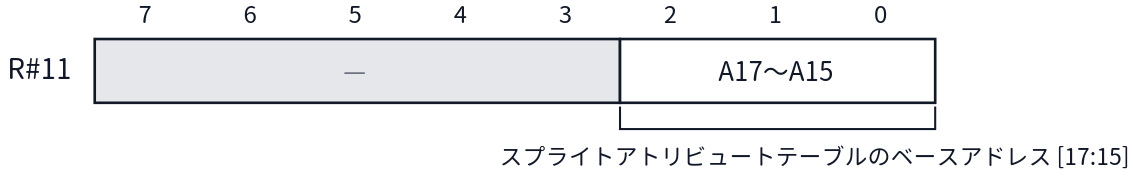
### R#6 Sprite Pattern Generator Table Address




### R#10 Color Table Address (H)

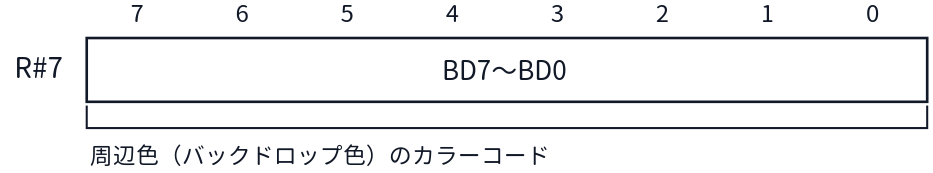



### R#11 Sprite Attribute Table Address (H)




## 3.3 色・表示制御レジスタ

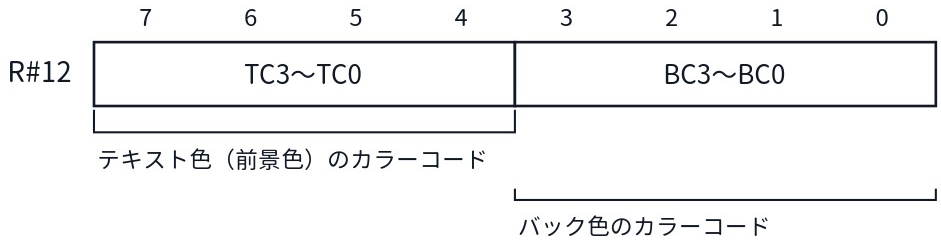
### R#7 Background Color




- アクセス: 書き込み専用
- リセット値: 0

レジスタには書き込んだ 1 バイト全体が保持されます。画面モードによっては上位ニブルだけが使われるなど、表示側での解釈は画面モードごとに異なります（第 5 章参照）。

## R#12 Text Color / Back Color

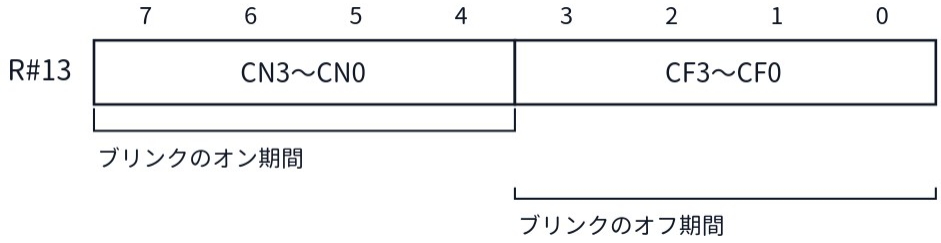



- アクセス: 書き込み専用
- リセット値: 0

ブリンク表示の期間には、R#7 の代わりに本レジスタの色が使われます。点滅の周期は R#13 で設定します。

【要確認】本レジスタのテキスト色フィールドのビット名表記について、資料間で表記が一致していません（RTL コメントの表記に誤記と思われる箇所があります）。正式なビット名をご確認ください。

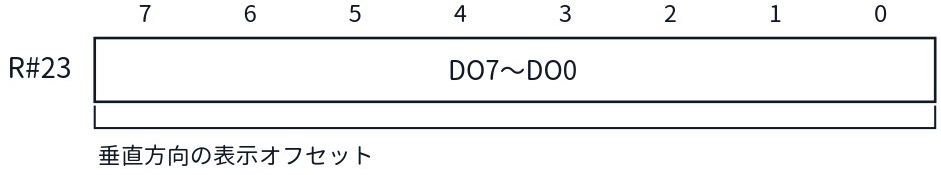
## R#13 Blinking Period




- アクセス: 書き込み専用
- リセット値: 0

R#12 の色を表示する期間（オン）と R#7 の色に戻る期間（オフ）の長さを設定します。

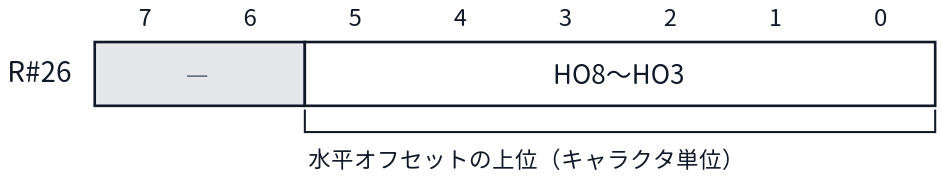
## R#23 Display Offset




- アクセス: 書き込み専用
- リセット値: 0

画面の表示開始位置をライン単位でずらします（垂直スクロール）。

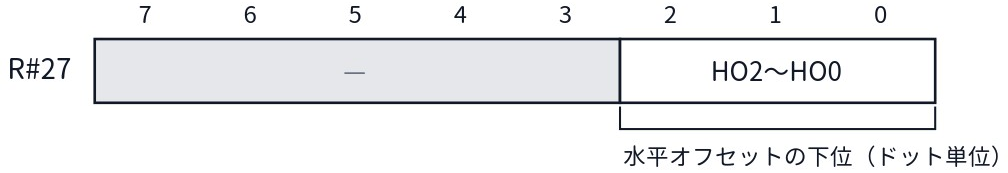
### R#26 Horizontal Offset (By Character)




- アクセス: 書き込み専用
- リセット値: 0

水平スクロールの量は、本レジスタ（上位）と R#27（下位）を合わせた 9 ビットで指定します。

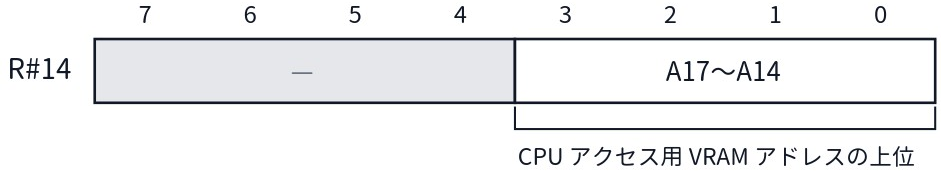
### R#27 Horizontal Offset (By Dot)




- アクセス: 書き込み専用
- リセット値: 0

## 3.4 アクセス制御レジスタ

### R#14 VRAM Access Base Address

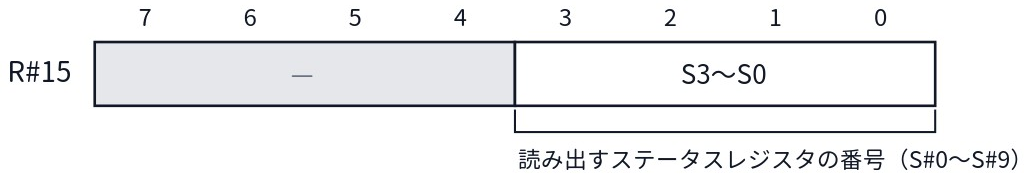



- アクセス: 書き込み専用
- リセット値: 0（VRAM アドレス全体として 0）

Port#0 経由で CPU が VRAM にアクセスする際のアドレス上位を指定します。下位（A13〜A0）は Port#1 のプロトコルで設定します（第 2 章参照）。アドレスレジスタは読み出し・書き込み共通の 18 ビットです。

R#8 の VR ビットが 0 のときは、A17～A14 は 0 に固定されます。また互換モード（V58=1）では A17 がマスクされます。

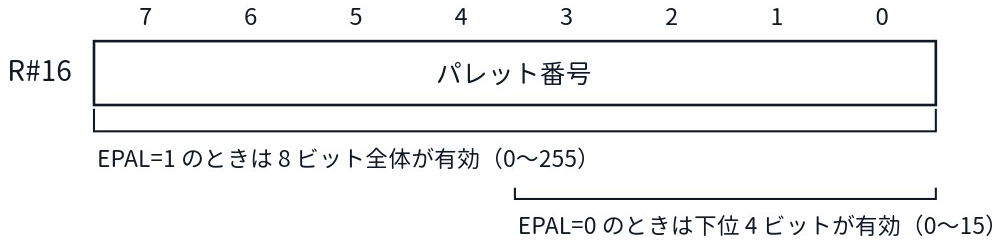
## R#15 Status Register Pointer




- アクセス: 書き込み専用
- リセット値: 0

Port#1 を読むと、本レジスタで指定したステータスレジスタの値が返ります（第 4 章参照）。リセット時は 0（S#0）を指します。S#10～S#15 を指定した場合は FFh が返ります。

## R#16 Color Palette Pointer

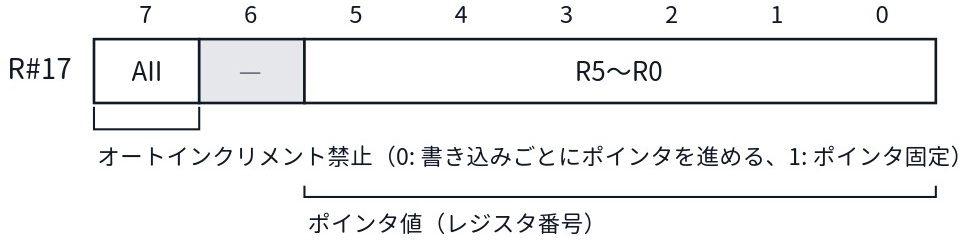



- アクセス: 書き込み専用
- リセット値: パレットアドレス 0

Port#2 でパレットデータを書く際の開始番号を指定します。EPAL（R#20 bit4）が 0 のときは下位 4 ビットだけが有効で、1 のときは 8 ビット全体が有効です。

副作用として、R#16 に書き込むとパレット書き込みのフェーズ（何バイト目を受け付けるか）が先頭に戻ります。パレット書き込みの途中で番号を変更した場合は、必ず本レジスタを書き直してからデータを送ってください。

### R#17 Control Register Pointer




- アクセス: 書き込み専用
- リセット値: ポインタ 0、AII=0

Port#3 経由の間接書き込みで使うポインタです。なお本レジスタ自体は、Port#1 による直接指定でのみ設定できます。Port#3 経由でポインタが R#17 を指していても、その書き込み値は無視されます（ただし AII=0 の場合はポインタだけはインクリメントします）。

### 未実装のレジスタ（R#22、R#24、R#28〜R#31）

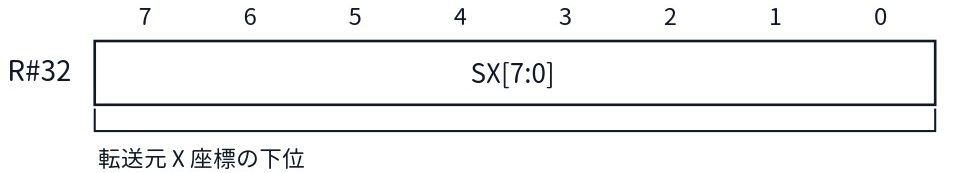
R#22 と R#24 は番号自体が割り当てられておらず、R#28〜R#31 も未実装です。これらの番号への書き込みは無視されます。

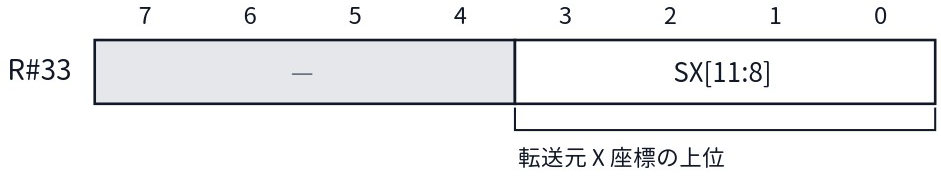
## 3.5 コマンドレジスタ

R#32〜R#58 は VDP コマンドのパラメータと実行指示のためのレジスタです。すべて書き込み専用です。各コマンドの動作そのものについては第 8 章を参照してください。ここではレジスタとしての仕様（ビット構成、リセット値、副作用）を説明します。

座標やドット数は、複数のレジスタにまたがる多ビット値として構成されます。以降、「(L)」は下位バイト、「(H)」は上位ビットを保持するレジスタです。

### R#32 Source X (L) / R#33 Source X (H)



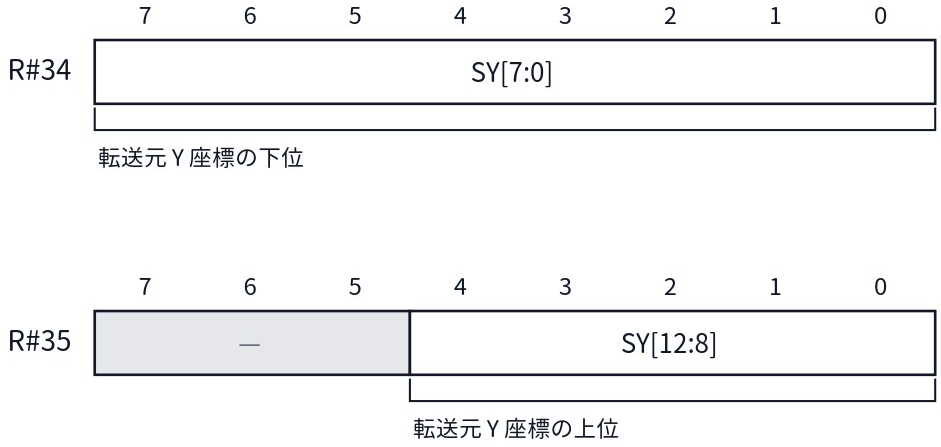



- リセット値: 0

SX は 12 ビットまで保持されますが、LRMM 以外のコマンドでは開始時に SX[11:9] が 0 にクリアされるため、実効は 9 ビットです。

副作用として、R#32 / R#33 への書き込みは、LFMC / LFMM 用フォントアドレスレジスタの [7:0] / [15:8] にも同時に書き込まれます（次項の R#34 も参照）。

## R#34 Source Y (L) / R#35 Source Y (H)

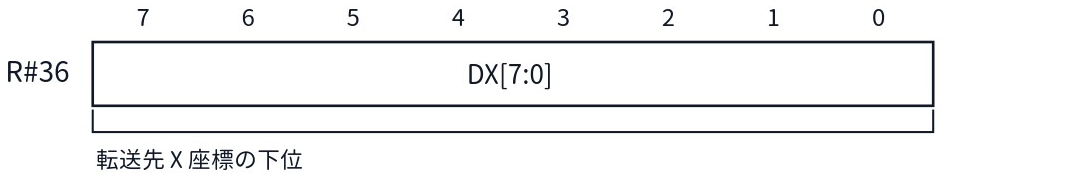


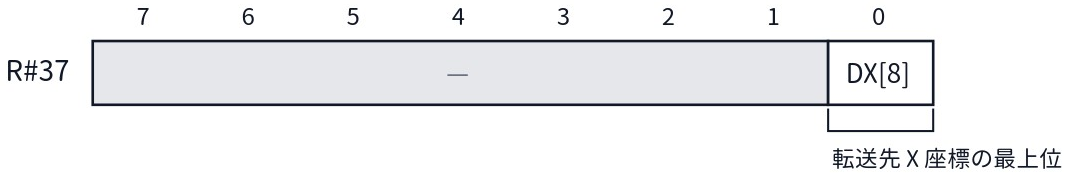

- リセット値: 0

SY は 13 ビット保持されます。コマンド開始時、256KB モードでは SY[12:11] が、128KB モードでは SY[12:10] がクリアされます（実効はそれぞれ 11 ビット / 10 ビット）。

副作用として、R#34 への書き込みはフォントアドレスレジスタの [17:16] にも同時に書き込まれます。つまり R#32〜R#34 の 3 レジスタで、フォントアドレス（18 ビット）が上書きされる点に注意してください。

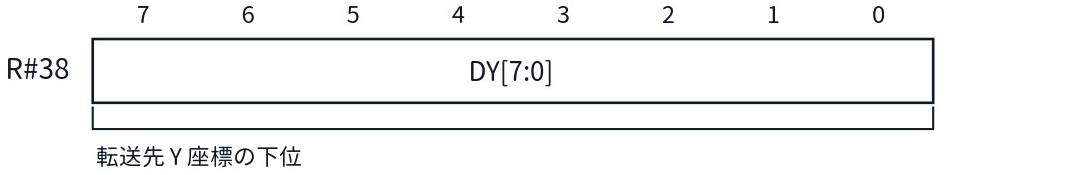
## R#36 Destination X (L) / R#37 Destination X (H)

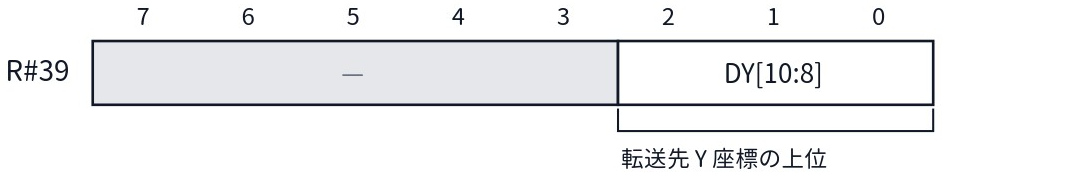



- リセット値: 0

## R#38 Destination Y (L) / R#39 Destination Y (H)

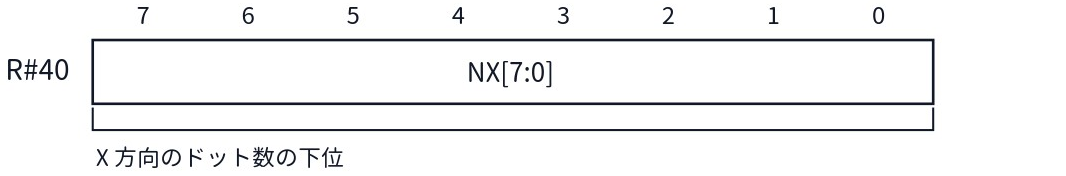


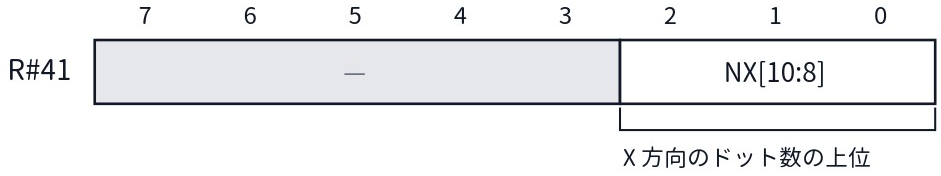




- リセット値: 0

R#39 の有効ビットはモードで異なります。256KB モードでは bit2～bit0 の 3 ビットが有効ですが、128KB モードでは bit1～bit0 のみが有効で、DY[10] は 0 に固定されます。

## R#40 Number Of Dots X (L) / R#41 Number Of Dots X (H)

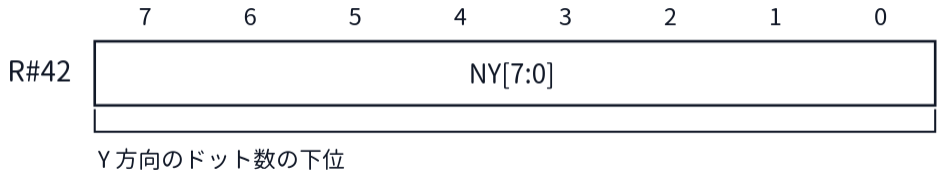


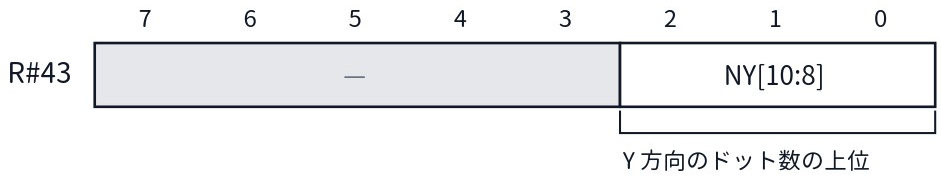



- リセット値: 0

NX は 11 ビット保持されますが、LRMM 以外のコマンドでは開始時に NX[10:9] が 0 にクリアされるため、実効は 9 ビットです。

## R#42 Number Of Dots Y (L) / R#43 Number Of Dots Y (H)

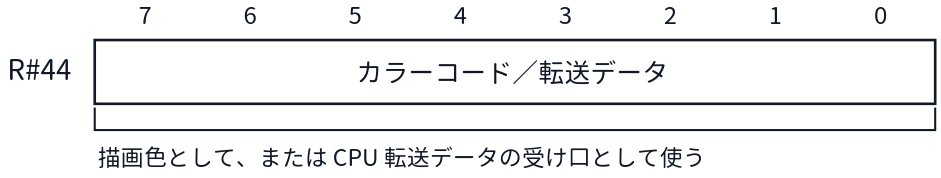



- リセット値: 0

R#43 の有効ビットは R#39 と同じくモード依存です（256KB モードで 3 ビット、128KB モードでは上位が制限されます）。

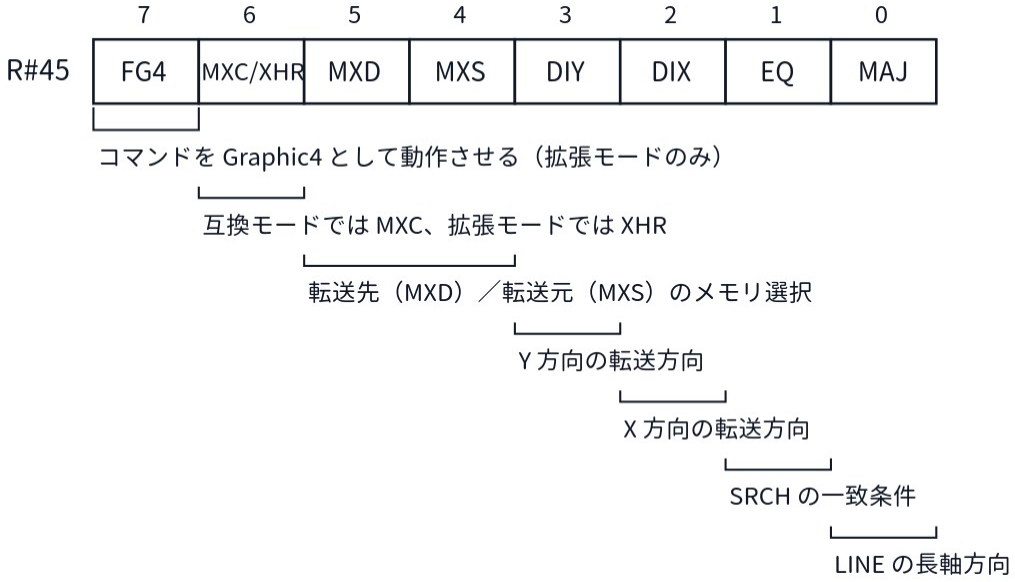
## R#44 Color




- リセット値: 0

HMMV / LMMV / PSET などでは描画色として使われます。HMMC / LMMC / LFMC では、CPU から転送するデータの受け口になります。逆に LMCM の実行中は、読み出された画素の値が本レジスタに格納され、S#7 経由で読み出せます（第 4 章参照）。

## R#45 Argument




- リセット値: すべて 0

| ビット | 名称 | 意味 |
|---|---|---|
| 7 | FG4 | 【V9968 拡張】1 のとき、画面モード指定にかかわらず VDP コマンドが Graphic4（SCREEN 5）として動作する。<br>拡張モードでのみ有効（互換モードでは 0 にマスクされて保持される） |
| 6 | MXC / XHR | 互換モードでは MXC、拡張モードでは XHR（LRMM で画素を縦長として扱う）として取り込まれる |
| 5 | MXD | 転送先メモリ選択（V9968 では拡張 RAM が廃止されており、対象は VRAM のみ） |
| 4 | MXS | 転送元メモリ選択（同上） |
| 3 | DIY | Y 方向の転送方向（0: 下方向、1: 上方向） |
| 2 | DIX | X 方向の転送方向（0: 右方向、1: 左方向） |
| 1 | EQ | SRCH の一致条件（境界色と等しい／等しくない） |
| 0 | MAJ | LINE の長軸方向（0: X 方向、1: Y 方向） |

MXC は互換モードでのみ意味を持ちます。MXC=1 の間、CPU からの VRAM アクセスはマスクされます（書き込みは実行されず、読み出しは FFh を返します）。

bit6 はモードによって意味が変わる二重定義のビットです。互換モード（V58=1）では MXC として、拡張モード（V58=0）では XHR として取り込まれます。また bit7 の FG4 は、拡張モードでないと 0 にマスクされて保持されます。

XHR は拡張モードでのみ意味を持ちます。XHR=1 のとき、LRMM は画素を縦長画素として扱います。Graphic5、Graphic6（SCREEN 6、7）で回転表示を正しいアスペクトで行うための仕掛けです（第 8 章参照）。

MXS / MXD は V9938 / V9958 由来の拡張 RAM 選択ビットですが、V9968 では拡張 RAM が廃止されているため、転送元・転送先として選択できるのは VRAM のみです。

FG4 は拡張モードでのみ意味を持ちます。FG4=1 のとき、VDP コマンドは画面モード指定が何であれ Graphic4（SCREEN 5）として動作します。SCREEN 1〜4 を含め、表示がどの画面モードであってもコマンド側は Graphic4 の座標・色形式で処理されます。この機能は、SpriteMode3 のスプライトパターン（アドレス計算が Graphic4 形式）へ VDP コマンドで描画するために設けられたもので、例えば表示は SCREEN 6、7、8 のまま、スプライトパターンに線を描くといった使い方ができます（第 6 章・第 8 章参照）。なお、表示自体が Graphic4（SCREEN 5）の場合は、もともとコマンドが Graphic4 として動作するため、FG4 の有無で動作は変わりません。

## R#46 Command




- リセット値: コマンドコード 0（STOP）、論理演算 0

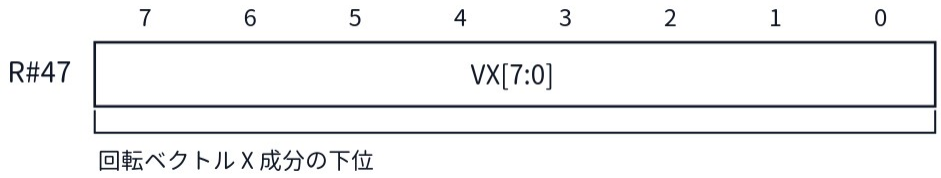
本レジスタに書き込むと、その瞬間にコマンドの実行が開始されます。ほかのパラメータレジスタ（R#32〜R#45、R#47〜R#58）は、R#46 を書く前に設定しておいてください。

コマンドコードの割り当ては次の通りです。LRMM / LFMC / LFMM は V9968 で新設されたコマンドで、拡張モード（V58=0）でのみ有効です。

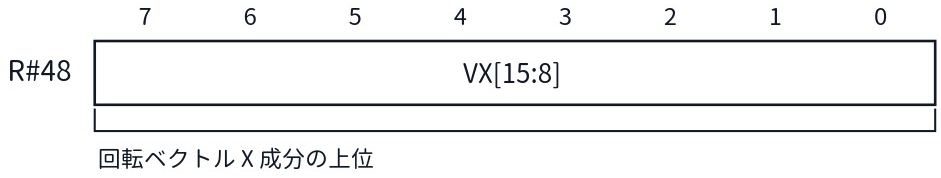
| コード | コマンド | コード | コマンド |
|---|---|---|---|
| 1111 | HMMC | 0111 | LINE |
| 1110 | YMMM | 0110 | SRCH |
| 1101 | HMMM | 0101 | PSET |
| 1100 | HMMV | 0100 | POINT |
| 1011 | LMMC | 0011 | LRMM【V9968 新設】 |
| 1010 | LMCM | 0010 | LFMC【V9968 新設】 |
| 1001 | LMMM | 0001 | LFMM【V9968 新設】 |
| 1000 | LMMV | 0000 | STOP |

なお、旧資料に新コマンドの有効条件として「ECOM ビット」という記述があります。ECOM は、かつて新コマンドの有効化のために設けられたビットでしたが、現在の設計では V58 に統合されており、独立した ECOM ビットは存在しません。有効条件は拡張モード（V58=0）であることです。

## R#47 Rotation Vector X (L) / R#48 Rotation Vector X (H)【V9968 追加】




回転ベクトル X 成分の下位




回転ベクトル X 成分の上位

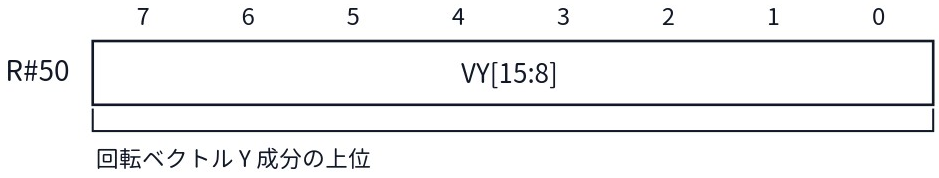
- リセット値: 0

LRMM コマンドで使う回転ベクトルの X 成分です。VX は符号付き 16 ビットで、LRMM の実行時には小数部 8 ビットの固定小数点数（8.8 形式）として符号拡張され、座標に加算されます。

## R#49 Rotation Vector Y (L) / R#50 Rotation Vector Y (H)【V9968 追加】




回転ベクトル Y 成分の下位

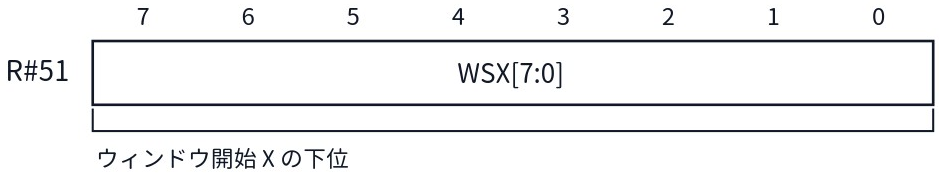



回転ベクトル Y 成分の上位

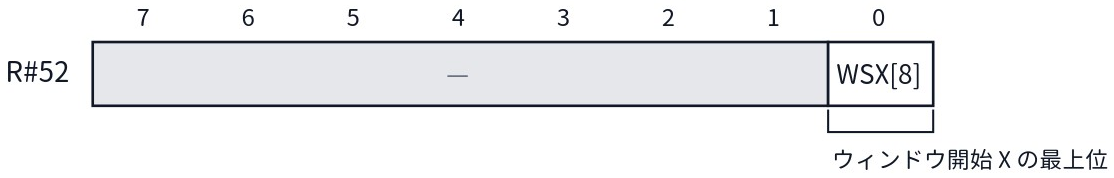
- リセット値: 0

回転ベクトルの Y 成分です。形式は VX と同じです。

## R#51 Window Start X (L) / R#52 Window Start X (H)【V9968 追加】



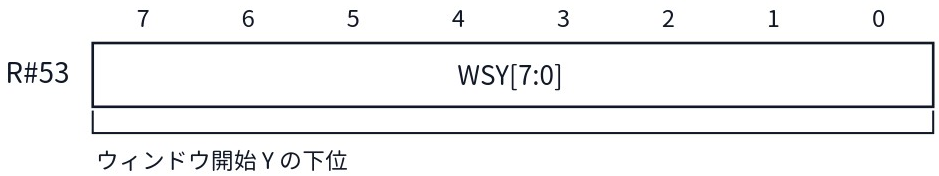

ウィンドウ開始 X の下位



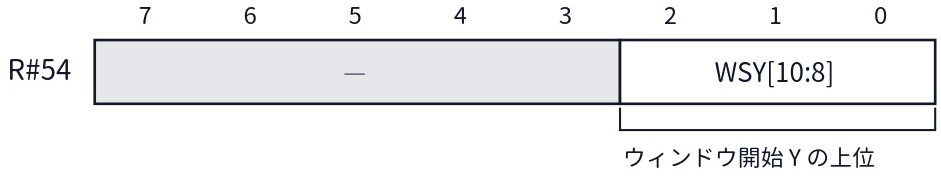

ウィンドウ開始 X の最上位

- リセット値: 0

## R#53 Window Start Y (L) / R#54 Window Start Y (H)【V9968 追加】

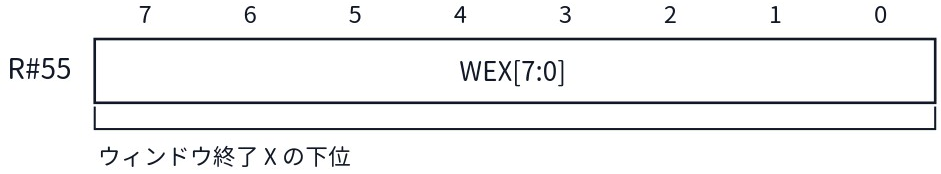


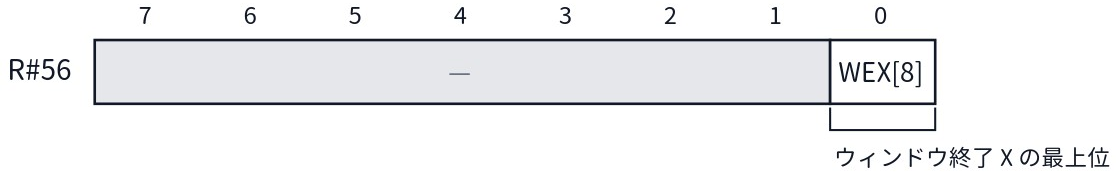

ウィンドウ開始 Y の下位




- リセット値: 0

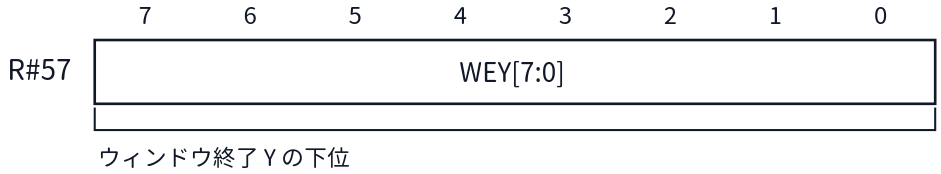
## R#55 Window End X (L) / R#56 Window End X (H)【V9968 追加】

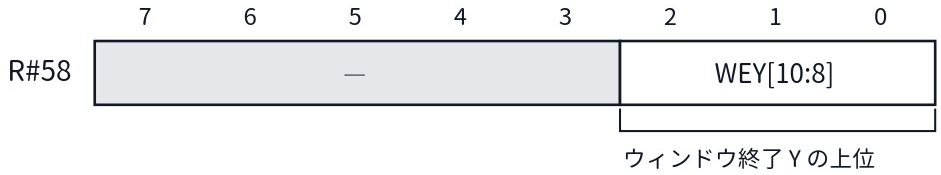



- リセット値: WEX 全体で 1FFh

## R#57 Window End Y (L) / R#58 Window End Y (H)【V9968 追加】




- リセット値: WEY 全体で 7FFh

ウィンドウレジスタ（R#51～R#58）は、LRMM の転送元参照ウィンドウの開始点と終了点を指定します。これらのレジスタが効くのは LRMM のみで、他のコマンドには影響しません。ウィンドウは開始点・終了点の両端を含む範囲であり、参照点が (WSX, WSY)～(WEX, WEY) の内側にあるドットは転送元から読み取られ、外側に出たドットは R#44（CLR）の

色で置き換えられます。終了点のリセット値が最大値（WEX=1FFh、WEY=7FFh）になっているため、リセット直後は全域がウィンドウの内側です。

## 未実装のレジスタ（R#59～R#63）

R#59～R#63 は未実装です。これらの番号への書き込みは無視されます。

# 第 4 章 ステータスレジスタ詳説

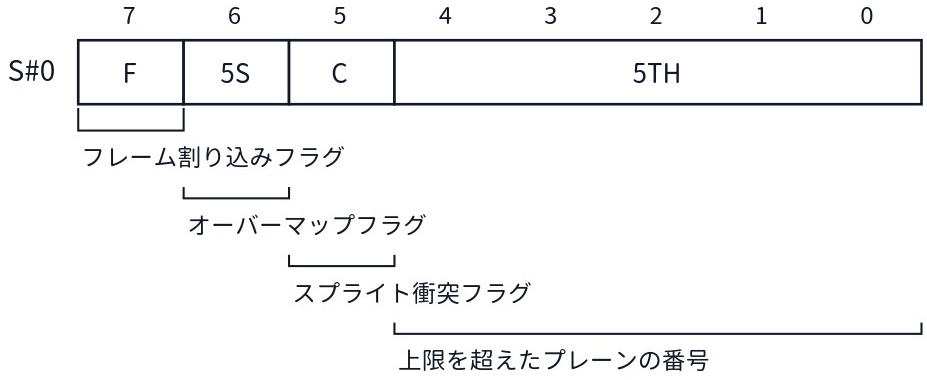
本章では、V9968 のステータスレジスタ S#0〜S#9 について、1 レジスタずつ詳しく説明します。

ステータスレジスタはすべて**読み出し専用**です。読み出しは、R#15（Status Register Pointer）に番号を指定してから Port#1 を読む、という手順で行います（第 2 章・第 3 章参照）。リセット時のポインタは 0（S#0）を指します。

レジスタによっては、**読み出すとフラグがクリアされる**ものがあります。各レジスタの節で「読み出し時の副作用」として明記しますので、ポーリングで状態を監視する場合は注意してください。また、本章の各表で「固定」としたビットは、常にその値で読み出されます。

なお、S#10〜S#15 を指定して読み出した場合は、常に FFh が返ります（最終節参照）。

## S#0



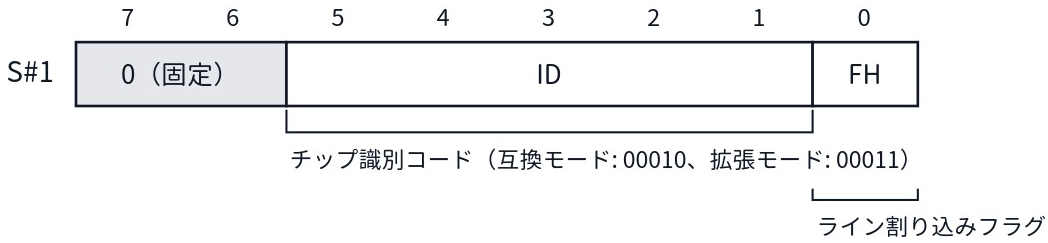

読み出し時の副作用：**F、C、5S がクリアされます**（5TH の値は保持されます）。

5S と 5TH は、1 ラインに表示できるスプライト数の上限を超えたことを知らせる仕組みです。上限と、5S が立つタイミングはスプライトモードによって異なります。

| 条件 | 1 ラインの上限 | 5S が立つ枚数 |
|---|---|---|
| Sprite mode1 | 4 枚 | 5 枚目 |
| Sprite mode2 | 8 枚 | 9 枚目 |
| Sprite mode3、または S16=1 のときの mode1 / 2 | 16 枚 | 17 枚目 |

5TH には上限を超えたプレーンの番号が入ります。5TH は 5 ビットのため、プレーン番号が 32 以上の場合は 1Fh が入ります。

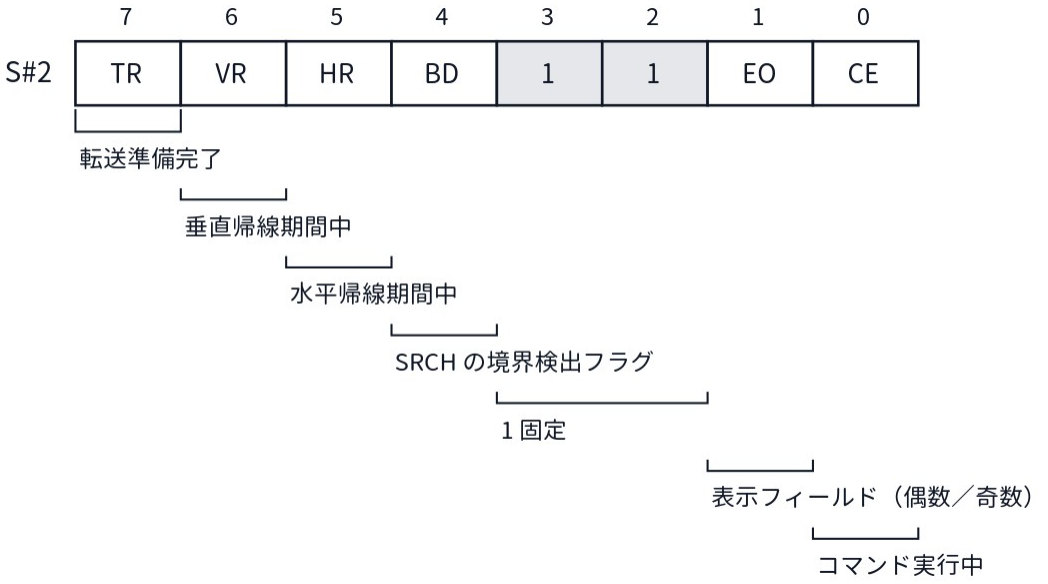
### S#1




ID の値は、互換モード（V58=1）では 00010、拡張モード（V58=0）では 00011 です。ソフトウェアが V9958 相当として動作しているか、V9968 の拡張モードで動作しているかを判別する手がかりになります。

読み出し時の副作用：**FH がクリアされます**。

### S#2

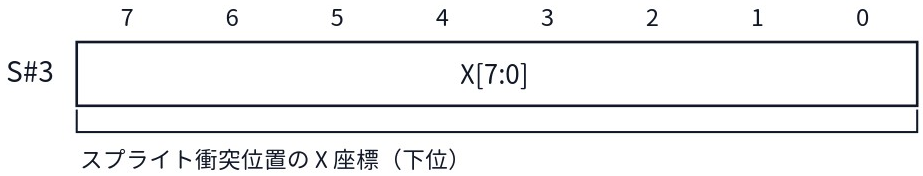



読み出し時の副作用：ありません。

CE は、VDP コマンドが実行中の間 1 になります。コマンドの完了待ちは CE のポーリングでも行えますし、R#20 の CEIE を使えば割り込みで知ることもできます。

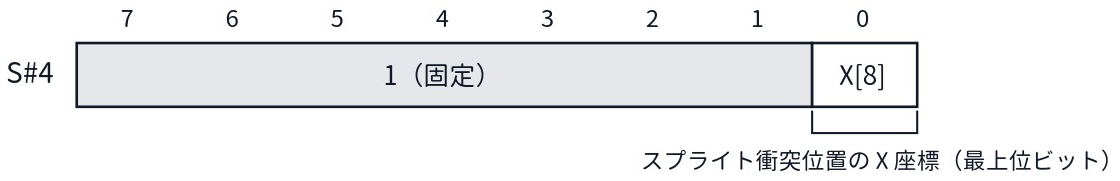
bit3 と bit2 は常に 1 で読み出されます。旧資料にはこの 2 ビットの記載がありませんでしたが、未使用ビットとして 0 になるとは限りませんので、ソフトウェアでは値を判定に使わないでください。

### S#3




スプライト同士が衝突した位置の X 座標の下位 8 ビットです。上位ビットは S#4 にあります。座標のクリアについては S#5 の節を参照してください。

### S#4



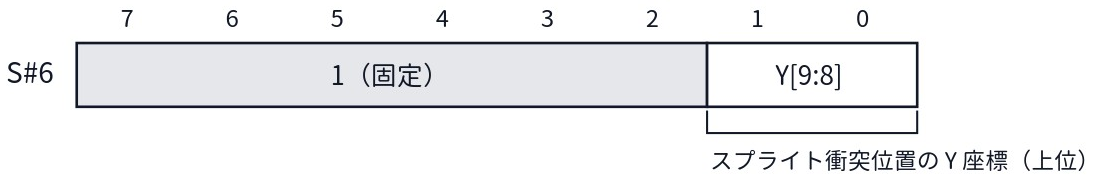

### S#5



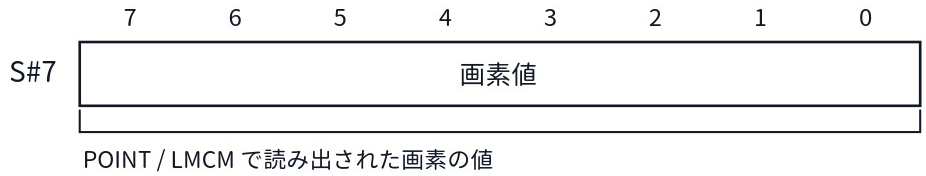

スプライト同士が衝突した位置の Y 座標の下位 8 ビットです。上位ビットは S#6 にあります。

読み出し時の副作用：**衝突座標（X / Y の両方）がクリアされます**。衝突座標を読み出す場合は、先に S#3 / S#4 を読んでから、最後に S#5 を読んでください。

### S#6



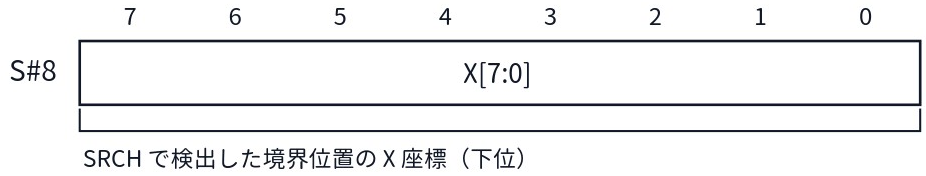
### S#7




POINT コマンドや LMCM コマンドで読み出された画素値は、R#44 に格納されたうえで、本レジスタ S#7 経由で読み出します。

本レジスタの読み出しは、コマンドエンジン側の転送制御（TR の制御）に「読み出し完了」の合図として使われます。LMCM の連続読み出しでは、S#7 を読むことが次の画素への進行条件になります。

### S#8




上位ビットは S#9 にあります。BD フラグのクリアについては S#9 の節を参照してください。

### S#9




読み出し時の副作用：**BD（S#2 bit4）がクリアされます。**

### S#10〜S#15

S#10〜S#15 は実装されていません。R#15 でこれらの番号を指定して Port#1 を読むと、常に **FFh** が返ります。

## 割り込み出力との関係

最後に、ステータスフラグと割り込み出力の関係をまとめておきます。V9968 の割り込み出力は、次の 3 条件の OR を

取ったもの（負論理）です。

- ライン割り込みフラグ（S#1 の FH）が立っている かつ R#0 の IE1=1
- フレーム割り込みフラグ（S#0 の F）が立っている かつ R#1 の IE0=1
- コマンド完了の要因がある かつ R#20 の CEIE=1

各要因の発生状態は Port#4 の読み出しでも確認できます（第 2 章参照）。コマンド完了割り込みを使う場合は、CEIE が R#20 にあるため、Port#4 で LockExtReg を解除してから R#20 を設定する必要があります。

# 第 5 章 スクリーンモード

## 5.1 スクリーンモードについて

V9968 のスクリーンモードは、V9958 と同等であり、新しいスクリーンモードは追加されていません。TMS9918 から受け継がれてきたテキストモードやパターン（キャラクタ）方式のモードと、V9938 で追加されたビットマップモード、さらに V9958 で追加された YJK 方式のモードまで、全 12 モードがそのまま利用できます。

本章では、モードの一覧と選択方法、それにモードごとに VRAM 上のテーブル配置を説明します。各モードの説明は、従来の V9938／V9958 の動作を前提としていますが、従来の資料（V9938／V9958 テクニカルデータブック）では詳しく触れられていなかった細部の動作についても、V9968 の動作が確定しているものは本書に明記します。また、V9958 では未定義とされていた挙動のうち、実際に使えるものは V9968 の正式な機能として格上げして説明します。そのような項目については「**9958 Compatible**」と付記します。

なお本章で「パレット」と呼ぶのは、第 7 章で説明する 256 エントリのパレット（各色 RGB 各 5 ビット）です。従来モードで指定する色番号は、このパレットのエントリを指します。リセット直後のパレット初期値については第 7 章を参照してください。

## 5.2 モードの選択方法

スクリーンモードは、コントロールレジスタ R#0 のビット 3〜1（M5、M4、M3）と、R#1 のビット 4（M1）、ビット 3（M2）の、合計 5 ビットの組み合わせで選択します。組み合わせとモードの対応は次のとおりです。

| モード | 略称 | M5 | M4 | M3 | M2 | M1 | 対応する BASIC の SCREEN | 画面 | 色 | スプライト |
|---|---|---|---|---|---|---|---|---|---|---|
| Graphic1 | G1 | 0 | 0 | 0 | 0 | 0 | SCREEN 1 | 32×24 パターン | パレット 16 色 | モード 1 |
| Text1 | T1 | 0 | 0 | 0 | 0 | 1 | SCREEN 0（WIDTH 40） | 40×24 文字（6×8 ドット） | 2 色 | なし |
| Multicolor | MC | 0 | 0 | 0 | 1 | 0 | SCREEN 3 | 64×48 ブロック | パレット 16 色 | モード |

| モード | 略称 | M5 | M4 | M3 | M2 | M1 | 対応する BASIC の SCREEN | 画面 | 色 | スプライト |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | 1 |
| Graphic2 | G2 | 0 | 0 | 1 | 0 | 0 | SCREEN 2 | 256×192ドット | パレット 16 色 | モード 1 |
| Graphic3 | G3 | 0 | 1 | 0 | 0 | 0 | SCREEN 4 | 256×192ドット | パレット 16 色 | モード 2 |
| Graphic4 | G4 | 0 | 1 | 1 | 0 | 0 | SCREEN 5 | 256×212ドット | パレット 16 色 | モード 2 |
| Text2 | T2 | 0 | 1 | 0 | 0 | 1 | SCREEN 0（WIDTH 80） | 80×24 文字（6×8 ドット） | 2 色＋ブリンク色 | なし |
| Graphic5 | G5 | 1 | 0 | 0 | 0 | 0 | SCREEN 6 | 512×212ドット | パレット 4 色 | モード 2 |
| Graphic6 | G6 | 1 | 0 | 1 | 0 | 0 | SCREEN 7 | 512×212ドット | パレット 16 色 | モード 2 |
| Graphic7 | G7 | 1 | 1 | 1 | 0 | 0 | SCREEN 8 | 256×212ドット | 256 色（直接指定） | モード 2 |
| Graphic7-YAE | G7-YAE | 1 | 1 | 1 | 0 | 0 | SCREEN 10／11 | 256×212ドット | YJK＋パレット 16 色 | モード 2 |
| Graphic7-YJK | G7-YJK | 1 | 1 | 1 | 0 | 0 | SCREEN 12 | 256×212ドット | YJK（約 19,268 色） | モード 2 |

Graphic7 の変種である YAE／YJK は、モードビットではなく R#25 のビット 3（YJK）とビット 4（YAE）で切り替えます（5.13、5.14 節）。

**9958 Compatible（Text1／Multicolor の M3 ビット）** Text1 と Multicolor では、M3（R#0 ビット 1）はドントケアです。すなわち Text1 は {M5,M4,M3,M2,M1} = 00001 でも 00101 でも、Multicolor は

00010 でも 00110 でも選択できます。V9958 ではこの挙動は保証されていませんでしたが、V9968 では正式な仕様とします。

このほかに、モードと組み合わせて使う設定として次があります。詳細は第 3 章を参照してください。

- 表示ライン数：R#9 のビット 7（LN）で、ビットマップモードの縦表示を 192 ライン（LN=0）／212 ライン（LN=1）に、Text2 の行数を 24 行／26.5 行に切り替えます。本章のメモリ量は、特に断らない限り 212 ライン（Text2 は 24 行）の場合を示します。
- 表示位置の調整：水平スクロール（R#26、R#27）や表示開始ライン（R#23）は、画面の表示位置を変えるもので、VRAM 上のテーブル配置は変わりません。
- V58 ビット（R#21 ビット 0）：リセット直後は 1 で、V9958 互換モードとして動作します。0 にすると拡張モードとなり、256KB の VRAM 全体が使えます。本章のテーブル配置そのものは両モードで共通ですが、ベースアドレスに使えるビットが変わります（5.15 節）。

## テーブルのベースアドレスレジスタ

各テーブルの先頭アドレスは、次のレジスタで指定します（スプライト関係は第 6 章）。レジスタに書き込めるのはアドレスの上位ビットだけで、下位ビットは 0 とみなされます。そのため、テーブルを置ける位置はレジスタごとに決まった境界に限られます。

| レジスタ | 対象テーブル | 与えるアドレスビット |
|---|---|---|
| R#2 | パターンネームテーブル／ビットマップ | A17〜A10 |
| R#3 | カラーテーブル／ブリンクテーブル（下位） | A13〜A6 |
| R#10 | カラーテーブル／ブリンクテーブル（上位） | A17〜A14 |
| R#4 | パターンジェネレータテーブル | A17〜A11 |
| R#5、R#11 | スプライトアトリビュートテーブル（第 6 章） | A17〜A7 相当 |
| R#6 | スプライトパターンジェネレータテーブル（第 6 章） | A17〜A11 |

カラーテーブルとブリンクテーブルの先頭アドレスは、R#3 と R#10 を組み合わせて A17〜A6（64 バイト単位）で指定します。

## 5.3 Text1 モード

40×24 文字のテキストモードです。文字は 6×8ドットで、256 種類の文字を表示できます。このモードではスプライトは使えません。MSX-BASIC では SCREEN 0（WIDTH 40）に相当します。

| 項目 | 内容 |
|---|---|
| 画面 | 40 文字×24 行 |
| 文字 | 256 種、6×8ドット、1 文字ごとにフォント定義可能 |
| 色 | 全文字共通の 2 色（前景色／背景色） |
| 必要メモリ | パターンジェネレータテーブル 2,048 バイト、パターンネームテーブル 960 バイト |

### パターンネームテーブル（R#2）

画面上の 1 文字に 1 バイトが対応し、値がそのまま文字コード（0〜255）になります。アドレスは、ベースアドレスに「行×40＋列」を加えたものになります。

先頭アドレスは R#2 で 1KB 単位に指定できます。

| R#2 の値 | 00h | 01h | 02h | … | 7Fh | 80h | … | FEh | FFh |
|---|---|---|---|---|---|---|---|---|---|
| 先頭アドレス | 00000h | 00400h | 00800h | … | 1FC00h | 20000h | … | 3F800h | 3FC00h |

先頭アドレス = R#2 の値 × 400h です。MSX-BASIC の初期値は 00000h です。

### パターンジェネレータテーブル（R#4）

文字フォントを定義するテーブルで、1 文字あたり 8 バイト、先頭から文字コード順に並びます。各バイトのうち画面に表示されるのは上位 6 ビットで、下位 2 ビットは表示されません（文字幅 6 ドットのため）。

アドレス = ベースアドレス ＋ 文字コード×8 ＋ ライン（0〜7）

先頭アドレスは R#4 で 2KB 単位に指定できます。

| R#4 の値 | 00h | 01h | 02h | … | 7Eh | 7Fh |
|---|---|---|---|---|---|---|
| 先頭アドレス | 00000h | 00800h | 01000h | … | | |

3F000h | 3F800h |

先頭アドレス = R#4 の値 × 800h です。MSX-BASIC の初期値は 00800h です。

### 色の指定

色は R#7 で指定し、全文字に共通です。上位 4 ビットが前景色（フォントパターンの "1" の部分）、下位 4 ビットが背景色（"0" の部分）を表し、それぞれパレットの色番号です。周辺色は背景色と同じになります。

### VRAM アドレスの折り返し

Text1 モード（および Multicolor、Graphic1、Graphic2）で CPU から VRAM にアクセスするときのアドレスの折り返しには注意が必要です。アドレスレジスタの下位 14 ビットが 3FFFh の次は 0000h に戻ります（ビット 13 からビット 14 への桁上がりはブロックされます。これは TMS9918 との互換性を保つための仕様です）。一方、R#14 で上位アドレスを指定すれば 04000h〜3FFFFh の範囲にもアクセスできますが、その場合も 7FFFh の次は 4000h に戻る折り返しが残ります。VRAM アクセスの詳細は第 11 章を参照してください。

## 5.4 Text2 モード

80×24 文字のテキストモードです。文字は 6×8 ドットで、文字ごとにブリンク（点滅）させることができます。このモードでもスプライトは使えません。MSX-BASIC では SCREEN 0（WIDTH 80）に相当します。

| 項目 | 内容 |
|---|---|
| 画面 | 80 文字×24 行（R#9 の LN=1 で 26.5 行） |
| 文字 | 256 種、6×8 ドット、1 文字ごとにフォント定義可能、文字ごとにブリンク可能 |
| 色 | 2 色（前景色／背景色）＋ブリンク時の 2 色 |
| 必要メモリ | パターンジェネレータテーブル 2,048 バイト、パターンネームテーブル 1,920 バイト、ブリンクテーブル 240 バイト（26.5 行時はそれぞれ 2,160 バイト、270 バイト） |

### パターンネームテーブル（R#2）

Text1 と同じく、1 文字に 1 バイト（文字コード）が対応します。アドレスは、ベースアドレスに「行×80＋列」を加えたものになります。先頭アドレスは R#2 で 4KB 単位に指定できます（アドレス = R#2 の値 × 400h のうち、実際に効くのは

A17〜A12 です）。

### パターンジェネレータテーブル（R#4）

Text1 とまったく同じ構造と機能を持ちます。5.3 節を参照してください。

### ブリンクテーブル（R#3、R#10）

どの文字をブリンクさせるかを記録するテーブルで、1 ビットが画面上の 1 文字に対応します。対応するビットが "1" の文字にブリンク属性が与えられます。ビットと文字の対応は、パターンネームテーブルの先頭から順に、バイトの上位ビットから割り当てられます。

アドレス = ベースアドレス ＋（行×80＋列）÷ 8

先頭アドレスは R#3 と R#10 で 512 バイト単位に指定できます。

### 色とブリンクの指定

通常の色は Text1 と同じく R#7（上位 4 ビット＝前景色、下位 4 ビット＝背景色）で指定します。ブリンク属性を与えられた文字は、R#7 の色と R#12 で指定するブリンク色（同じく上位＝前景、下位＝背景）とを交互に繰り返して表示されます。繰り返しの周期は R#13 で設定し、上位 4 ビットが R#7 の色を表示する時間、下位 4 ビットがブリンク色を表示する時間を、それぞれ 1/6 秒単位で表します。どちらかを 0 にすると、その期間は表示されません（詳細は第 3 章）。

## 5.5 Multicolor モード

1 文字分の領域（8×8 ドット）を 4×4 ドットのブロック 4 個に分割し、ブロックごとに色を指定する低解像度グラフィックモードです。画面は 64×48 ブロックで構成されます。MSX-BASIC では SCREEN 3 に相当し、スプライトはモード 1 が使えます。

| 項目 | 内容 |
|---|---|
| 画面 | 64×48 ブロック（ブロックは 4×4 ドット） |
| 色 | パレット 16 色、ブロックごとに指定可能 |
| 必要メモリ | パターンジェネレータテーブル（色データ）2,048 バイト、パターンネームテーブル 768 バイト |

### パターンネームテーブル（R#2）

画面上のパターン位置（32×24）に 1 バイトずつ対応する、768 バイトのテーブルです。先頭アドレスは R#2 で 1KB 単位に指定できます。

### パターンジェネレータテーブル（R#4）

このモードでは、パターンジェネレータテーブルがそのまま色データを保持します。1 パターンは 8 バイトで、そのうち 2 バイトずつが上下左右 4 ブロック分の色を表します。各バイトの上位 4 ビットが左ブロック、下位 4 ビットが右ブロックの色（パレット番号）です。

表示されるバイトは、表示ラインによって決まります。パターン内のバイト位置は「表示ラインを 32 ライン周期で見たときの、4 ラインごとのグループ番号」（0〜7）です。つまり、同じパターン名でも、画面の縦位置（32 ライン周期）によって異なる 2 バイト組が表示されます。

アドレス = ベースアドレス ＋ パターン番号×8 ＋ バイト位置（0〜7）

先頭アドレスは R#4 で 2KB 単位に指定できます。周辺色は R#7 の下位 4 ビットで指定します。

## 5.6 Graphic1 モード

8×8ドットのパターンを 32×24 個並べる、キャラクタグラフィックモードです。256 種のパターンを表示でき、色は 8 パターンごとに指定します。MSX-BASIC では SCREEN 1 に相当し、スプライトはモード 1 が使えます。

| 項目 | 内容 |
|---|---|
| 画面 | 32×24 パターン（256×192ドット） |
| パターン | 256 種、8×8ドット、1 パターンごとに図形定義可能 |
| 色 | パレット 16 色、8 パターンごとに指定可能 |
| 必要メモリ | パターンジェネレータテーブル 2,048 バイト、カラーテーブル 32 バイト、パターンネームテーブル 768 バイト |

### パターンネームテーブル（R#2）

画面上のパターン位置と 1 対 1 に対応する 768 バイトのテーブルで、値がパターン番号（0〜255）になります。先頭ア

ドレスは R#2 で 1KB 単位に指定できます。

### パターンジェネレータテーブル（R#4）

1 パターンあたり 8 バイトで、各バイトがパターンの横 1 ラインに対応し、"1" のドットが前景色、"0" のドットが背景色で表示されます。

アドレス = ベースアドレス ＋ パターン番号×8 ＋ ライン（0〜7）

先頭アドレスは R#4 で 2KB 単位に指定できます。

### カラーテーブル（R#3、R#10）

8 パターンごとに 1 バイトを使い、上位 4 ビットがそのパターン群の前景色、下位 4 ビットが背景色を指定します。

アドレス = ベースアドレス ＋ パターン番号 ÷ 8

先頭アドレスは R#3 と R#10 で 64 バイト単位に指定できます。周辺色は R#7 の下位 4 ビットで指定します。

## 5.7 Graphic2 モード

Graphic1 のパターンジェネレータテーブルとカラーテーブルを、画面の上段・中段・下段の 3 組に拡張したモードです。各段で別々の 256 パターンを使えるため、実質 768 種のパターンを表示でき、擬似的な 256×192 ドットのグラフィック表示に使われます。色はパターンの横 1 ライン（8 ドット）ごとに 2 色まで指定できます。MSX-BASIC では SCREEN 2 に相当し、スプライトはモード 1 が使えます。

| 項目 | 内容 |
|---|---|
| 画面 | 32×24 パターン（256×192 ドット）、3 バンク構成 |
| パターン | 実質 768 種、8×8 ドット |
| 色 | パレット 16 色、横 1 ラインごとに 2 色まで |
| 必要メモリ | パターンジェネレータテーブル 6,144 バイト、カラーテーブル 6,144 バイト、パターンネームテーブル 768 バイト |

### パターンネームテーブル（R#2）

768 バイトのテーブルで、Graphic1 と同じ配置です。上段（0〜7 行）、中段（8〜15 行）、下段（16〜23 行）の各行群が、それぞれ対応するバンクのパターンを参照します。先頭アドレスは R#2 で 1KB 単位に指定できます。

### パターンジェネレータテーブル（R#4）

3 バンク分で 6,144 バイトです。

アドレス = ベースアドレス ＋ バンク×2,048 ＋ パターン番号×8 ＋ ライン（0〜7）

バンクは表示行の群（行 ÷ 8）で決まりますが、ベースアドレスの A12、A11 にあたるビットと AND が取られます。したがって、R#4 の当該ビットを 0 にすると、そのバンクは他バンクと同じパターン集合を参照します。先頭アドレスは R#4 で 8KB 単位に指定できます。

### カラーテーブル（R#3、R#10）

パターンジェネレータテーブルと同じ大きさ（6,144 バイト）で、パターンの横 1 ラインごとに、上位 4 ビット＝前景色、下位 4 ビット＝背景色を指定します。

アドレス = ベースアドレス ＋ バンク×2,048 ＋ パターン番号×8 ＋ ライン（0〜7）

カラーテーブル側も、バンク選択とパターン群の選択に対して、ベースアドレスの対応ビットとの AND によるマスクがかかります。ベース側のビットを 0 にすると、対応する範囲のパターンで色を共用できます。先頭アドレスは R#3 と R#10 で 8KB 単位に指定できます。周辺色は R#7 の下位 4 ビットで指定します。

## 5.8 Graphic3 モード

VRAM のテーブル配置は Graphic2 とまったく同じです。違いはスプライトがモード 2 になる点だけです。MSX-BASIC では SCREEN 4 に相当します。配置については 5.7 節を参照してください。

## 5.9 Graphic4 モード

256×212 ドットのビットマップモードです。1 バイトが 2 ドットに対応し、ドットごとにパレット 16 色から色を指定できます。VDP コマンドが使用できます。MSX-BASIC では SCREEN 5 に相当し、スプライトはモード 2 が使えます。

| 項目 | 内容 |
|---|---|
| 画面 | 256×212ドット（LN=0 で 192 ライン） |
| 色 | パレット 16 色、ドットごとに指定可能 |
| 必要メモリ | 24,576 バイト（192 ライン時、6000h）／27,136 バイト（212 ライン時、6A00h） |

### ビットマップの配置（R#2）

ドット (X, Y) のアドレスは次の式で求めます。

アドレス = ベースアドレス ＋ Y×128 ＋ X÷2

X が偶数のドットはバイトの上位 4 ビット、奇数のドットは下位 4 ビットが色（パレット番号）を表します。1 ページの大きさは 32,768 バイト（8000h）で、ベースアドレスは R#2 で 32KB 単位に指定できます。互換モードでは 00000h、08000h、10000h、18000h の 4 箇所、拡張モードでは A17 も有効になり 20000h～38000h を加えた 8 箇所が選べます。周辺色は R#7 の下位 4 ビットで指定します。

## 5.10 Graphic5 モード

512×212ドットのビットマップモードです。1 バイトが 4ドットに対応し、ドットごとにパレット 4 色から色を指定できます。MSX-BASIC では SCREEN 6 に相当し、スプライトはモード 2、VDP コマンドも使用できます。

| 項目 | 内容 |
|---|---|
| 画面 | 512×212ドット（LN=0 で 192 ライン） |
| 色 | パレット 4 色、ドットごとに指定可能 |
| 必要メモリ | 24,576 バイト（192 ライン時、6000h）／27,136 バイト（212 ライン時、6A00h） |

### ビットマップの配置（R#2）

アドレス = ベースアドレス ＋ Y×128 ＋ X÷4

1 バイト内の 4ドットは、X÷4 の剰余が 0、1、2、3 の順に、ビット 7～6、5～4、3～2、1～0 の 2ビットずつが対応します。ベースアドレスの指定方法は Graphic4 と同じです。

なお、このモードでは周辺色とスプライトの色指定にタイリングが行われます。色指定の 4 ビットのうち、上位 2 ビットが偶数 X のドット、下位 2 ビットが奇数 X のドットの色を表します（R#7 による周辺色指定もこの形式です）。

## 5.11 Graphic6 モード

512×212 ドットのビットマップモードです。1 バイトが 2 ドットに対応し、ドットごとにパレット 16 色を指定できます。MSX-BASIC では SCREEN 7 に相当し、スプライトはモード 2、VDP コマンドも使用できます。

| 項目 | 内容 |
|---|---|
| 画面 | 512×212 ドット（LN=0 で 192 ライン） |
| 色 | パレット 16 色、ドットごとに指定可能 |
| 必要メモリ | 49,152 バイト（192 ライン時、0C000h）／54,272 バイト（212 ライン時、0D400h） |

### ビットマップの配置（R#2）

アドレス = ベースアドレス ＋ Y×256 ＋ X÷2

X が偶数のドットはバイトの上位 4 ビット、奇数のドットは下位 4 ビットが色を表します。1 画面は 64KB（10000h）の範囲に収まり、ベースアドレスは 00000h または 10000h（拡張モードでは 20000h、30000h も可）から R#2 で選択します。周辺色は R#7 の下位 4 ビットで指定します。

## 5.12 Graphic7 モード

256×212 ドットのビットマップモードです。1 バイトが 1 ドットにそのまま対応し、256 色を直接指定します。パレットは介しません。MSX-BASIC では SCREEN 8 に相当し、スプライトはモード 2、VDP コマンドも使用できます。

| 項目 | 内容 |
|---|---|
| 画面 | 256×212 ドット（LN=0 で 192 ライン） |
| 色 | 256 色（RGB 直接指定） |
| 必要メモリ | 49,152 バイト（192 ライン時、0C000h）／54,272 バイト（212 ライン時、0D400h） |

### ビットマップの配置（R#2）

アドレス = ベースアドレス ＋ Y×256 ＋ X

1 バイトの内訳は、ビット 7〜5 が緑（3 ビット）、ビット 4〜2 が赤（3 ビット）、ビット 1〜0 が青（2 ビット）の輝度情報です。ベースアドレスの指定方法は Graphic6 と同じです。

なお、Graphic7 は SCREEN 7 以上のモードであるため、VRAM の物理配置はインタリーブ（偶数アドレスが DRAM0、奇数アドレスが DRAM1）になります。ただし、スプライトモード 3 を使う場合はフラットな配置に戻ります。物理配置の詳細と、切り替え時にデータの並びが変わる点への注意は 11.1 節を参照してください。SCREEN 10〜12（5.13 節、5.14 節）も同様です。

【要確認】Graphic7 モードでの R#7（周辺色）の解釈。従来資料では他モードと同様に R#7 で周辺色を指定するとされていますが、このモードでは R#7 の 8 ビット全体を GRB 形式として解釈するのか、下位 4 ビットのみを使うのか、原稿・資料からは確定できませんでした。

### サンプルプログラム

サンプルプログラム `g7_demo.asm` は、Graphic7 の 256 色すべてを表示する例です。画面を 8×8 のブロックに分け、ブロックの列で緑（G）、ブロックの行で赤（R）、ブロック内の 4 本の帯で青（B）を変えることで、256 色の組み合わせを一覧できます。実行結果を図 5.1 に示します。

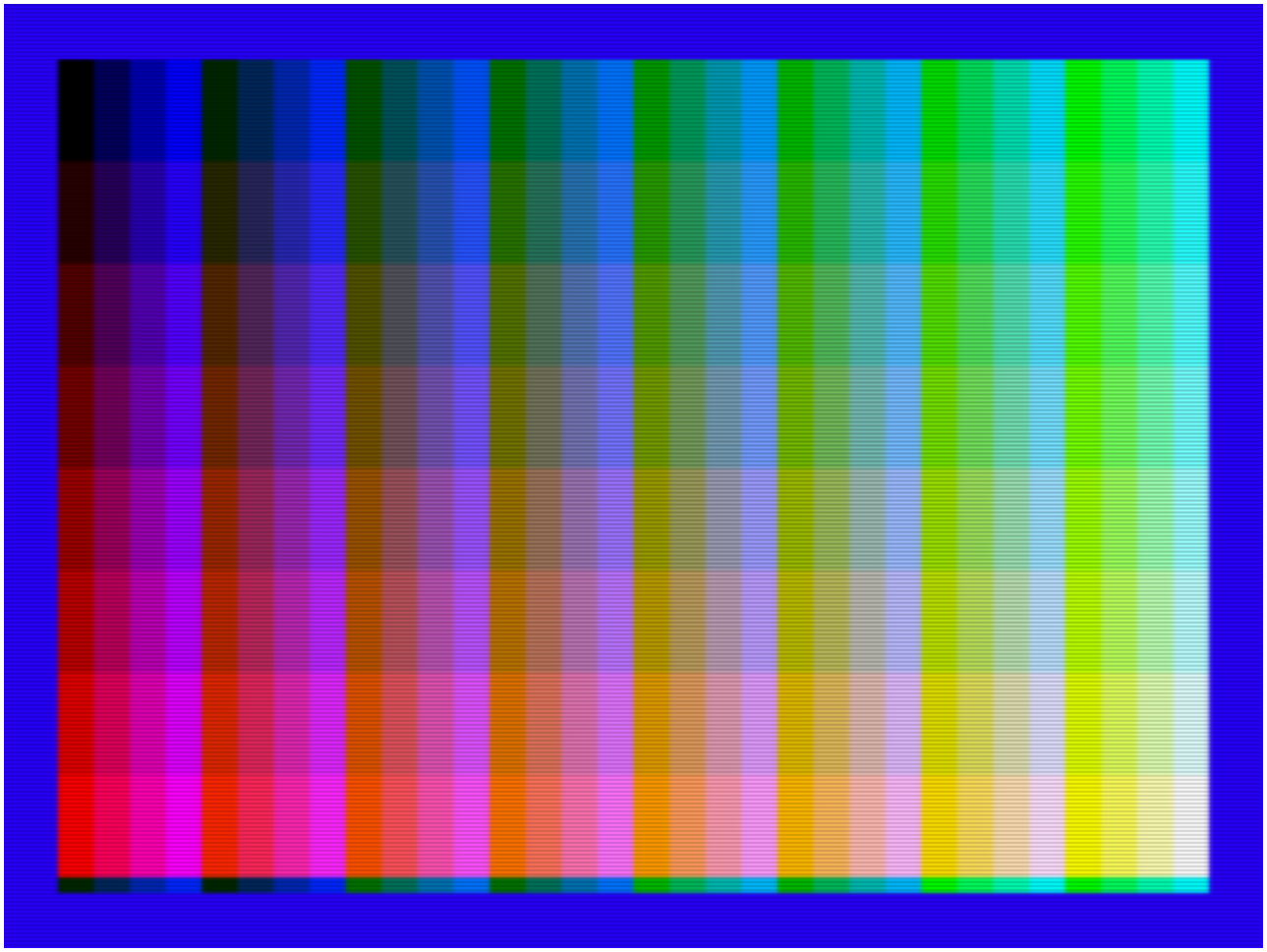


図 5.1 g7_demo.asm の実行結果（Graphic7 の 256 色）

Graphic7 では 1 バイトが 1ドットにそのまま対応するため、画像は書き込みアドレスを先頭に設定し、CPU から Port#0（98h）へ 54,272 バイト（212 ライン時）を順に書き込むだけで表示できます。パレットは介しません。1 ライン分の書き込みは次のとおりで、ブロック列（G）、ブロック行（R）、ブロック内の帯（B）から色バイトを合成し、同じ色が続く区間はまとめて書き出しています。

```
draw_line::
        ld      c, 0            ; C = G (block column, 0..7)
g_loop:
        ld      l, 0            ; L = B (stripe, 0..3)
b_loop:
        ; color byte = (G << 5) | (R << 2) | B
        ld      a, c
        sla     a
        sla     a
        sla     a
        sla     a
```

```
        sla     a           ; A = G << 5
        ld      h, a
        ld      a, e
        sla     a
        sla     a           ; A = R << 2
        or      a, h
        or      a, l            ; A = the dot's color byte
        ld      h, 8            ; one stripe is 8 dots wide
px_loop:
        out     [0x98], a       ; write one dot (address auto-increments)
        dec     h
        jr      nz, px_loop
        inc     l
        ld      a, l
        cp      a, 4
        jr      nz, b_loop
        inc     c
        ld      a, c
        cp      a, 8
        jr      nz, g_loop
        ret
```

ブロック行は 26 ラインごとに R を進め、8 ブロック行で 208 ラインを描きます。残りの 4 ライン（208〜211 ライン目）は R = 7 のまま描くため、図 5.1 の下端には細い帯が見えます（呼び出し側のループはソースファイル参照）。

## 5.13 Graphic7-YAE モード（SCREEN 10／11）

Graphic7 の変種で、YJK 方式の自然画表示と、パレット 16 色による表示とを、ドットごとに混在させることができます。MSX-BASIC では SCREEN 10 と SCREEN 11 に相当します。両者は表示方法が同じで、違いは VDP コマンドがデータを RGB（パレット）として解釈するか、YJK として解釈するかにあります（第 8 章）。

設定は、Graphic7 のモードビットに加えて R#25 のビット 3（YJK）とビット 4（YAE）の両方を 1 にします。なお、YJK=0、YAE=1 の組み合わせは仕様で決められていない設定ですので、使用しないでください。

VRAM の配置は Graphic7 とまったく同じ（アドレス = ベースアドレス ＋ Y×256 ＋ X）です。異なるのは 1 バイトの解釈で、水平に隣り合う 4 ドットを 1 グループとして扱います。

| バイト | ビット 7〜4 | ビット 3 | ビット 2〜0 |
|---|---|---|---|
| 1ドット目 | Y またはパレット番号 | A | K の下位 3 ビット |
| 2ドット目 | Y またはパレット番号 | A | K の上位 3 ビット |
| 3ドット目 | Y またはパレット番号 | A | J の下位 3 ビット |
| 4ドット目 | Y またはパレット番号 | A | J の上位 3 ビット |

- ビット 3（A）が 0 のドットは YJK 方式で表示されます。このときビット 7〜4 は輝度 Y の上位 4 ビットであり、Y は偶数値（0、2、4、…、30）の 16 段階になります。
- ビット 3（A）が 1 のドットは、ビット 7〜4 がパレット番号（0〜15）として解釈され、パレットの色で表示されます。

J、K の意味と RGB への変換は 5.14 節と同じです。

## 5.14 Graphic7-YJK モード（SCREEN 12）

Graphic7 の変種で、色を YJK 方式で表す自然画モードです。輝度成分 Y をドットごとに持ち、色差成分 J、K を水平 4 ドットで共有することで、限られたビット数で約 19,268 色の同時表示を実現しています。MSX-BASIC では SCREEN 12 に相当します。

設定は、Graphic7 のモードビットに加えて R#25 のビット 3（YJK）を 1、ビット 4（YAE）を 0 にします。VRAM の配置は Graphic7 とまったく同じです。

4 ドット 1 グループの各バイトの内訳は次のとおりです。

| バイト | ビット 7〜3 | ビット 2〜0 |
|---|---|---|
| 1ドット目 | Y | K の下位 3 ビット |
| 2ドット目 | Y | K の上位 3 ビット |
| 3ドット目 | Y | J の下位 3 ビット |
| 4ドット目 | Y | J の上位 3 ビット |

- Y は 5 ビットの符号なし値（0〜31）で、ドットごとに指定できます。

- J、K はそれぞれ 6 ビットの 2 の補数（−32〜31）で、4 ドットで共有します。J は赤の色差、K は緑の色差を表します。
- RGB への変換は、R = Y ＋ J、G = Y ＋ K、B = 5/4×Y − J/2 − K/4 で行われ、結果は 0〜31 の範囲に丸められます（内部の色信号は各成分 5 ビットです）。

## 5.15 V9968 における拡張と相違点

最後に、スクリーンモードに関する V9968 の位置付けをまとめます。

- **新しいスクリーンモードはありません。** 全 12 モードは V9958 までの資産がそのまま動作します。
- **M3 ドントケアの正式化（9958 Compatible）**：Text1 と Multicolor では M3 の値によらずモードが選択できます（5.2 節）。
- **256KB VRAM（拡張モード）**：V58 ビット（R#21 ビット 0）を 0 にした拡張モードでは、R#2、R#4、R#6、R#10、R#11 に書く A17 がそのまま有効になります。互換モード（V58=1）では、これらのレジスタに書いた A17 は 0 として保持されます。これにより、ビットマップモードのページ数が増えます（Graphic4／5 では 8 ページ、Graphic6／7 では 4 箇所のベースが選択可能）。なお、R#21 に 1 を書くと、上記 5 つのテーブルのベースアドレスの A17 はすべて 0 に戻ります。
- **VRAM アドレスの折り返し**：CPU から VRAM にアクセスする際のアドレスの折り返し幅は、モードによって 14 ビット／17 ビット／18 ビットと異なります。Text1 など {M5,M4}=00 のモードでは 14 ビットに折り返します。詳細は第 11 章を参照してください。

- **フラットインターレース（FIL）**：従来のインターレース表示では、page0 と page1（または page2 と page3）を垂直方向に半画素ずらして交互に表示しても、2 つのページは別画面のままで、ページをまたいで線を引いてもつながりませんでした。FIL（R#20 ビット 5）を 1 にすると、page0 と page1 を VRAM 上で上下につなげた 1 枚の縦に長い画面として表示でき、VDP コマンドも結合した高解像度の画面として扱います（第 3 章・第 8 章参照）。

- **ドットの並びは従来と同一**：Graphic5／Graphic6／Graphic7 のドットの並び（ビット割り当て）は、従来（MSX2 テクニカルハンドブックの式 4.2〜4.4）と同一です。

---

### 本章の出典

- 原稿「V9968 Programmer's Manual（画面モード編）」：章構成と Text1 の詳細、互換性に関する方針。
- RTL 検証ノート（v9968_registers.md）：モード選択ビットと M3 ドントケア、ベースアドレスレジスタのビット割り

当て、A17 マスクの動作。

- MSX2 テクニカルハンドブック（テクハン wiki「3 章 MSX2 の画面モード」）：従来モード（Text1〜Graphic7）の仕様、テーブル配置、座標からアドレスを求める式。
- YJK の符号化形式については、従来資料（Grauw "The YJK screen modes"、MSX2+ 世代の公開資料）で相互に一致する内容を採用しました。

# 第 6 章 スプライト

スプライトは、背景（パターン面）とは独立に、画面上の任意の位置へキャラクタパターンを表示する機能である。本章では V9968 の 3 つのスプライトモードについて、VRAM 上のデータ配置、表示の優先順位、衝突検出、各モードの制限を説明する。

レジスタのビット定義そのものは第 3 章を、本章に現れるステータスレジスタ（S#0、S#3〜S#6）の詳細は第 4 章を参照されたい。

## 6.1 スプライトモードの概要

V9968 のスプライトには次の 3 つのモードがある。

| モード | 由来 | 利用できる画面モード | プレーン数 | 1 ラインに表示できる数 |
|---|---|---|---|---|
| スプライトモード 1 | TMS9918 互換 | Graphic1、Graphic2、Multicolor | 32 枚（#0〜#31） | 4 枚（S16=1 のとき 16 枚） |
| スプライトモード 2 | V9938 で追加 | Graphic3〜Graphic7 | 32 枚（#0〜#31） | 8 枚（S16=1 のとき 16 枚） |
| スプライトモード 3 | V9968 で追加（新モード） | Graphic1〜Graphic7、Multicolor | 64 枚（#0〜#63） | 16 枚 |

モード 1 とモード 2 は、設定されている画面モードによって自動的に選択される（従来と同じである）。テキストモード（Text1、Text2）ではスプライトは表示されない。

モード 3 は、R#20 の SP3 ビットを 1 にすることで、モード 1／モード 2 の代わりに有効になる新モードである。従来モードの「画面モードごとに決まる 2 種類のスプライト」を置き換えるもので、Graphic1〜Graphic7 と Multicolor のすべてで利用できる。

各モードに共通するレジスタは次の通りである。

| レジスタ | 役割 |
|---|---|
| R#5、R#11 | スプライトアトリビュートテーブルのベースアドレス |

| レジスタ | 役割 |
|---|---|
| | |
| R#6 | スプライトパターンジェネレータテーブルのベースアドレス |
| R#1 bit1（SI）、bit0（MAG） | モード 1／2 のスプライトサイズ（8×8／16×16）と拡大指定 |
| R#8 bit1（SPD） | 1 にするとスプライト表示全体を禁止する（全モード共通） |
| R#20 bit3（SP3） | 1 にするとスプライトモード 3 が有効になる |
| R#20 bit7（S16） | 1 にするとモード 1／2 で 1 ラインの表示上限が 16 枚になる（6.7 参照） |
| R#20 bit1（SVNS） | 1 にするとスプライトが垂直スクロール（R#23）の影響を受けなくなる（6.2 参照） |

## 6.2 共通の約束事

### 座標と表示位置

スプライトの Y 座標は「表示したいラインの 1 つ上」を指す。すなわち、アトリビュートに書いた Y の値に対し、スプライトはライン Y+1 から表示され始める。画面最上段（ライン 0）に表示したい場合は Y = 255（-1）と書く。これは従来（TMS9918／V9938）と同じ約束である。

X 座標は画面左端が 0 である。モード 1／2 では EC（Early Clock）ビットを使うと表示位置を 32 ドット左へずらすことができ、画面左端から少しずつ姿を現すような表示ができる。

なお、横 512 ドットの高解像度画面モード（Graphic5、Graphic6）では、スプライトの 1 ドットは画面の横 2 ドット分に相当し、スプライトの絶対的な大きさは他のモードと変わらない。これも従来と同じである。

通常、スプライトの表示位置は垂直スクロール（R#23）の影響を受け、背景と一緒にスクロールする。R#20 の SVNS ビットを 1 にすると、スプライトは R#23 の指定に影響を受けなくなり、Y 座標が画面上の絶対位置を指すようになる（Y=0 で画面の一番上に表示される）。背景だけをスクロールさせてスプライトを画面に固定したい場合に使う（第 3 章の R#20 の節を参照）。

### 優先順位

プレーン番号の小さいものほど優先順位が高い。同じドットに複数のプレーンが重なった場合、原則として最も番号の小さいプレーンのドットが表示される（モード 2 の CC ビットによる例外は 6.4.3 で述べる）。

1 ラインに表示できるプレーン数にはモードごとに上限があり（表 6.1 参照）、上限を超えた分はそのラインでは表示されない。この判定はラインごとに行われるため、上下に少しずれたスプライト同士では「一部だけ消える」ことがある。上限を超えた場合の通知（5S フラグと 5 番目以降のプレーン番号）については 6.6.2 で述べる。

### 表示の打ち切り

アトリビュートの Y 座標に特定の値を書くと、そのプレーン以降の走査が打ち切られ、後続のプレーンは表示されなくなる。打ち切り値はモードによって異なる。

| モード | 打ち切り値 |
|---|---|
| モード 1 | Y = 208（D0h） |
| モード 2、モード 3 | Y = 216（D8h） |

不要なプレーンを確実に消したい場合は、この打ち切りを使うのが確実である（従来と同じ使い方である）。

## 6.3 スプライトモード 1

モード 1 は TMS9918 のスプライトそのものである。Graphic1、Graphic2、Multicolor で使われる。

### 6.3.1 アトリビュートテーブル

プレーン 1 枚につき 4 バイトで、ベースアドレス（R#5、R#11）からプレーン番号順に並ぶ。

| オフセット | 内容 |
|---|---|
| +0 | Y 座標 |
| +1 | X 座標 |
| +2 | パターン番号（0〜255） |

| オフセット | 内容 |
|---|---|
| +3 | bit7：EC（Early Clock）／ bit3-0：色コード |

色コードはプレーン全体で 1 色だけを指定する。パターンのビットが 1 の部分がこの色で描かれる（色の決まり方は第 7 章参照）。EC を 1 にすると表示位置が 32ドット左へずれる。

### 6.3.2 パターンテーブル

パターンは 1ドット 1ビットで、1 の部分だけが色を持つ。

- 8×8ドット（SI=0）：1 パターン 8 バイト。パターン番号 0〜255 がそのまま使われる。
- 16×16ドット（SI=1）：8×8 パターンを 4 個（左上、右上、左下、右下の順）組み合わせて使う。指定するパターン番号の下位 2ビットは無視され、4 の倍数に切り下げた番号のパターン群が表示される。

MAG=1（R#1 bit0）にすると、1ドットが縦横 2 倍に拡大表示される。サイズと拡大は全プレーン共通の設定である。

### 6.3.3 表示制限

- 同時に表示できるのは 32 枚、1 ラインには優先順位の高い順に 4 枚まで（S16=1 のときは 16 枚まで。6.7 参照）。
- 上限を超えたプレーンは、そのラインでは表示されない（6.6.2 参照）。

## 6.4 スプライトモード 2

モード 2 は V9938 で追加されたモードで、Graphic3〜Graphic7 で使われる。モード 1 との主な違いは、(1) 1 ラインごとに色を指定できる、(2) 1 ラインに 8 枚まで表示できる（V9968 では S16 により 16 枚まで拡張できる。6.7 参照）、(3) 衝突した座標を知ることができる、の 3 点である。

### 6.4.1 アトリビュートテーブル

モード 1 と同じくプレーン 1 枚につき 4 バイトである。

| オフセット | 内容 |
|---|---|
| +0 | Y 座標 |

| オフセット | 内容 |
|---|---|
| +1 | X 座標 |
| +2 | パターン番号（0〜255） |
| +3 | モード 2 では色指定に使わない（6.4.4 の拡張を除く） |

パターンテーブルの形式と、SI／MAG の扱いはモード 1 と同じである。

### 6.4.2 スプライトカラーテーブル

色の情報はアトリビュートテーブルとは別のスプライトカラーテーブルに置かれる。カラーテーブルは、アトリビュートテーブルと同じ R#5、R#11 で決まる位置に自動的に配置される。具体的には、アトリビュートテーブルのアドレスではベースの bit9 までが使われるのに対し、カラーテーブルでは bit9 が 0 に固定される。このため、両者が重ならないよう、ベースアドレスの A9 は必ず 1 に指定しなければならない（従来と同じ注意点である）。

カラーテーブルはプレーン 1 枚につき 16 バイトで、パターンの各ラインに 1 バイトずつ対応する（8×8 の場合は先頭の 8 バイト、16×16 の場合は 16 バイト全部を使う）。

| ビット | 内容 |
|---|---|
| bit7 | EC（Early Clock。このラインだけ左へ 32ドットずらす） |
| bit6 | CC（優先順位の合成。6.4.3 参照） |
| bit5 | IC（1 のラインは衝突を発生しない。6.6.1 参照） |
| bit3-0 | 色コード（このラインのパターンが 1 の部分の色） |

### 6.4.3 CC ビットと色の OR 合成

CC ビットは、重なったスプライト同士の色を合成するための仕組みである。

CC=1 のプレーンは、単独では表示の優先順位を持たず、「自分より優先順位が高く、かつ CC=0 で、最も自分に近いプレーン」と同じ優先順位で扱われる。そのようなプレーン同士が重なったドットでは、表示される色は各プレーンの色コードの論理和（OR）になる。CC=1 のプレーン同士が数珠つなぎになっている場合は、そのすべてが最初の CC=0 プレーンにぶら下がる形で合成される。

この仕組みを使うと、モード 2 でも「複数枚を重ねて多色のキャラクタを描く」ことができる。なお、CC による合成で重なったドットは衝突とはみなされない。

### 6.4.4 V9968 での拡張：EPAL 時のパレットセット

拡張パレットモード（R#20 の EPAL = 1、第 7 章参照）では、モード 2 のスプライトもパレットセット（第 7 章）を持てるようになる。パレットセットにはアトリビュート第 4 バイト（オフセット +3）の下位 4 ビットが使われる。EPAL = 0 のとき、および EPAL = 1 でもこのバイトが 0 のときは、従来通りパレットセット 0（パレット番号 0〜15）が使われる。

なお、Graphic7（SCREEN8）では、V9938 と異なりスプライトの色もパレット経由になる。背景と同じ 256 色パレットを 16 色ずつのグループ（パレットセット）に分けて使い、スプライトはグループ内の 16 色で表示される。各グループの先頭の色は透明扱いになるため、実質的に使えるのは 15 色である（7.4.3 節／7.4.4 節参照）。

### 6.4.5 表示制限

- 同時に表示できるのは 32 枚、1 ラインには優先順位の高い順に 8 枚まで（S16=1 のときは 16 枚まで。6.7 参照）。
- 上限を超えたプレーンは、そのラインでは表示されない（6.6.2 参照）。

### 6.4.6 サンプルプログラム

サンプルプログラム `spr_demo.asm` は、SCREEN 5（Graphic 4、スプライトモード 2）で 16×16 ドットのボールを 4 枚表示する例です。実行結果を図 6.1 に示します。



図 6.1 spr_demo.asm の実行結果（スプライトモード 2）

アトリビュートテーブル（7600h）には、表示する順に Y 座標、X 座標、パターン番号を並べ、最後に Y = 216 のエントリでリストを終えます。

```
attribute_table:
        db      60, 40, 0, 0        ; plane 0: rainbow ball
        db      40, 110, 0, 0       ; plane 1: yellow ball
        db      90, 180, 0, 0       ; plane 2: light blue ball
        db      140, 70, 0, 0       ; plane 3: magenta ball
        db      216, 0, 0, 0        ; plane 4: Y=216 ends the list
```

左端のボール（プレーン 0）だけ、カラーテーブル（7400h）の 16 バイトすべてに異なる色を設定し、1 ラインごとに色が変わる縞模様にしています。これがモード 2 の特徴で、色はスプライト単位ではなくライン単位に指定します（→6.4.2）。残りの 3 枚は全ライン同じ色です（ソースファイル参照）。

```
        ; plane 0: a different color on every line
        db      0x06, 0x08, 0x08, 0x09, 0x09, 0x0b, 0x0b, 0x0a
        db      0x0a, 0x0c, 0x02, 0x02, 0x03, 0x03, 0x05, 0x05
```

パターンテーブル（7800h）にはボールの絵柄 32 バイトを置きます（データはソースファイル参照）。なお、16×16ドットにするには R#1 の SI ビットを 1 にします。

## 6.5 スプライトモード 3（V9968 新モード）

モード 3 は V9968 で新設されたスプライトモードである。従来モードの不満――1 枚に 1 色（モード 1）／1 ラインに 1 色（モード 2）しか使えない、重ね描きのための CC 合成が扱いにくい――を解消するために設計されており、次の特長を持つ。

- プレーンは 64 枚（#0〜#63）、1 ラインに 16 枚まで表示できる。
- パターンはドットごとに 4 ビットの色コードを持ち、ドットバイドットで色付けできる。
- プレーンごとに拡大・縮小（任意倍率）ができる。
- プレーンごとに左右反転、上下反転ができる。
- プレーンごとに半透明（背景とのブレンド）ができる。
- プレーンごとにパレットセット（第 7 章）を指定でき、256 色パレット全体を使える。

モード 3 では SI／MAG（R#1）は使わない。サイズや倍率はプレーンごとにアトリビュートで指定する。

### 6.5.1 アトリビュートテーブル

プレーン 1 枚につき 8 バイトで、ベースアドレス（R#5、R#11）からプレーン番号順に並ぶ。なお、モード 3 でもアドレス生成には従来と同じ仕掛けがあり、アトリビュートテーブルのベースアドレスの bit8、bit7 は、プレーン番号の上位ビットとの AND マスクとして働く。ベースアドレスは 512 バイト境界にそろえて配置すること。

| オフセット | 内容 |
|---|---|
| +0 | Y7–Y0（Y 座標の下位 8 ビット） |
| +1 | bit7-6：SZ1、SZ0（パターンサイズ）／ bit5-2：0（未使用。書き込んでも無視される）／ bit1-0：Y9、Y8（Y 座標の上位 2 ビット。Y は合計 10 ビット） |
| +2 | MGY7–MGY0（縦の表示ライン数） |
| +3 | bit7-6：TP1、TP0（半透明）／ bit5：RVY（上下反転）／ bit4：RVX（左右反転）／ bit3-0：PS3–PS0（パレットセット） |

| オフセット | 内容 |
|---|---|
| | |
| +4 | X7–X0（X 座標の下位 8 ビット） |
| +5 | bit7：0（未使用）／ bit6-4：PTS2、PTS1、PTS0（パターンシートのページ番号。1 ページは 256 ライン＝32KB）／ bit3-2：0（未使用）／ bit1-0：X9、X8（X 座標の上位 2 ビット。X は合計 10 ビット） |
| +6 | MGX7–MGX0（横の表示ドット数） |
| +7 | bit7-4：PY3–PY0 ／ bit3-0：PX3–PX0（パターン番号。PX、PY はパターンシート上のタイル座標で、上位ニブルが PY、下位ニブルが PX。6.5.2 参照） |

各フィールドの意味は次の通りである。

- **Y 座標（10 ビット）**：表示位置の約束は従来と同じで、スプライトはライン Y+1 から、MGy ライン分だけ表示される。Y = 216 を書くとそのプレーン以降の走査が打ち切られる（6.2 参照）。
- **X 座標（10 ビット）**：画面左端が 0。スプライトは X から MGx ドットの範囲に表示される。X に大きな値（10 ビットで見た負数に相当する値）を与えると、左端から一部が見える状態で表示できる。
- **SZ（パターンサイズ）**：パターンの縦ドット数を決める。パターンの横幅は常に 16 ドットである。

| SZ | パターンの大きさ（横×縦ドット） | 1 パターンのバイト数 |
|---|---|---|
| 0 | 16×16 | 128 バイト |
| 1 | 16×32 | 256 バイト |
| 2 | 16×64 | 512 バイト |
| 3 | 16×128 | 1024 バイト |

- **MGx、MGy（表示サイズ）**：パターンを画面上に表示するドット数（横）とライン数（縦）。パターンはこの矩形に拡大・縮小されて表示される。等倍で表示したい場合は、MGx = 16、MGy = パターンの縦ドット数（SZ に対応する値）を指定する。

- **RVX、RVY**：パターンの左右反転、上下反転。
- **パレットセット**：このプレーンの色コードに適用するパレットセット（0〜15）。実際のパレット番号は {パレットセット, 色コード} の 8 ビットになる（第 7 章参照）。
- **パターン番号と PTS**：パターンは +7 バイトのパターン番号（上位ニブルが PY、下位ニブルが PX）で指定する。PX、PY はパターンシート上のタイル座標、+5 バイトの PTS はそのページ番号であり、パターンの位置はシート座標で (PX×16, PY×16 + PTS×256) になる（6.5.2 参照）。

半透明の指定は、+3 バイトの bit7-6（TP1、TP0）で行う。

| bit7-6 | 透過率 | 表示色の合成 |
|---|---|---|
| 0 | 0% | スプライトの色そのまま（不透明） |
| 1 | 25% | スプライト 75% ＋ 背景 25% |
| 2 | 50% | スプライト 50% ＋ 背景 50% |
| 3 | 75% | スプライト 25% ＋ 背景 75% |

### 6.5.2 パターンテーブル

モード 3 のパターンは、横 16ドット × 4ビット（ドットごとの色コード）である。1 ライン分は 8 バイトで、各バイトの上位ニブルが左のドット、下位ニブルが右のドットを表す。色コード 0 のドットは透明（描画されず、衝突もしない）。

モード 3 のパターンデータは、横 256ドット（4ビット／ドット）の「パターンシート」として配置されるものと捉えると分かりやすい。シート上には 16×16ドットのタイルが横に 16 個並び、それが縦に続いている。アトリビュート +7 バイトの PX、PY はこのシート上のタイル座標であり、+5 バイトの PTS（bit6-4、PTS2–PTS0）は、シートを 256 ラインごとに区切ったページの番号である。なお、+7 バイトの値は「パターン番号」としても読め、その場合は パターン番号 ＝ PY × 16 ＋ PX である。

パターンのアドレス計算は、現在の画面モードによらず、常に Graphic4（SCREEN 5）相当の計算になる。つまり、パターンシートはパターンジェネレータテーブルベース（R#6）を先頭として、1 ライン 256ドット × 4ビットのビットマップとまったく同じ並びで VRAM に置かれる。このため、表示が SCREEN 6、7、8 など他の画面モードのときでも、R#45 の FG4 を 1 にすれば、VDP コマンドでこのパターンデータへ直接描画できる（第 3 章・第 8 章参照）。

モード 3 のパターンは連続したアドレス範囲として読み出されるため、SP3 を有効にすると、SCREEN 7 以上でも VRAM の物理配置はインタリーブではなくフラットに戻る（11.1 節参照）。このため、SCREEN 7 以上で SP3 を切り替え

ると VRAM データの物理的な並びが変わる。背景のビットマップなど、モード 3 以外のデータを併用する場合は切り替えの前後でデータの配置がずれる点に注意すること。

パターン内のドットの位置を(x, y)とする。x は横方向の位置で 0〜15、y は縦方向の位置で 0〜127 の範囲を取る(実際に使う y の上限は、SZ で決まるパターンの縦ドット数 − 1 である)。このドットのシート座標(X, Y)は次の通りである。

- X = PX × 16 + x
- Y = ((PY × 16 + y) & 255) + PTS × 256

Y の計算では、ページ内での位置(PY × 16 + y)を先に 255 でマスク(& 255)してから、ページの先頭(PTS × 256)を加える。このため、パターンの下端がページの境界を越える場合、その続きは次のページには入らず、同じページの先頭に戻って置かれる。

ドットのバイトアドレスは、Graphic4 のドットアドレスそのものである(4 ビット／ドットで 2 ドットが 1 バイトになるため、>> 1 となる)。

```
アドレス = ベース
+ ((PX × 16 + x) + (((PY × 16 + y) & 255) + PTS × 256) × 256) >> 1
```

バイト内のどちらのニブルかは X の最下位ビット(すなわち x の最下位ビット)で決まり、偶数のドットが上位ニブル、奇数のドットが下位ニブルになる。

パターンがページ境界をまたがない場合は、上の式は次の展開形と一致する。各項はシートの寸法(1 ライン 256 ドット × 4 ビット = 128 バイト)から決まる。

```
ライン先頭のアドレス = ベース
+ PTS × 32768          (ページ送り：256 ライン × 128 バイト)
+ PY × 2048            (タイル行送り：16 ライン × 128 バイト)
+ y × 128              (シートの 1 ライン分)
+ PX × 8               (タイルの横位置：16 ドット × 4 ビット)
(ライン内は 8 バイト連続。右半分 8 ドット分は +4 から)
```

シートの各ラインには 16 個のタイルが横に並んでいるため、パターンデータは VRAM 上では行方向にインターリーブされた配置になる。

y は、表示ラインに対応するパターン内の行位置である。拡大・縮小がかかっている場合は「表示オフセット × パターン行数 ÷ MGy」に相当する行が y として選ばれ(RTL では逆数テーブルによる除算で求めている)、RVY = 1 のときは上下が反転する。

### 6.5.3 拡大・縮小の見え方

MGx、MGy にパターンの大きさと異なる値を指定すると、パターンは矩形いっぱいに引き伸ばされ、あるいは縮められて表示される。縮小時はドットが間引かれ、拡大時は同じドットが連続して表示される。サンプリングはパターンの端でクランプされるため、矩形の外側にパターンがはみ出すことはない。

MGx = 0 を指定した場合は、横方向に最大の 256 ドットへ拡大して表示される（0 は「表示なし」ではない）。同様に、MGy = 0 を指定した場合は、縦方向に最大の 256 ラインへ拡大して表示される。

### 6.5.4 表示制限

- 同時に表示できるのは 64 枚、1 ラインには優先順位の高い順に 16 枚まで。
- 17 枚目以降のプレーンは、そのラインでは表示されない（6.6.2 参照）。

### 6.5.5 サンプルプログラム

サンプルプログラム `spr3_demo.asm` は、SCREEN 5（Graphic 4、スプライトモード 3）で、16×16 ドットの矢印とボールの 2 種類のタイルを使い、プレーンごとに異なる表示倍率・反転・透明度を設定する例です。実行結果を図 6.2 に示します。上段は矢印を等倍、2 倍、3 倍で表示したもの、下段左は左右反転（RVX）、下段中は上下反転（RVY）、右はマゼンタの矩形の上にボールを TP = 2（50% ブレンド）で重ねたものです。

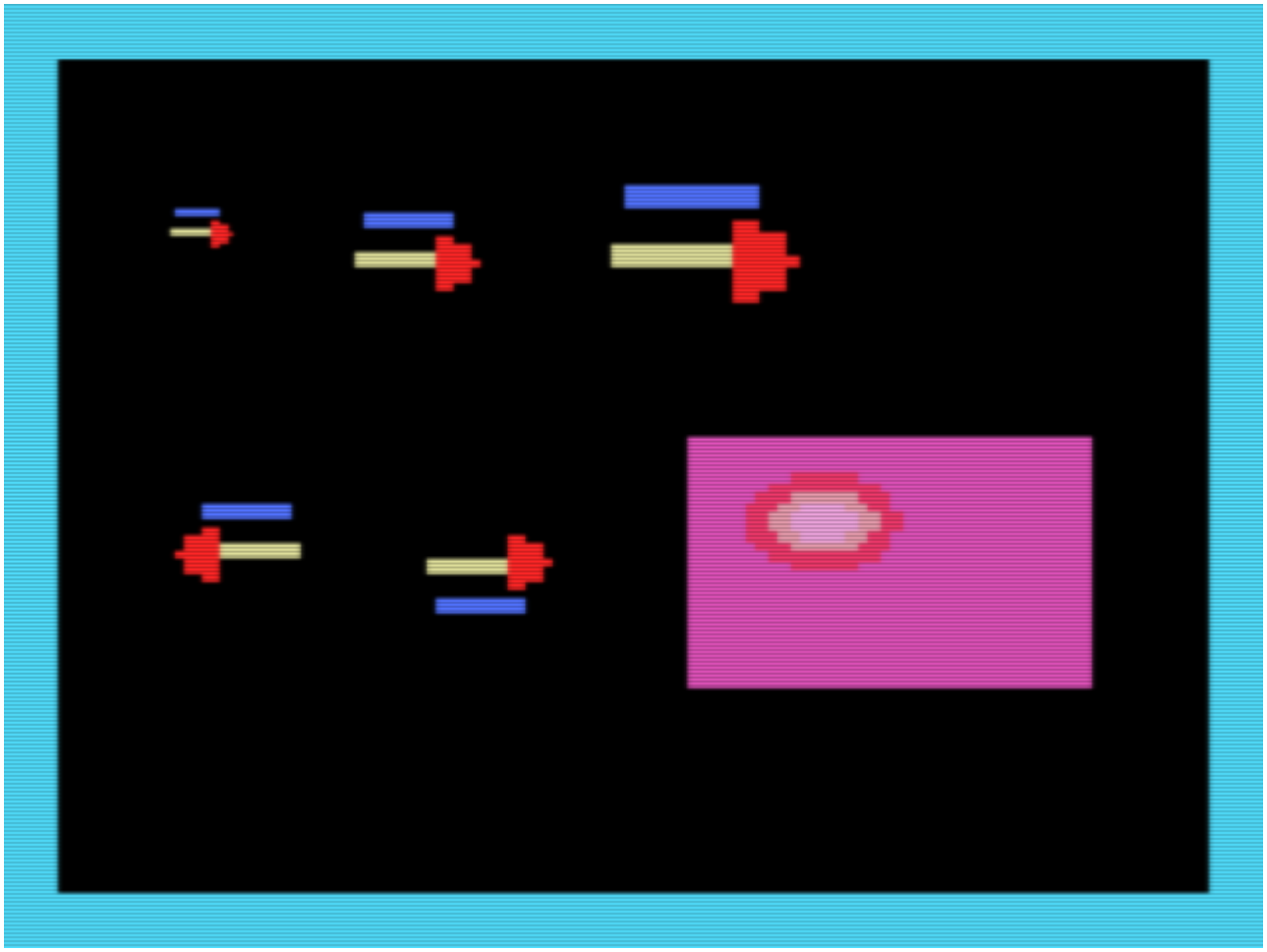


図 6.2 spr3_demo.asm の実行結果（スプライトモード 3）

モード 3 のアトリビュートは 1 プレーン 8 バイトで、Y 座標、倍率（MGY、MGX）、反転・透明度・パレットセット、X 座標、パターン位置をプレーンごとに持ちます（→6.5.1）。このサンプルのアトリビュートテーブルは次のとおりです。並びの最後は Y = 216 のエントリで終えます。

```
attribute_table:
        ; plane 0: arrow x1 at (24, 36)
        db      36, 0x00, 16, 0x00, 24, 0x00, 16, 0x00
        ; plane 1: arrow x2 at (64, 36)
        db      36, 0x00, 32, 0x00, 64, 0x00, 32, 0x00
        ; plane 2: arrow x3 at (120, 28)
        db      28, 0x00, 48, 0x00, 120, 0x00, 48, 0x00
        ; plane 3: arrow x2, RVX (points left) at (24, 110)
        db      110, 0x00, 32, 0x10, 24, 0x00, 32, 0x00
        ; plane 4: arrow x2, RVY (stripe at bottom) at (80, 110)
        db      110, 0x00, 32, 0x20, 80, 0x00, 32, 0x00
        ; plane 5: ball 40x40, TP = 2 (50%), at (150, 104)
```

```
        db      104, 0x00, 40, 0x80, 150, 0x00, 40, 0x01
        ; plane 6: terminator
        db      216, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
```

矢印は上が青い帯、軸が黄色、鏃（やじり）が赤という非対称な絵柄にしてあるため、RVX では鏃が左を向き、RVY では青い帯が下に来ることがはっきり分かります。倍率はパターン全体が MGX×MGY の矩形に引き伸ばされて表示されるため、3 倍の矢印では元のドットが 3×3 のブロックになっています（→6.5.3）。

パターンは Graphic 4 と同じ形式のシート（1 ライン 128 バイト）に置くため、タイルの各行は 128 バイト間隔で書き込みます（→6.5.2）。なお、このサンプルではスプライトモード 3 を有効にしてから、アトリビュートテーブルとパターンシートの基底アドレスを R#5、R#11、R#6 で明示的に設定しています（BIOS 実装によって初期値が異なるためです）。

## 6.6 衝突検出と表示上限の通知

### 6.6.1 スプライト同士の衝突

「衝突」とは、透明でないスプライトのドット同士が同じ座標を占めることである。衝突が起こると S#0 の C ビット（bit5）が 1 になる。

- モード 1：衝突の発生だけが分かる。座標は得られない。
- モード 2：CC = 1 のドット、および IC = 1 のラインのドットは衝突を発生しない。また、色コード 0 のドットは、通常は透明として衝突しないが、R#8 の TP ビットが 1 のとき（色 0 を不透明として扱うとき）は衝突の対象になる。
- モード 3：色コード 0 でないドット同士が重なれば衝突する。CC／IC に相当する仕組みはない。

モード 2 とモード 3 では、最初に衝突した座標を S#3〜S#6 から読み取ることができる。ただし、レジスタから得られる値は実座標そのものではなく、次の換算が必要である（従来と同じである）。

```
実際に衝突した X 座標 = （S#3、S#4 で表される値） − 12
実際に衝突した Y 座標 = （S#5、S#6 で表される値） − 8
```

衝突座標は S#5 を読み出すとリセットされる。また S#0 を読み出すと、C ビットを含む各フラグがクリアされる。詳細は第 4 章を参照されたい。

### 6.6.2 表示上限の通知（5S と 5TH）

1 ラインに表示できる上限を超えるプレーンが並んだ場合、S#0 の 5S ビット（bit6）が 1 になり、S#0 の下位 5 ビット（5TH）に「表示できなくなった最初のプレーン」の番号が入る。

| モード | 5S が立つ条件 | 5TH に入る番号 |
|---|---|---|
| モード 1（S16=0） | 1 ラインに 5 枚以上並んだ | 5 枚目のプレーン番号 |
| モード 2（S16=0） | 1 ラインに 9 枚以上並んだ | 9 枚目のプレーン番号 |
| モード 3、または S16=1 のモード 1／2 | 1 ラインに 17 枚以上並んだ | 17 枚目のプレーン番号（ただし 5TH は 5 ビットしかないため、プレーン番号が 32 以上の場合は 31 が入る） |

5S／5TH も S#0 の読み出しでクリアされる。

## 6.7 その他の拡張ビットについて

R#25 の SPS ビットは、V9968 で新設されたスプライト拡張ビットである。SPS は拡張モード（V58=0）のときにのみ書き込むことができ、1 にするとスプライトの表示プライオリティが循環的に変化する（表示優先順位のローテーション）。この動作は Sprite mode1 / 2 / 3 のすべてで利用できる。なお、R#25 の SP2 は水平スクロールのページ数選択ビットであり、スプライトとは関係しない（第 3 章の R#25 の節を参照）。

R#20 の S16 ビットは、V9968 で新設されたスプライト拡張ビットである（初期値 0）。1 ラインに表示できるスプライト数の上限を拡張するもので、Sprite mode3 では常に 1 ライン 16 枚まで表示されるため、S16 は効果を持たない。Sprite mode2 では、S16=0 のとき従来互換の 8 枚まで、S16=1 のとき 16 枚まで表示できる。Sprite mode1 では、S16=0 のとき従来互換の 4 枚まで、S16=1 のとき 16 枚まで表示できる。表示上限を超えた場合の通知（5S／5TH が立つ枚数）も、この上限に合わせて変わる（6.6.2 参照）。

---

本章の従来モード（モード 1／2）の記述は MSX2 テクニカルハンドブック（テクハン wiki 版）第 5 章と照合した。モード 3 および V9968 拡張部分は RTL（vdp_sprite_select_visible_planes.v、vdp_sprite_info_collect.v、vdp_sprite_makeup_pixel.v、vdp_timing_control_sprite.v、vdp_sprite_divide_table.v）と検証済みレジスタノートに基づく。

# 第 7 章 カラーパレットと色表現

V9968 の画面の色は、ほとんどの画面モードで「色コード → カラーパレット → RGB」という間接形式で決まる。本章では、パレット RAM の構造と書き込み方法、リセット時の状態、そして画面モードごとに色がどう解釈されるかを説明する。

レジスタのビット定義そのものは第 3 章を、各画面モードの VRAM データ形式は第 5 章を参照されたい。

## 7.1 パレット RAM の構造

カラーパレットは、V9968 内部のパレット RAM に保持される。

- エントリ数は 256。
- 各エントリは R、G、B それぞれ 5 ビット（各 0〜31）の組であり、全体では 32,768 色の中から選べる。
- 画面データやスプライトが持つ色情報は、原則として「パレットセット（上位 4 ビット）＋色コード（下位 4 ビット）」の 8 ビットのパレット番号として解釈され、対応するエントリの RGB が表示色になる。

従来（V9938／V9958）ではパレットは 16 エントリしかなく、パレット番号は色コード 4 ビットそのものだった。V9968 ではパレットが 256 エントリに拡張され、拡張パレットモード（EPAL、R#20 bit4）やスプライトモード 3 などを使うと、上位 4 ビット（パレットセット）も含めた 256 色すべてを参照できる。EPAL = 0 の通常の使い方ではパレットセットは 0 に固定され、従来通り 16 色のパレットとして動作する。

パレット RAM は書き込み専用であり、CPU から読み戻すことはできない。

## 7.2 パレットへの書き込み

パレットへの書き込みは、R#16（カラーパレットポインタ）でエントリ番号を指定し、Port#2 にデータを書くことで行う。書くべきバイト数と形式は EPAL の値で異なる。どちらの形式でも、R#16 を書き直すと書き込みフェーズ（何バイト目か）が先頭に戻り、1 エントリ分の書き込みが完了するたびにポインタは自動的にインクリメントされる。

### 7.2.1 EPAL = 0（通常形式、V9938／V9958 互換）

- R#16 でエントリ番号（0〜15）を指定し、Port#2 に 2 バイト書く。
- ポインタの自動インクリメントは下位 4 ビットの範囲で行われる（15 の次は 0 に戻る）。

| バイト | ビット配置 |
|---|---|
| 1 バイト目 | [0][R2][R1][R0][0][B2][B1][B0] |
| 2 バイト目 | [0][0][0][0][0][G2][G1][G0] |

各色は 3 ビット（0〜7）で指定する。内部では 5 ビットへ拡張されて格納される。拡張は「3 ビット値を上位に置き、その上位 2 ビットを下位にも複製する」方式（5 ビット値 = {v[2:0], v[2:1]}）であり、たとえば 7 は 31、6 は 27、3 は 13 になる。

### 7.2.2 EPAL = 1（拡張形式）

- R#16 でエントリ番号（0〜255）を指定し、Port#2 に 3 バイト書く。
- 各バイトは [0][0][0][値 4][値 3][値 2][値 1][値 0] の形で 5 ビットの値を 1 色分ずつ持ち、順に R、G、B を書く。
- ポインタの自動インクリメントは 8 ビット全範囲で行われる（255 の次は 0 に戻る）。

EPAL = 1 では、初期化したいエントリだけを選んで書き換えることも、0 番から順に 256 エントリ分をまとめて流し込むこともできる。

## 7.3 リセット時のパレット

リセット直後のパレットの状態は次の通りである。

- **エントリ 0〜15 だけが初期化され、V9958 互換の初期色が入る。**
- **エントリ 16〜255 は不定である。** EPAL やスプライトモード 3 でこれらのエントリを使う場合は、必ずプログラム側で初期化してから使うこと。

初期色（エントリ 0〜15）は、RGB 各 3 ビットで表すと次の通りである。5 ビットへの拡張は 7.2.1 の方式による。

| 番号 | 色 | R | G | B |
|---|---|---|---|---|
| 0 | 透明 | 0 | 0 | 0 |
| 1 | 黒 | 0 | 0 | 0 |
| 2 | 緑 | 1 | 6 | 1 |
| 3 | 明るい緑 | 3 | 7 | 3 |

| 番号 | 色 | R | G | B |
|---|---|---|---|---|
| | | | | |
| 4 | 暗い青 | 1 | 1 | 7 |
| 5 | 明るい青 | 2 | 3 | 7 |
| 6 | 暗い赤 | 5 | 1 | 1 |
| 7 | 水色（シアン） | 2 | 6 | 7 |
| 8 | 赤 | 7 | 1 | 1 |
| 9 | 明るい赤 | 7 | 3 | 3 |
| 10 | 暗い黄 | 6 | 6 | 1 |
| 11 | 黄 | 6 | 6 | 3 |
| 12 | 暗い緑 | 1 | 4 | 1 |
| 13 | 紫（マゼンダ） | 6 | 2 | 5 |
| 14 | 灰 | 5 | 5 | 5 |
| 15 | 白 | 7 | 7 | 7 |

## 7.4 画面モードごとの色表現

### 7.4.1 パレット参照モード（Text、Graphic1〜4、Graphic6、Multicolor）

これらのモードでは、画面データの色コード（4 ビット）がそのままパレット番号の下位 4 ビットになり、パレットセットは 0 である。すなわち、エントリ 0〜15 の色が表示される。

色コード 0 は特別で、通常は「透明」として扱われ、そのドットには周辺色（バックドロップカラー、R#7）が透けて見える。R#8 の TP ビットを 1 にするとこの扱いは解除され、色コード 0 もエントリ 0 の色で表示されるようになる。スプライトについても同じ規則である（第 6 章参照）。

周辺色そのものも、パレット参照モードでは R#7 の下位 4 ビットでエントリ 0〜15 のいずれかを指す。

### 7.4.2 Graphic5（2 ビット色）

Graphic5 では色コードが 2 ビットであり、エントリ 0〜3 が使われる（パレットセットは 0）。スプライトについては、モード 2 の色コード 4 ビットが「左半分 2 ビット／右半分 2 ビット」の 2 組として解釈され、横に並ぶ 2 ドットずつに別々の色が付く。これは、Graphic5 の横 1 ドットがスプライトの半ドットに相当することに対応する従来通りの扱いである。

### 7.4.3 Graphic7 の RGB モード（EPAL = 0、YJK = 0）

Graphic7 では、画面データ 1 バイトが 1 ドットの色を直接表し、パレットは使われない。ビット割り当ては従来（V9938 の SCREEN8）と同じである。

| ビット | 内容 |
|---|---|
| bit7-5 | G（3 ビット） |
| bit4-2 | R（3 ビット） |
| bit1-0 | B（2 ビット） |

内部では各色とも 5 ビットに拡張される。R と G は 3 ビット値の {v[2:0], v[2:1]}、B は 2 ビット値から {v[1:0], v[1]・v[0], v[1:0]} の形で拡張される。周辺色（R#7）もこのモードでは同じ 8 ビットの直接色として解釈される。

なお、スプライトの色はこのモードでもパレット経由である（スプライトの色コード 4 ビットがエントリ 0〜15 を指す）。

これは V9968 の仕様であり、従来の V9938 とは異なる。V9938 では Graphic7 のスプライトはパレットを使わない固定 16 色だったが、V9968 ではスプライトの色も背景と同じ 256 色パレットから決まる。256 色は 16 色ずつの 16 グループ（パレットセット）に分けられ、スプライトはグループを 1 つ選んで、そのグループ内の 16 色で表示される。ただし、各グループの先頭の色（色コード 0）は透明扱いになるため、実質的に使えるのは 15 色である。パレットセットの決まり方については 7.4.7 節と第 6 章を参照されたい。

### 7.4.4 Graphic7 の拡張パレットモード（EPAL = 1）

EPAL = 1 の Graphic7 では、画面データ 1 バイトがそのままパレット番号（0〜255）として解釈され、256 色すべてを同時に使える。この場合、周辺色（R#7）も 8 ビットのパレット番号として解釈される。

### 7.4.5 YJK モード（R#25 の YJK = 1）

YJK モードは、Graphic7 の画面データを輝度 Y と色差 J、K で解釈するモードである（従来の SCREEN10〜12 相当）。4ドットを 1 組として、J と K が組内のドットで共有される。各ドットの Y は 5 ビット、J と K は符号付き 6 ビット（-32〜31）である。RGB への変換は次の式で行われ、各色とも 0〜31 の範囲にクランプされる。

```
R = Y + J
G = Y + K
B = (5×Y − (2×J + K) + 2) ÷ 4
```

画面データのバイト配置（どのバイトが Y／J／K を持つか）は第 5 章（Graphic7-YJK）を参照されたい。YJK モードでも周辺色とスプライトの色はパレットで決まる。

### 7.4.6 YAE モード（R#25 の YAE = 1）

YAE は YJK モードの一種で、ドットごとに「YJK 変換した色」と「パレットの色」のどちらで表示するかを切り替えられるモードである（従来の SCREEN11 相当）。どちらの色を使うかは画面データ側のフラグで決まる。データ形式の詳細は第 5 章（Graphic7-YAE）を参照されたい。

### 7.4.7 スプライトの色

スプライトの色コード（4 ビット）は、パレットセットと組み合わされて 8 ビットのパレット番号になる。

| スプライトモード | パレットセットの決まり方 |
|---|---|
| モード 1 | 常に 0（エントリ 0〜15） |
| モード 2（EPAL = 0） | 常に 0（エントリ 0〜15） |
| モード 2（EPAL = 1） | アトリビュート第 4 バイトの下位 4 ビット（6.4.4 参照） |
| モード 3 | アトリビュートのパレットセットフィールド（6.5.1 参照） |

モード 3 では、これに加えてプレーン単位の半透明ブレンド（透過率 0／25／50／75%）がパレットで決まった表示色に対して適用される（6.5.1 参照）。

## 7.5 サンプルプログラム：パレットの書き換えとカラーバー

サンプルプログラム pal_demo.asm は、パレットの 16 エントリすべてを自作の色の並び（ランプ）に書き換え、画面に 16 本の縦バーを描いて各エントリの色をそのまま表示する例です。実行結果を図 7.1 に示します。初期パレットとは全く異なる色の流れになっていることから、パレットの書き換えが効いていることが分かります。



図 7.1 pal_demo.asm の実行結果（書き換えたパレットによるカラーバー）

パレットへの書き込みは、R#16 に開始エントリ番号を設定し、パレットポート（Port#2）へ 2 バイトずつ書くだけです。エントリ番号は 1 エントリ書くごとに自動的に増えます（→7.2.1）。

```
        ; ---- rewrite palette entries 0..15
        ld      a, 0            ; R#16 = 0 (start entry)
        ld      b, 16
        call        vdp_reg
        ld      hl, palette_table
        ld      b, 32           ; 16 entries x 2 bytes
palette_loop:
        ld      a, [hl]
```

```
        out     [0x9a], a       ; palette port (auto-increments)
        inc     hl
        djnz        palette_loop
```

パレットデータは、1 バイト目に R と B、2 バイト目に G を入れた 2 バイト形式（EPAL = 0）です。本サンプルでは、黒から青・紫・赤・黄・緑を経て白へ至るランプにしています。

```
        db      0x00, 0x00      ;  0: R=0 G=0 B=0 (black)
        db      0x13, 0x00      ;  1: R=1 G=0 B=3
        db      0x24, 0x00      ;  2: R=2 G=0 B=4
        db      0x35, 0x01      ;  3: R=3 G=1 B=5
        ;  （中略）
        db      0x77, 0x07      ; 15: R=7 G=7 B=7 (white)
```

バーの描画には HMMV を使い、バー i を色 i で塗ります（バーの描画コードはソースファイル参照）。色 0 はグラフィックモードでは透明色として扱われ、背景色（パレット 0 番 = 黒）がそのまま見えるため、左端のバー 0 は背景に溶け込んでいます。

---

本章のパレット書き込み形式と RAM 構造は検証済みレジスタノートおよび RTL（vdp_cpu_interface.v、vdp_color_palette.v、vdp_color_palette_ram.v）に基づく。初期色の表は MSX2 テクニカルハンドブック（テクハン wiki 版）の初期パレット表と照合した。リセット時の初期化範囲（先頭 16 エントリのみ、16〜255 は不定）は著者確認事項に基づく。

# 第 8 章 VDP コマンド

VDP コマンドは、VRAM 上の矩形転送・塗りつぶし・直線描画・色サーチなどをハードウェアで実行する機能です。CPU はパラメータをレジスタに設定してコマンドを起動するだけでよく、描画処理の多くを VDP に任せられます。本章では、まずコマンド共通の実行モデルを説明し、次に各コマンドを個別に解説します。

コマンド関係のレジスタ（R#32〜R#58）のビット構成そのものは第 3 章で説明しました。本章では「コマンドをどう使うか」に絞って記述します。なお、本章の従来コマンド（V9938/V9958 からある 13 コマンド）の動作説明は『MSX2 テクニカルハンドブック』（テクハン wiki 版）を参照し、V9968 の実装（RTL）と照合したうえで記述しています。

## 8.1 コマンドの実行モデル

### 8.1.1 コマンド一覧

V9968 で実行できるコマンドは次の 16 種類です。R#46（コマンドレジスタ）の上位 4 ビットがコマンドコード、下位 4 ビットがロジカルオペレーションの指定です。

| コマンド | 区分 | 転送元 | 転送先 | 単位 | R#46 上位 4 ビット |
|---|---|---|---|---|---|
| HMMC | 高速転送 | CPU | VRAM | バイト | 1111 |
| YMMM | 高速転送 | VRAM | VRAM | バイト | 1110 |
| HMMM | 高速転送 | VRAM | VRAM | バイト | 1101 |
| HMMV | 高速転送 | VDP | VRAM | バイト | 1100 |
| LMMC | 論理転送 | CPU | VRAM | ドット | 1011 |
| LMCM | 論理転送 | VRAM | CPU | ドット | 1010 |
| LMMM | 論理転送 | VRAM | VRAM | ドット | 1001 |
| LMMV | 論理転送 | VDP | VRAM | ドット | 1000 |
| LINE | 描画 | VDP | VRAM | ドット | 0111 |

| コマンド | 区分 | 転送元 | 転送先 | 単位 | R#46 上位 4 ビット |
|---|---|---|---|---|---|
| | | | | | |
| SRCH | 探索 | VDP | VRAM | ドット | 0110 |
| PSET | 描画 | VDP | VRAM | ドット | 0101 |
| POINT | 読み出し | VRAM | VDP | ドット | 0100 |
| LRMM | 論理転送（新設） | VRAM | VRAM | ドット | 0011 |
| LFMC | フォント描画（新設） | CPU | VRAM | ドット | 0010 |
| LFMM | フォント描画（新設） | VRAM | VRAM | ドット | 0001 |
| STOP | 停止 | — | — | — | 0000 |

LRMM、LFMC、LFMM は V9968 で新設されたコマンドです。これらは**拡張モード（R#21 の V58 ビットが 0）のときだけ有効**で、V9958 互換モード（V58=1）では STOP コマンドとして扱われます（→第 3 章）。なお、従来資料にある「R#20 の ECOM ビット」は、かつて新コマンドの有効化のために設けられたビットでしたが、現在の設計では V58 に統合されており、独立した ECOM ビットは存在しません。

### 8.1.2 パラメータレジスタ

コマンドのパラメータは、起動前に R#32〜R#58 へ設定します。主なものの役割は次のとおりです（ビット構成の詳細は第 3 章）。

| レジスタ | 名称 | 内容 |
|---|---|---|
| R#32, R#33 | SX | 転送元 X 座標（9 ビット） |
| R#34, R#35 | SY | 転送元 Y 座標（10 ビット） |
| R#36, R#37 | DX | 転送先 X 座標（9 ビット） |
| R#38, R#39 | DY | 転送先 Y 座標（10 ビット） |

| レジスタ | 名称 | 内容 |
|---|---|---|
| R#40, R#41 | NX | X 方向のドット数（10 ビット、0 は 1024 として扱う） |
| R#42, R#43 | NY | Y 方向のドット数（10 ビット、0 は 1024 として扱う） |
| R#44 | CLR | カラーコード／CPU との転送データ |
| R#45 | ARG | 引数レジスタ（MAJ、DIX、EQ、DIY、MXS、MXD、XHR/FG4） |
| R#46 | CMR | コマンドコード（書き込みで実行開始） |
| R#47, R#48 | VX | 回転ベクトル X（LRMM 用、符号付き 16 ビット、8.8 固定小数点） |
| R#49, R#50 | VY | 回転ベクトル Y（LRMM 用、符号付き 16 ビット、8.8 固定小数点） |
| R#51, R#52 | WSX | ウィンドウ開始 X 座標（LRMM 用） |
| R#53, R#54 | WSY | ウィンドウ開始 Y 座標（LRMM 用） |
| R#55, R#56 | WEX | ウィンドウ終了 X 座標（LRMM 用） |
| R#57, R#58 | WEY | ウィンドウ終了 Y 座標（LRMM 用） |

LFMC／LFMM で使う前景色は R#44、背景色は R#12 に設定します（→8.15 節、8.16 節）。フォント描画専用の色レジスタはありません。

座標はページの区別なく、VRAM 全体を 1 つの大きな座標系として扱います。X 座標は 0〜511、Y 座標は 0〜1023 の範囲です。矩形を指定するコマンドでは、(SX, SY)または(DX, DY)を基準点とし、NX、NY で各辺のドット数を指定します。

引数レジスタ R#45 の各ビットは次の意味を持ちます。

| ビット | 名称 | 意味 |
|---|---|---|
| 0 | M | LINE の長軸方向（0：Y 軸が長軸、1：X 軸が長軸） |

| ビット | 名称 | 意味 |
|---|---|---|
| | AJ | |
| 2 | DIX | X 方向の転送方向（0：右方向、1：左方向）。SRCH では探索方向 |
| 1 | EQ | SRCH の一致条件（0：指定色と等しい点を探す、1：指定色と異なる点を探す） |
| 3 | DIY | Y 方向の転送方向（0：下方向、1：上方向） |
| 4 | MXS | 転送元メモリ選択（V9968 では拡張 RAM が廃止されているため、選択対象は VRAM のみ） |
| 5 | MXD | 転送先メモリ選択（同上） |
| 7 | FG4 | 拡張モード時のみ有効。1 のとき、画面モード指定にかかわらず VDP コマンドが Graphic4（SCREEN 5）として動作する（8.1.3 節参照） |
| 6 | XHR | 拡張モード時のみ有効。LRMM で画素を縦長画素として扱い、Graphic5、6 の回転表示を正しいアスペクトで見られるようにする（8.14 節参照）。互換モード時の MXC については第 3 章を参照 |

FG4（R#45 bit7、拡張モード時のみ有効）を 1 にすると、VDP コマンドは現在の画面モード指定にかかわらず Graphic4（SCREEN 5）として動作します。SCREEN 1～4 のように、通常はコマンドの対象にならない表示モードであっても、コマンド側は Graphic4 の座標・色形式で処理されます。主な用途は SpriteMode3 のスプライトパターンへの描画で、パターンのアドレス計算が Graphic4 形式であるため、表示は SCREEN 6、7、8 のままパターンデータへ線などを描けます（→6.5.2 節）。なお、表示自体が Graphic4（SCREEN 5）の場合は、もともとコマンドが Graphic4 として動作するため、FG4 の有無で動作は変わりません。

MXS/MXD による拡張 RAM の選択は、V9938/V9958 の仕様にある機能ですが、V9968 では拡張 RAM 自体が廃止されており、選択できるメモリは VRAM のみです。

### 8.1.3 実行開始と終了の確認

コマンドの実行手順は次のとおりです。

1. 必要なパラメータを R#32～R#45（および LRMM では R#47～R#58）に設定する
2. コマンドコードとロジカルオペレーションを R#46 に書き込むと、実行が開始される
3. 実行中はステータスレジスタ S#2 の CE ビット（ビット 0）が 1 になる
4. 実行が終了すると CE ビットが 0 に戻る

CPU は CE ビットをポーリングして終了を待ちます。コマンド実行中に別のコマンドを起動してはいけません。また、コマンド完了割り込み（R#20 の CEIE ビットで許可）を使うと、ポーリングなしに終了を知ることもできます（→第 10 章）。

実行中のコマンドを途中で止めるには、STOP コマンド（R#46 に 00h を書き込む）を実行します。

HMMC、LMMC、LMCM のように CPU とデータをやり取りするコマンドでは、S#2 の TR ビット（ビット 7）が転送レディを示します。CPU は TR が 1 のときだけ R#44（書き込み）または S#7（読み出し）にアクセスします。各コマンドの節で手順を説明します。

なお、コマンドの動作が保証されるのはビットマップ系の画面モード（GRAPHIC 4～GRAPHIC 7 相当）のときです。ただし、拡張モードで FG4=1 にすると、表示が他の画面モード（SCREEN 1～4 を含む）であってもコマンドは Graphic4 として動作するため、例えば表示は SCREEN 6、7、8 のまま、SpriteMode3 のスプライトパターン（Graphic4 形式）へ VDP コマンドで描画できます。

また、FIL=1（フラットインターレース、R#20 ビット 5。第 3 章参照）のときは、page0 と page1 を上下に結合した画面が 1 枚の高解像度画面として扱われ、VDP コマンドもこの結合した画面を対象に動作します。

### 8.1.4 ロジカルオペレーション

ドット単位で書き込むコマンドでは、R#46 の下位 4 ビットで、転送元（ソース）の色 SC と転送先（デスティネーション）の色 DC の間の演算を指定できます。

| コード | 名称 | 演算 |
|---|---|---|
| 0000 | IMP | DC = SC |

| コード | 名称 | 演算 |
|---|---|---|
| | | |
| 0001 | AND | DC = SC AND DC |
| 0010 | OR | DC = SC OR DC |
| 0011 | EOR | DC = SC XOR DC |
| 0100 | NOT | DC = NOT SC |
| 0101〜0111 | （予約） | IMP と同じ（SC をそのまま書き込む） |
| 1000 | TIMP | SC=0 なら DC そのまま、それ以外は DC = SC |
| 1001 | TAND | SC=0 なら DC そのまま、それ以外は DC = SC AND DC |
| 1010 | TOR | SC=0 なら DC そのまま、それ以外は DC = SC OR DC |
| 1011 | TEOR | SC=0 なら DC そのまま、それ以外は DC = SC XOR DC |
| 1100 | TNOT | SC=0 なら DC そのまま、それ以外は DC = NOT SC |
| 1101〜1111 | （予約） | TIMP と同じ（透明付き IMP として動作する） |

「T」の付く演算では、ソースの色が 0 のドットは書き換えの対象にならず、デスティネーションがそのまま残ります。図形の色の部分だけを重ね合わせる用途に使います。予約コードの扱い（0101〜0111 は IMP 相当、1101〜1111 は TIMP 相当）は V9968 の実装で確認したものです。

なお、V9968 の実装では、T 系演算の透明判定はソースの色コード全体が 0 かどうかで行われます。Graphic 6/7（4 ビット/ドット）で色を指定する際は、下位ニブルに色コードを入れてください（alexmoncks フォークのドキュメントによる実装注記）。

### 8.1.5 バイト単位コマンドの注意

HMMC、YMMM、HMMM、HMMV の高速転送コマンドはデータをバイト単位で扱います。このため、1 バイトに複数ドットが詰まる画面モードでは、X 座標や NX の下位ビットが参照されません（GRAPHIC 4 と 6 では下位 1 ビット、GRAPHIC

5 では下位 2 ビット）。ドット単位の位置合わせが必要な場合は、論理転送コマンドを使ってください。

### 8.1.6 コマンド終了時のレジスタの状態

コマンドの実行にともない、SX、SY、DX、DY、NX、NY などのパラメータレジスタは内部の進行に合わせて更新されます。終了時点の値は「どこまで処理したか」を反映したものになり、設定した値がそのまま残るとは限りません（たとえば NY は残り行数、SY/DY は最終行の次の座標を指します）。次のコマンドでは、必要なパラメータを毎回設定し直してください。詳細な更新規則は『MSX2 テクニカルハンドブック』第 6 章の表 4.7 を参照してください。

## 8.2 HMMC（CPU→VRAM 高速転送）

| 項目 | 内容 |
|---|---|
| コマンドコード | F0h |
| 転送単位 | バイト |
| ロジカルオペレーション | 指定不可 |
| 使用パラメータ | DX、DY、NX、NY、CLR、ARG（DIX、DIY） |

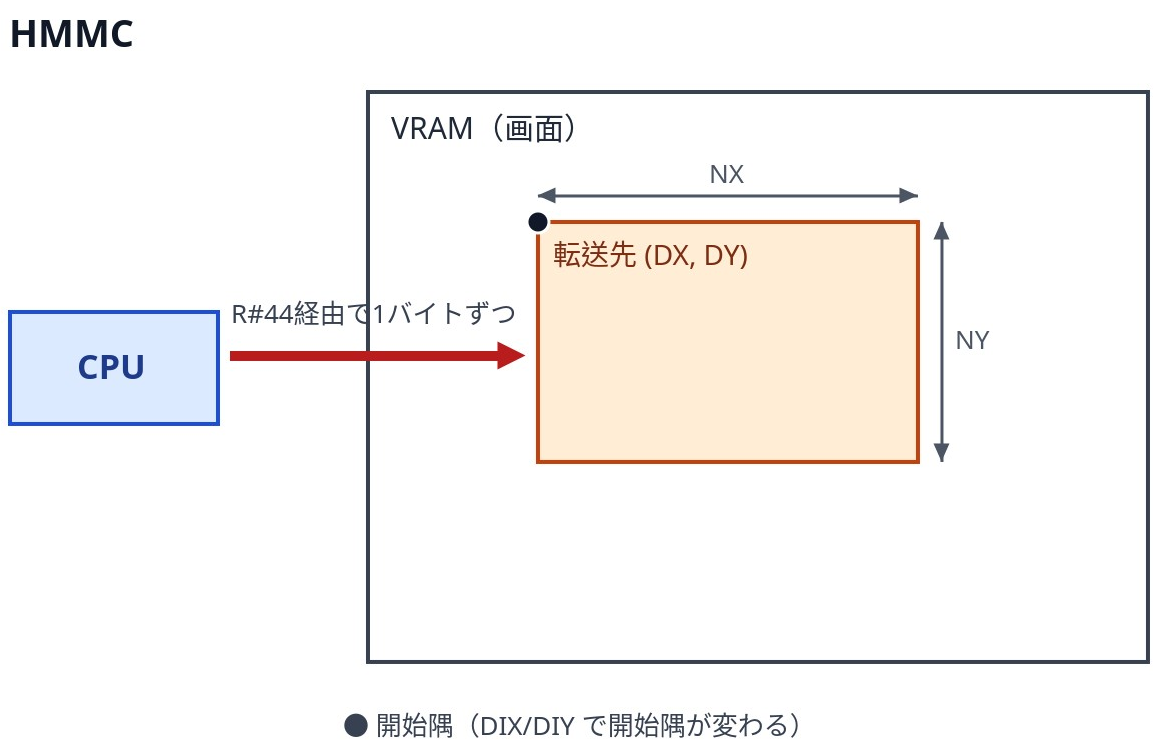


図 8.1 HMMC の座標関係

CPU から VRAM の指定領域へ、データをバイト単位で転送します。

手順は次のとおりです。まずパラメータを設定し、転送するデータの**最初の 1 バイトだけを R#44 に書き込んでから、**R#46 に F0h を書き込みます。VDP は R#44 のデータを受け取って VRAM に書き込み、以後は CPU からのデータ待ちになります。CPU は S#2 の TR ビットが 1 になるたびに、次の 1 バイトを R#44 へ書き込みます。すべてのデータを送り終え、CE ビットが 0 になれば転送完了です。

## 8.3 YMMM（Y 方向 VRAM→VRAM 高速転送）

| 項目 | 内容 |
|---|---|
| コマンドコード | E0h |
| 転送単位 | バイト |
| ロジカルオペレーション | 指定不可 |
| 使用パラメータ | SY、DX、DY、NY、ARG（DIX、DIY） |

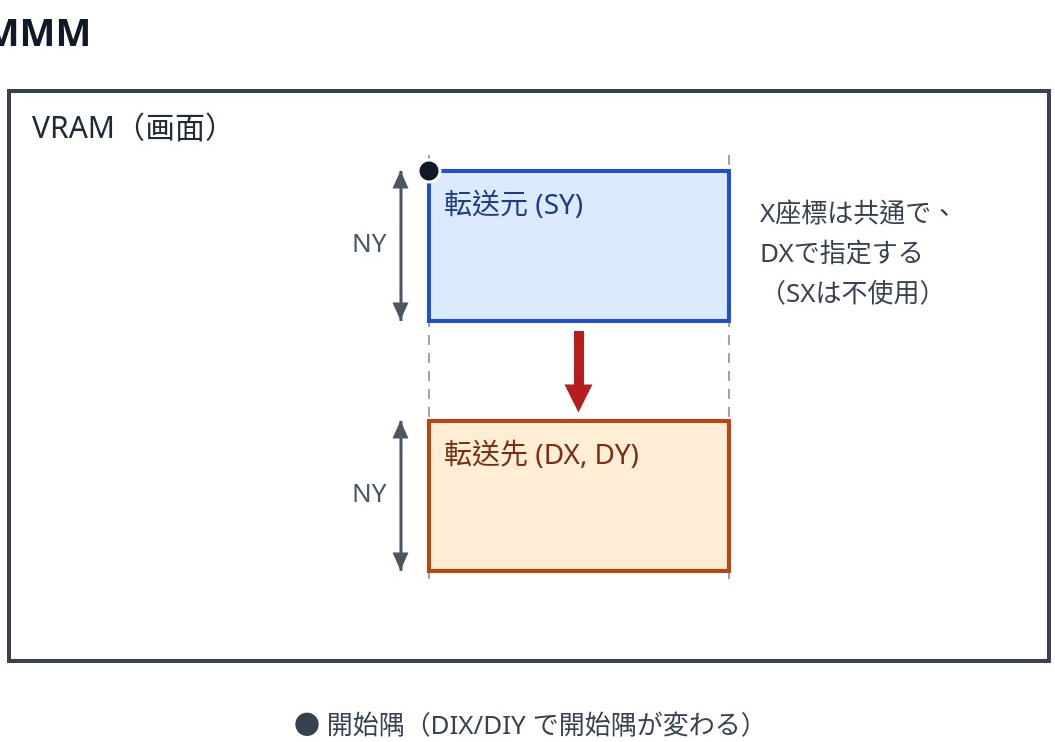



図 8.2 YMMM の座標関係

VRAM 内のデータを、同じ X 座標のまま Y 方向へバイト単位で転送します。転送元と転送先で X 座標は共通であり、DX で指定します（SX は使用しません）。縦方向のスクロールや、同じ列の複写に使います。転送は Y 方向だけに限られ、NX は使用しません。DIY で転送方向（上下）を選びます。

## 8.4 HMMM（VRAM→VRAM 高速転送）

| 項目 | 内容 |
|---|---|
| コマンドコード | D0h |
| 転送単位 | バイト |
| ロジカルオペレーション | 指定不可 |
| 使用パラメータ | SX、SY、DX、DY、NX、NY、ARG |

### HMMM

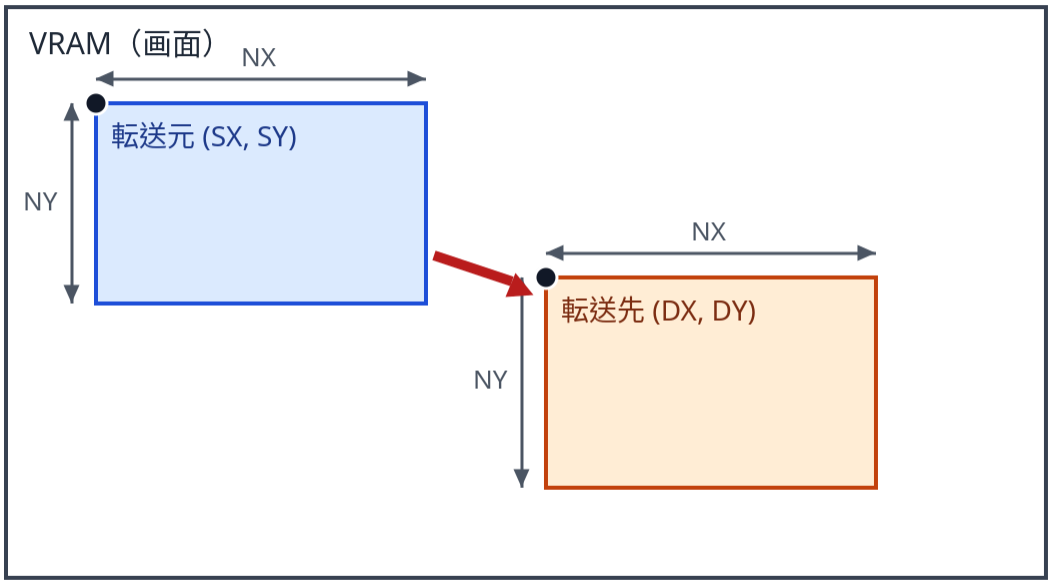



図 8.3 HMMM の座標関係

VRAM の指定領域のデータを、VRAM の他の領域へバイト単位で転送します。バイト単位のため、X 座標の下位ビットが参照されない点は HMMC と同じです（→8.1.5）。

## 8.5 HMMV（矩形の高速塗りつぶし）

| 項目 | 内容 |
|---|---|
| コマンドコード | C0h |
| 転送単位 | バイト |
| ロジカルオペレーション | 指定不可 |
| 使用パラメータ | DX、DY、NX、NY、CLR、ARG |

### HMMV

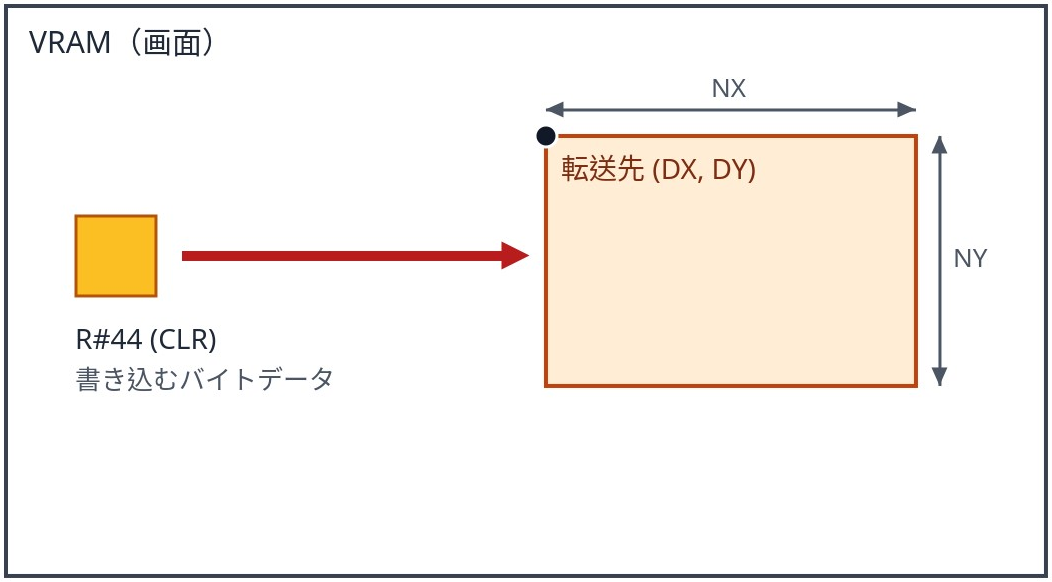



図 8.4 HMMV の座標関係

VRAM の指定領域を、R#44（CLR）に設定したバイトデータで塗りつぶします。CLR は色コードではなく、VRAM に書き込むバイトそのものです。

## 8.6 LMMC（CPU→VRAM 論理転送）

| 項目 | 内容 |
|---|---|
| コマンドコード | B0h＋ロジカルオペレーション |

| 項目 | 内容 |
| --- | --- |
| | |
| 転送単位 | ドット |
| 使用パラメータ | DX、DY、NX、NY、CLR、ARG |

### LMMC

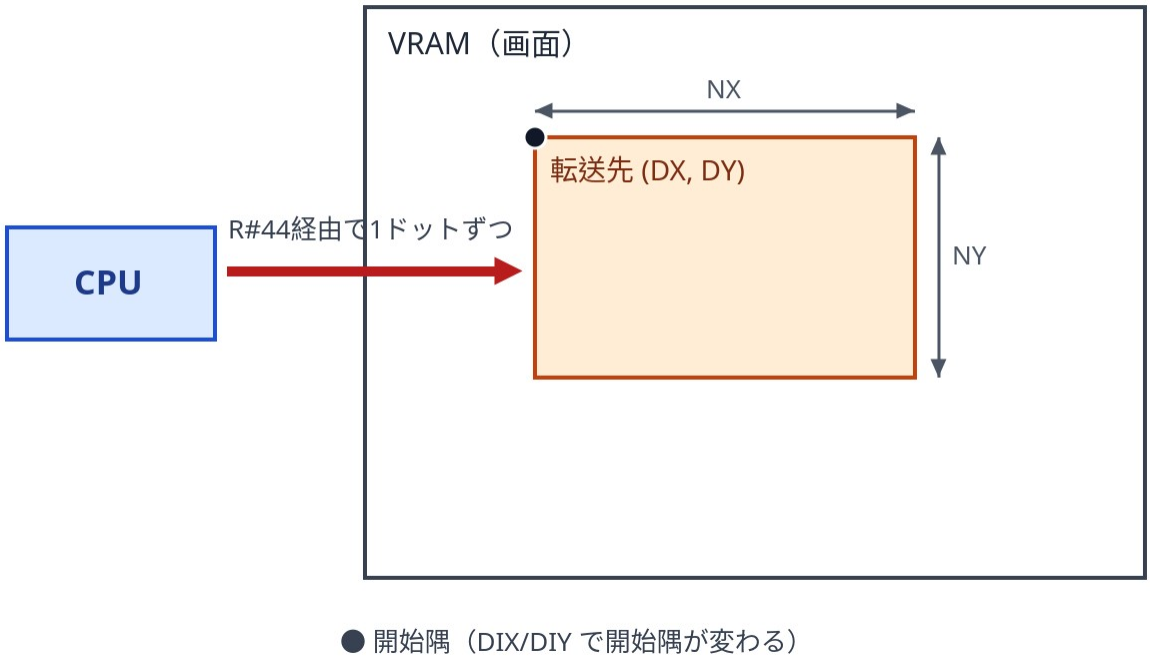



図 8.5 LMMC の座標関係

CPU から VRAM の指定領域へ、データをドット単位で転送します。論理転送コマンドでは、すべての画面モードで「1ドット＝1 バイト」の色コードを CPU が供給します。

手順は HMMC と同様で、最初の 1 バイトを R#44 に書き込んでから R#46 にコマンドを書き込み、以後は TR ビットを見ながら R#44 へ 1ドット分ずつ送ります。

## 8.7 LMCM（VRAM→CPU 論理転送）

| 項目 | 内容 |
| --- | --- |
| コマンドコード | A0h |

| 項目 | 内容 |
| --- | --- |
| | |
| 転送単位 | ドット |
| 使用パラメータ | SX、SY、NX、NY、ARG |

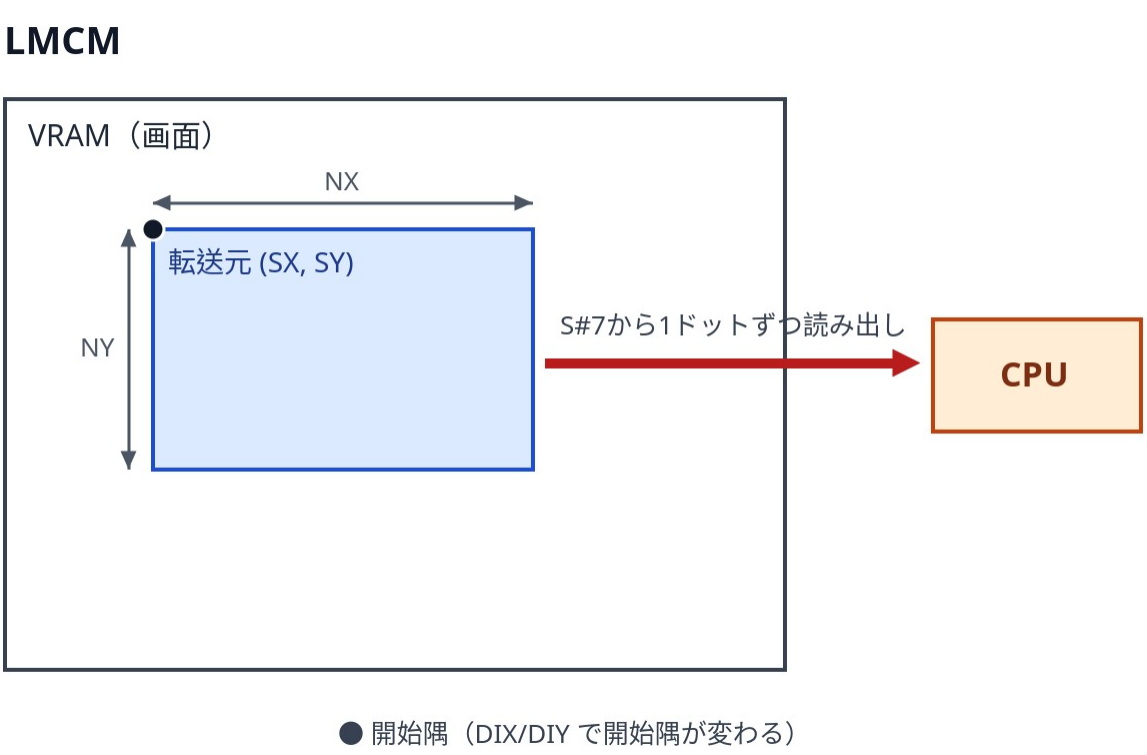



図 8.6 LMCM の座標関係

VRAM の指定領域のデータを、ドット単位で CPU へ転送します。コマンドを起動すると VDP がデータを用意し始めます。CPU は S#2 の TR ビットが 1 のとき、**S#7** から 1 ドット分の色コードを読み出します。CE ビットが 0 になれば全データの転送終了です。

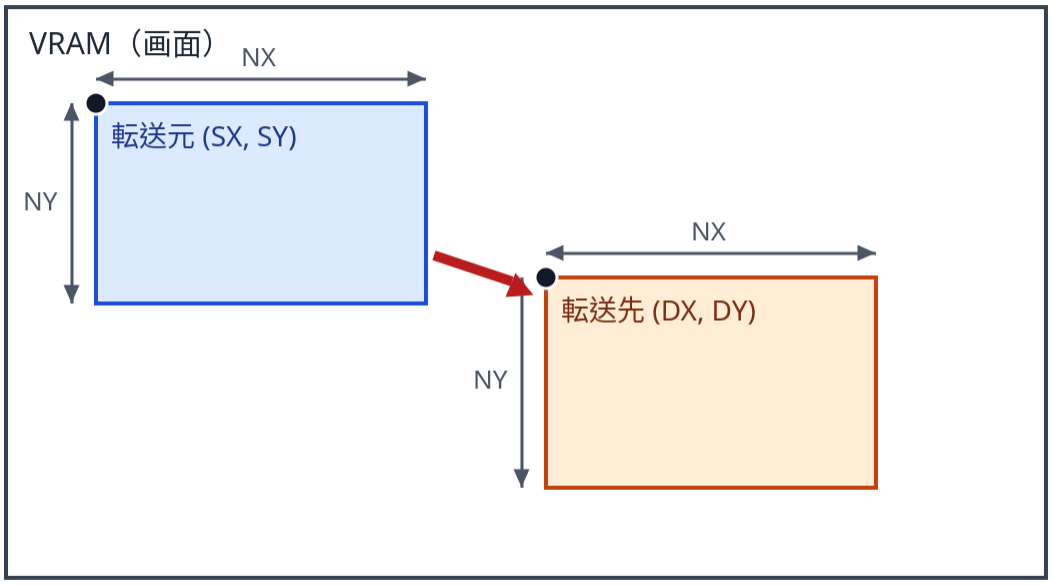
なお、読み出しに先立って S#7 を一度読み、TR ビットを初期化しておくのが従来の作法です。

## 8.8 LMMM（VRAM→VRAM 論理転送）

| 項目 | 内容 |
| --- | --- |
| コマンドコード | 90h＋ロジカルオペレーション |

| 項目 | 内容 |
| --- | --- |
| | |
| 転送単位 | ドット |
| 使用パラメータ | SX、SY、DX、DY、NX、NY、ARG |

### LMMM




● 開始隅（DIX/DIY で開始隅が変わる）

図 8.7 LMMM の座標関係

VRAM の指定領域のデータを、VRAM の他の領域へドット単位で転送します。TIMP などの透明付き演算と組み合わせると、色 0 を透明色とした図形の重ね合わせができます。

## 8.9 LMMV（矩形の論理塗りつぶし）

| 項目 | 内容 |
| --- | --- |
| コマンドコード | 80h＋ロジカルオペレーション |
| 転送単位 | ドット |
| 使用パラメータ | DX、DY、NX、NY、CLR、ARG |

| 項目 | 内容 |
| --- | --- |
| | |

### LMMV

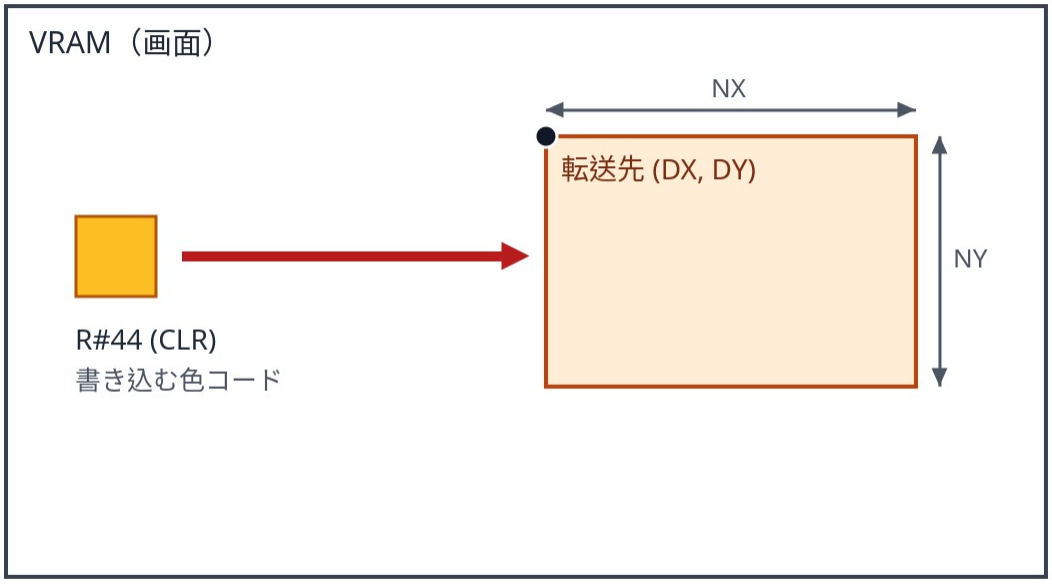


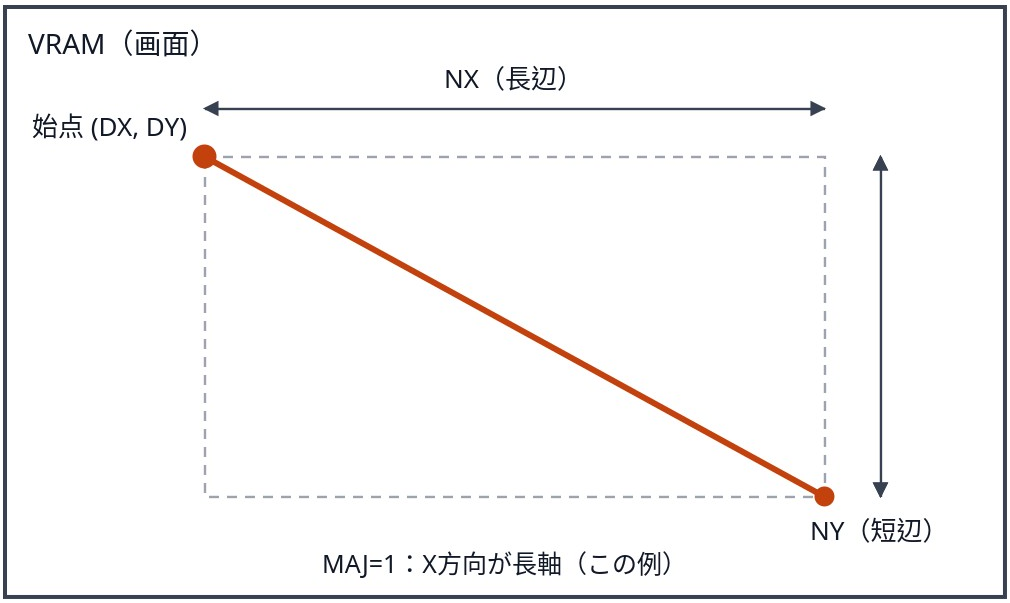

図 8.8 LMMV の座標関係

VRAM の指定領域を、R#44（CLR）の色コードでドット単位に塗りつぶします。既存の VRAM データとのロジカルオペレーションも指定できます。

## 8.10 LINE（直線描画）

| 項目 | 内容 |
| --- | --- |
| コマンドコード | 70h＋ロジカルオペレーション |
| 転送単位 | ドット |
| 使用パラメータ | DX、DY、NX（長辺）、NY（短辺）、CLR、ARG（MAJ、DIX、DIY） |

### LINE




描く色は R#44（CLR）。DIX・DIY で直線が伸びる方向を指定する

図 8.9 LINE の座標関係

(DX, DY)を始点として直線を描きます。直線は、それを対角線とする長方形の長辺・短辺で指定します。**NX には長辺のドット数、NY には短辺のドット数を設定し、ARG の MAJ ビットで長軸の方向を示します**（MAJ=1 なら X 軸方向が長軸、MAJ=0 なら Y 軸方向が長軸）。設定の前に両辺を比較し、長い方を必ず NX（長辺）に入れてください。DIX、DIY は直線が伸びる方向（左右・上下）を指定します。描く色は CLR です。

## 8.11 SRCH（色サーチ）

| 項目 | 内容 |
|---|---|
| コマンドコード | 60h |
| 転送単位 | ドット |
| 使用パラメータ | SX、SY、CLR、ARG（DIX、EQ） |
| 結果 | S#2 の BD ビット、S#8、S#9 |

### SRCH

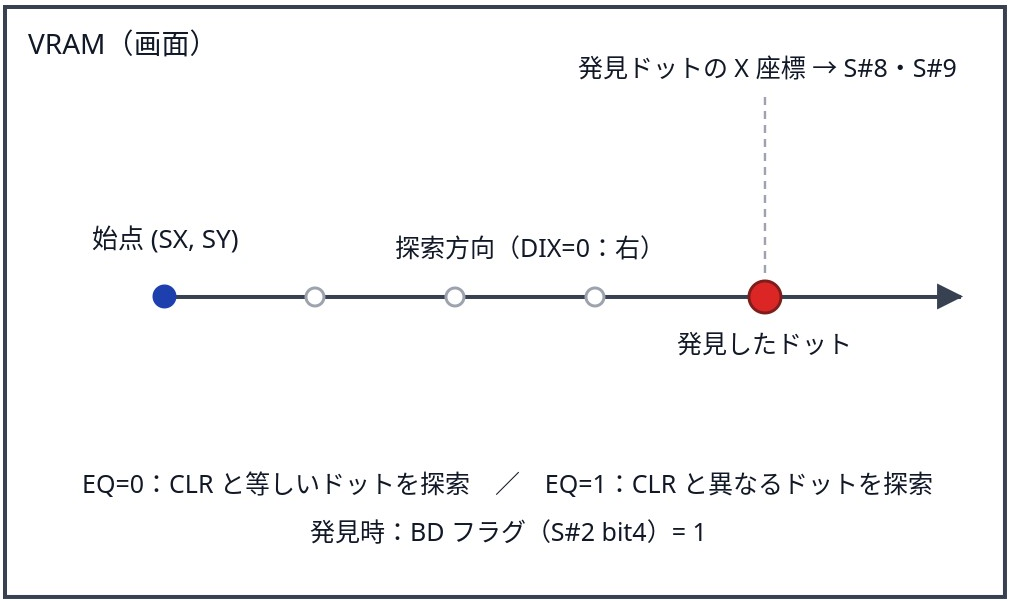



図 8.10 SRCH の座標関係

(SX, SY)から右（DIX=0）または左（DIX=1）に向かって走査し、条件に合う色のドットを探します。EQ=0 なら CLR と等しいドット、EQ=1 なら CLR と異なるドットが「発見」です。コマンドは、目的のドットが見つかるか、画面の端まで探して見つからなかった場合に終了します。

終了後、S#2 の BD ビット（ビット 4）が 1 であれば発見できたことを示し、発見したドットの X 座標が S#8（下位 8 ビット）と S#9（最上位ビット）に格納されます。ペイントルーチンの境界探索などに使います。

## 8.12 PSET（点描画）

| 項目 | 内容 |
|---|---|
| コマンドコード | 50h+ロジカルオペレーション |
| 転送単位 | ドット |
| 使用パラメータ | DX、DY、CLR、ARG |

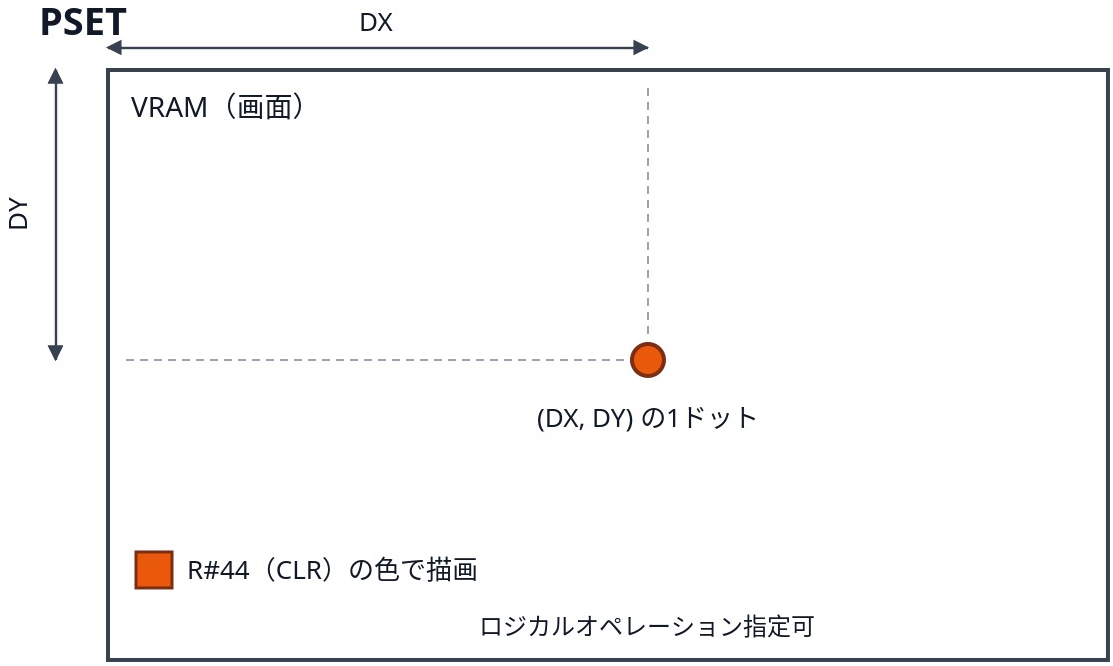



図 8.11 PSET の座標関係

(DX, DY)の 1ドットを CLR の色で描きます。

## 8.13 POINT（色読み出し）

| 項目 | 内容 |
|---|---|
| コマンドコード | 40h |
| 転送単位 | ドット |
| 使用パラメータ | SX、SY |
| 結果 | S#7 |

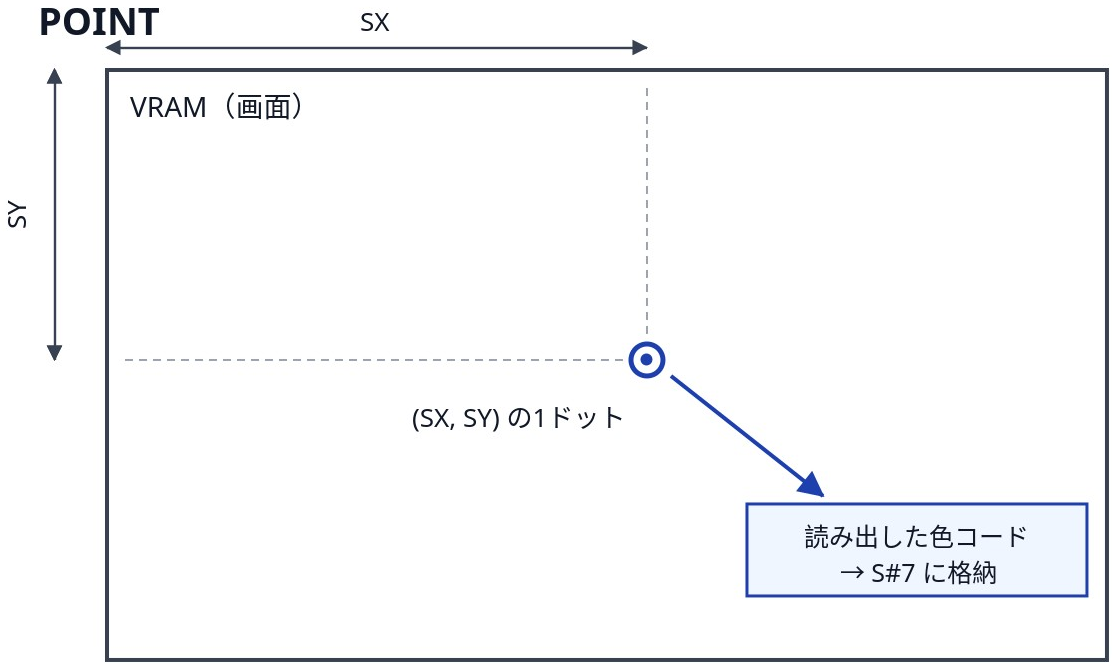



図 8.12 POINT の座標関係

(SX, SY)の 1ドットの色コードを読み出します。コマンド終了後、色コードは S#7 に格納されています。

## 8.14 LRMM（回転・拡大縮小付き論理転送）

| 項目 | 内容 |
|---|---|
| コマンドコード | 30h＋ロジカルオペレーション |
| 転送単位 | ドット |
| 使用パラメータ | SX、SY、DX、DY、NX、NY、CLR、ARG、VX（R#47、R#48）、VY（R#49、R#50）、WSX（R#51、R#52）、WSY（R#53、R#54）、WEX（R#55、R#56）、WEY（R#57、R#58） |
| 使用条件 | 拡張モード（V58=0）のときのみ有効 |

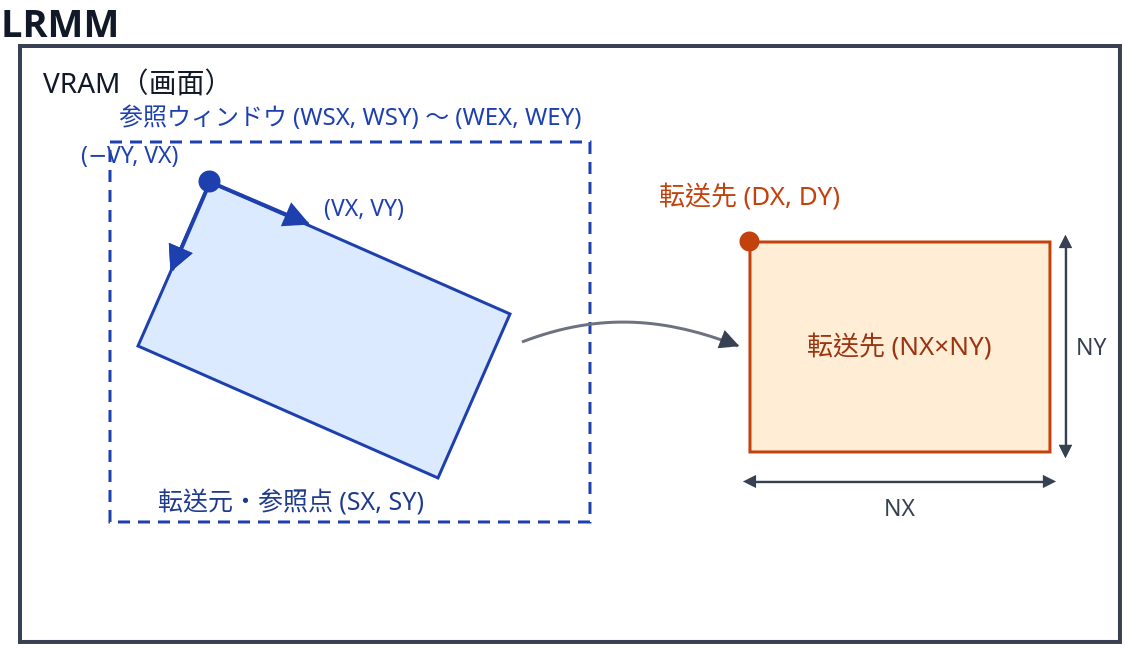



図 8.13 LRMM の座標関係

VRAM から VRAM へ、回転と拡大縮小をかけながらドット単位で転送する、V9968 新設のコマンドです。転送先の各ドットについて、転送元側の「参照点」にあるドットを読み取り、ロジカルオペレーションを経て描画します。

(SX, SY)は参照の開始点です。転送先の先頭ドット(DX, DY)には、(SX, SY)のドットが対応します。(SX, SY)は回転中心そのものではないため、図形の中心で回転させたい場合は、開始点を計算して設定する必要があります。

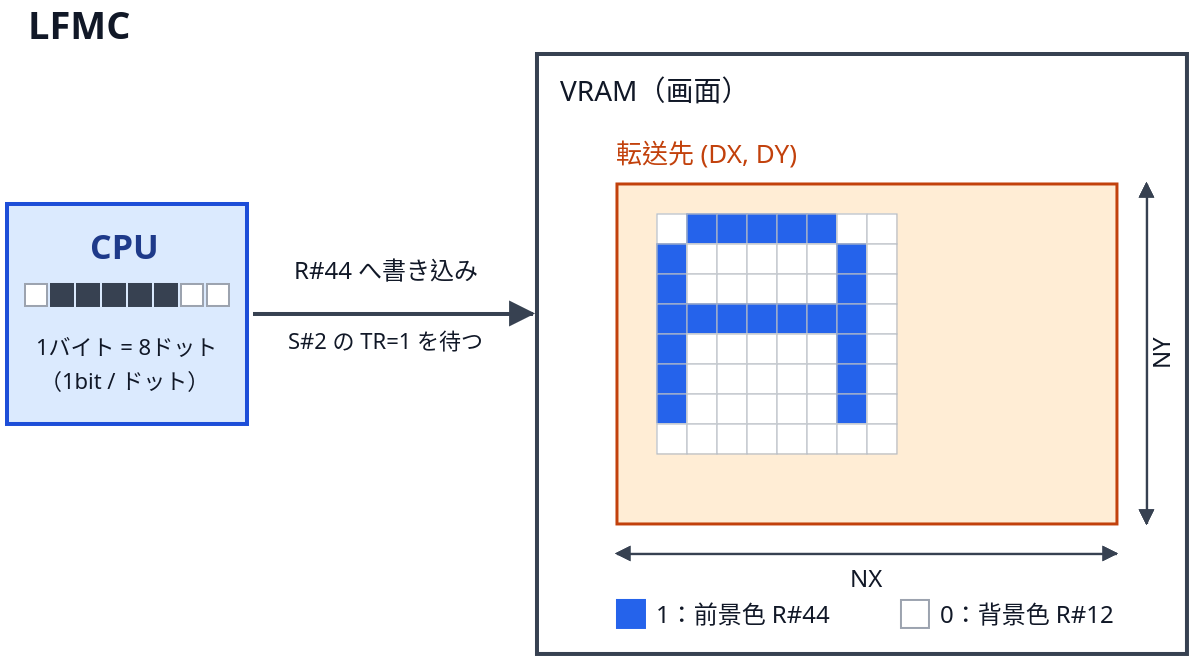
描画の進行に合わせて、参照点は回転ベクトル(VX, VY)ずつ移動します。転送先で X 方向に 1ドット進むごとに参照点は(VX, VY)進み、1 行分が終わって次の行へ移るときは(－VY, VX)進みます。VX、VY は符号付きの固定小数点数（小数部 8ビット）で、回転角 θ に対して VX = INT(256×cosθ)、VY = INT(256×sinθ) と設定します。256 が 1.0 に相当し、ベクトルの大きさが 1.0 より大きいと転送元を速く進むため図形は縮小され、小さいと拡大されます。

転送元には参照ウィンドウ(WSX, WSY)～(WEX, WEY)を指定できます。ウィンドウは開始点・終了点の両端を含む範囲で、参照点が内側にあるドットは転送元から読み取られます。参照点がウィンドウの外側に出た場合、そのドットの転送元画素の代わりに R#44（CLR）の色が使われ、そのうえでロジカルオペレーションを適用した結果が転送先へ描画されます。なお、ウィンドウレジスタ（R#51～R#58）が効くのは LRMM のみで、他のコマンドには影響しません。

引数レジスタ R#45 の bit6（XHR、拡張モード時）を 1 にすると、LRMM は画素を縦長画素として扱います。Graphic5、Graphic6（SCREEN 6、7）の画素は縦長のため、そのまま回転させると表示のアスペクト比が崩れます。XHR は、このような画面モードで回転表示を正しいアスペクトで見られるようにするための仕掛けです。

## 8.15 LFMC（CPU→VRAM フォント描画）

| 項目 | 内容 |
|---|---|
| コマンドコード | 20h＋ロジカルオペレーション |
| 転送単位 | ドット |
| 使用パラメータ | DX、DY、NX、NY、ARG、前景色（R#44）、背景色（R#12） |
| 使用条件 | 拡張モード（V58=0）のときのみ有効 |




入力は 1bit/ドット。描画時に画面モードのビット/ドットへ展開される

図 8.14 LFMC の座標関係

CPU から送られるフォントパターン（ビットマップ）を展開しながら VRAM へ描く、V9968 新設のコマンドです。入力データは 1 ビット/ドットで、描画時に画面モードのビット/ドットへ展開されます。ビットが 1 のドットは前景色、0 のドットは背景色で描かれます。LMMC がドット単位で色コードを送るのに対し、LFMC は 8 ドット分を 1 バイトに詰めて送るため処理が軽くなりますが、描けるのは前景色と背景色の 2 値だけです。

手順は次のとおりです。

1. 実行前に、前景色を R#44 へ、背景色を R#12 へ設定する

2. パラメータ（DX、DY、NX、NY、ARG）を設定し、R#46 にコマンドコードを書き込んで開始する
3. 以後は、S#2 の TR ビットが 1 になるのを待ってから、パターンデータを R#44 へ 1 バイトずつ書き込む。1 バイトは水平 8 ドット分で、MSB（ビット 7）側から消費される
4. CE ビットが 0 になれば描画完了

NX が 8 の倍数でない場合、端数のドットも 1 バイトを MSB 側から消費し、余ったビットは捨てられます。ARG の DIX、DIY も効き、DIX=1 で左右反転、DIY=1 で上下反転します（LMMC と同様）。

## 8.16 LFMM（VRAM→VRAM フォント描画）

| 項目 | 内容 |
|---|---|
| コマンドコード | 10h+ロジカルオペレーション |
| 転送単位 | ドット |
| 使用パラメータ | フォントアドレス（R#32〜R#34）、DX、DY、NX（文字数）、NY、ARG、前景色（R#44）、背景色（R#12） |
| 使用条件 | 拡張モード（V58=0）のときのみ有効 |

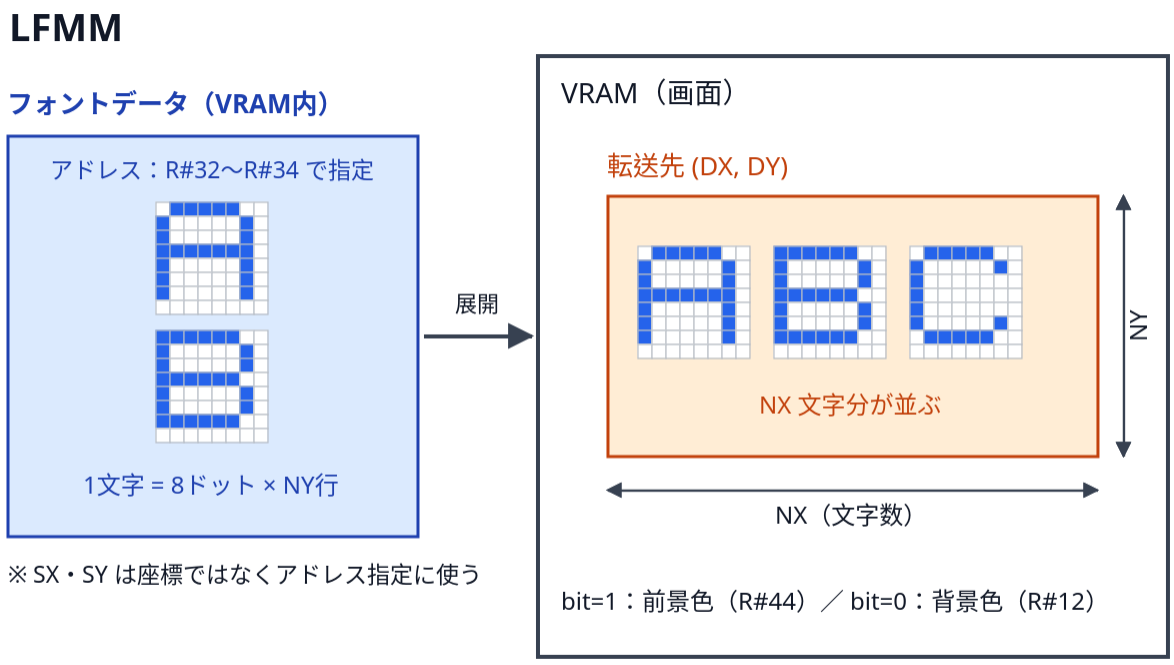



図 8.15 LFMM の座標関係

VRAM 内にあるフォントデータを読み取りながら展開し、VRAM の他の領域へ描くコマンドです。LFMC と同様、入力データは 1 ビット/ドットで、描画時に画面モードのビット/ドットへ展開され、ビットが 1 のドットは前景色（R#44）、0 のドットは背景色（R#12）で描かれます。

LFMM では、SX、SY は座標ではなくフォントデータのアドレス指定に使います。R#32 に A7〜A0、R#33 に A15〜A8、R#34 に A17〜A16 を設定し、3 バイトのリトルエンディアンで 256KB 空間内のフォントデータの先頭アドレスを指定します。R#35 は使用しないため、0 を書き込んでください。

フォントデータは、1 文字が横 8 ドット×NY 行のパターンで、1 行分が 1 バイト（8 ドット分、MSB 側から消費）です。NX はドット数ではなく**文字数**を指定します。描画は文字単位で進み、1 文字分の NY 行を描き終えてから次の文字へ移ります。転送先には、8×NY ドットの文字が NX 文字分並びます。漢字 ROM の全角文字を描く場合は、NY=16、NX=2 とし、1 文字ごとにアドレスを指定して 1 回ずつ実行する形になります。ARG の DIX、DIY も効きます。

文字列表示のように、VRAM 内のフォントを繰り返し描く用途に向きます。

## 8.17 STOP（コマンド停止）

| 項目 | 内容 |
|---|---|
| コマンドコード | 00h |

実行中のコマンドを中断します。パラメータは不要で、R#46 に 00h を書き込むだけで動作します。中断した時点までの描画結果は VRAM に残ります。

なお、拡張モードでは、従来 STOP として扱われていたコード 0011、0010、0001 がそれぞれ LRMM、LFMC、LFMM に割り当てられています（→8.1.1）。互換モードではこれらも STOP として扱われます。

## 8.18 サンプルプログラム：図形の描画とコピー

サンプルプログラム `cmd_demo.asm` は、SCREEN 5（Graphic 4）で VDP コマンドだけを使って図形を描く例です。処理の流れは、(1) HMMV で画面全体と矩形を塗りつぶし、(2) LINE で矩形に枠と対角線を引き、(3) LMMM で完成した矩形を別の場所へコピーする、というものです。実行結果を図 8.16 に示します。右下に矩形が複製されていることが、LMMM の動作をそのまま表しています。

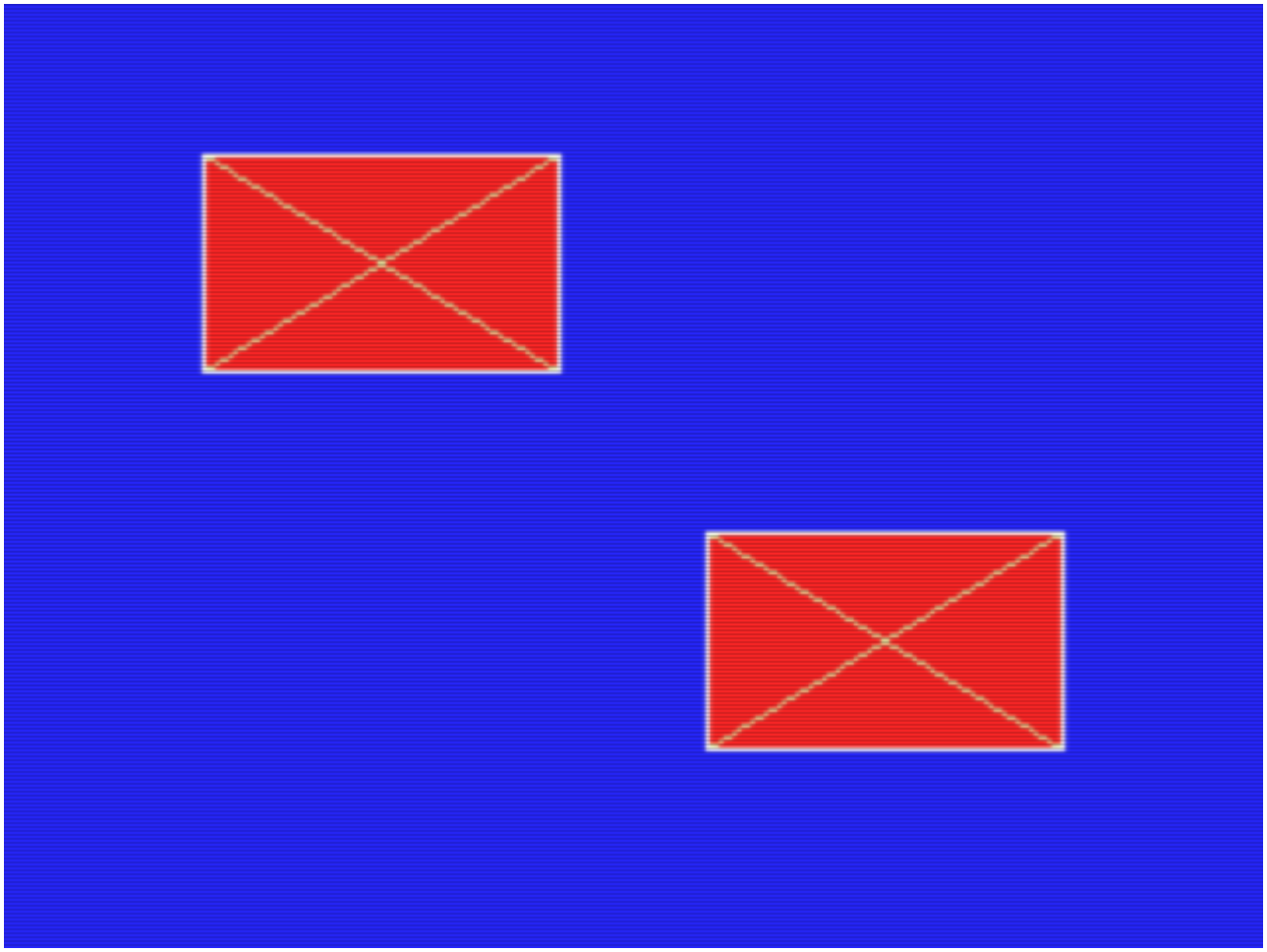


図 8.16 cmd_demo.asm の実行結果（HMMV、LINE、LMMM）

矩形の塗りつぶし（HMMV）の部分は次のとおりです。パラメータを R#36〜R#45 に設定し、最後に R#46 へコマンドコード C0h を書き込むと実行が始まります。CLR には色コードではなく書き込むバイトそのものを指定するため、色 8 で塗る場合は両ニブルに 8 を入れた 88h を設定します（→8.5）。

```
; ---- 2. HMMV: fill a box (32,24)-(111,79) with color 8 (red)
ld      hl, 32          ; DX = 32
ld      b, 36
call        vdp_reg16
ld      hl, 24          ; DY = 24
ld      b, 38
call        vdp_reg16
ld      hl, 80          ; NX = 80
ld      b, 40
call        vdp_reg16
ld      hl, 56          ; NY = 56
ld      b, 42
```

```
call        vdp_reg16
ld      a, 0x88         ; CLR: color 8 in both nibbles
ld      b, 44
call        vdp_reg
ld      a, 0xc0         ; HMMV (ARG is still 0)
ld      b, 46
call        vdp_reg
call        vdp_wait
```

対角線（LINE）では、**NX に長辺、NY に短辺のドット数を入れる**点が要点です。対角線 1 は始点 (32, 24) から X 方向に 79 ドット、Y 方向に 55 ドット進む線なので、長辺は X 側の 79 です。ARG では長軸の向き（MAJ）と、線が伸びる方向（DIX、DIY）を指定します（→8.10）。

```
; ---- 3. LINE: cross (X) inside the box, color 11
; diagonal 1: (32,24) -> (111,79)
ld      hl, 32          ; DX = 32
ld      b, 36
call        vdp_reg16
ld      hl, 24          ; DY = 24
ld      b, 38
call        vdp_reg16
ld      hl, 79          ; NX = 79 (long side)
ld      b, 40
call        vdp_reg16
ld      hl, 55          ; NY = 55 (short side)
ld      b, 42
call        vdp_reg16
ld      a, 11           ; CLR = 11
ld      b, 44
call        vdp_reg
ld      a, 0x00         ; ARG: MAJ=0 (X is the long axis)
ld      b, 45
call        vdp_reg
ld      a, 0x70         ; LINE, LOP = IMP
ld      b, 46
call        vdp_reg
call        vdp_wait
```

対角線 2（DIX = 1 で左方向へ伸びる）と、枠の 4 辺も同様に LINE で描きます（ソースファイル参照）。

最後に、描いた矩形を LMMM で (144, 120) へコピーします。転送元と転送先の両方の座標と大きさを設定し、R#46 へ 90h（LOP = IMP）を書き込みます（→8.8）。

```
; ---- 5. LMMM: copy the box (32,24,80x56) to (144,120)
ld      hl, 32          ; SX = 32
ld      b, 32
call        vdp_reg16
ld      hl, 24          ; SY = 24
ld      b, 34
call        vdp_reg16
ld      hl, 144         ; DX = 144
ld      b, 36
call        vdp_reg16
ld      hl, 120         ; DY = 120
ld      b, 38
call        vdp_reg16
ld      hl, 80          ; NX = 80
ld      b, 40
call        vdp_reg16
ld      hl, 56          ; NY = 56
ld      b, 42
call        vdp_reg16
ld      a, 0            ; ARG = 0
ld      b, 45
call        vdp_reg
ld      a, 0x90         ; LMMM, LOP = IMP
ld      b, 46
call        vdp_reg
call        vdp_wait
```

各コマンドの後では、S#2 の CE ビットをポーリングして終了を待っています（vdp_wait、→8.1.3）。パラメータレジスタはコマンドごとに更新されるため、次のコマンドでは必要なパラメータを毎回設定し直します（→8.1.6）。

## 8.19 サンプルプログラム：LRMM による回転コピー

サンプルプログラム `lrmm_demo.asm` は、SCREEN 5（Graphic 4）で 64×64 ドットの図形を描き、LRMM で 30 度回転させて右側へコピーする例です。実行結果を図 8.17 に示します。左が転送元の図形、右上が回転コピー、下が比較用の通常の LMMM コピーです。図形の左上の黄色いブロックと右下の緑のブロックの位置から、時計回りに 30 度回転していることが分かります。

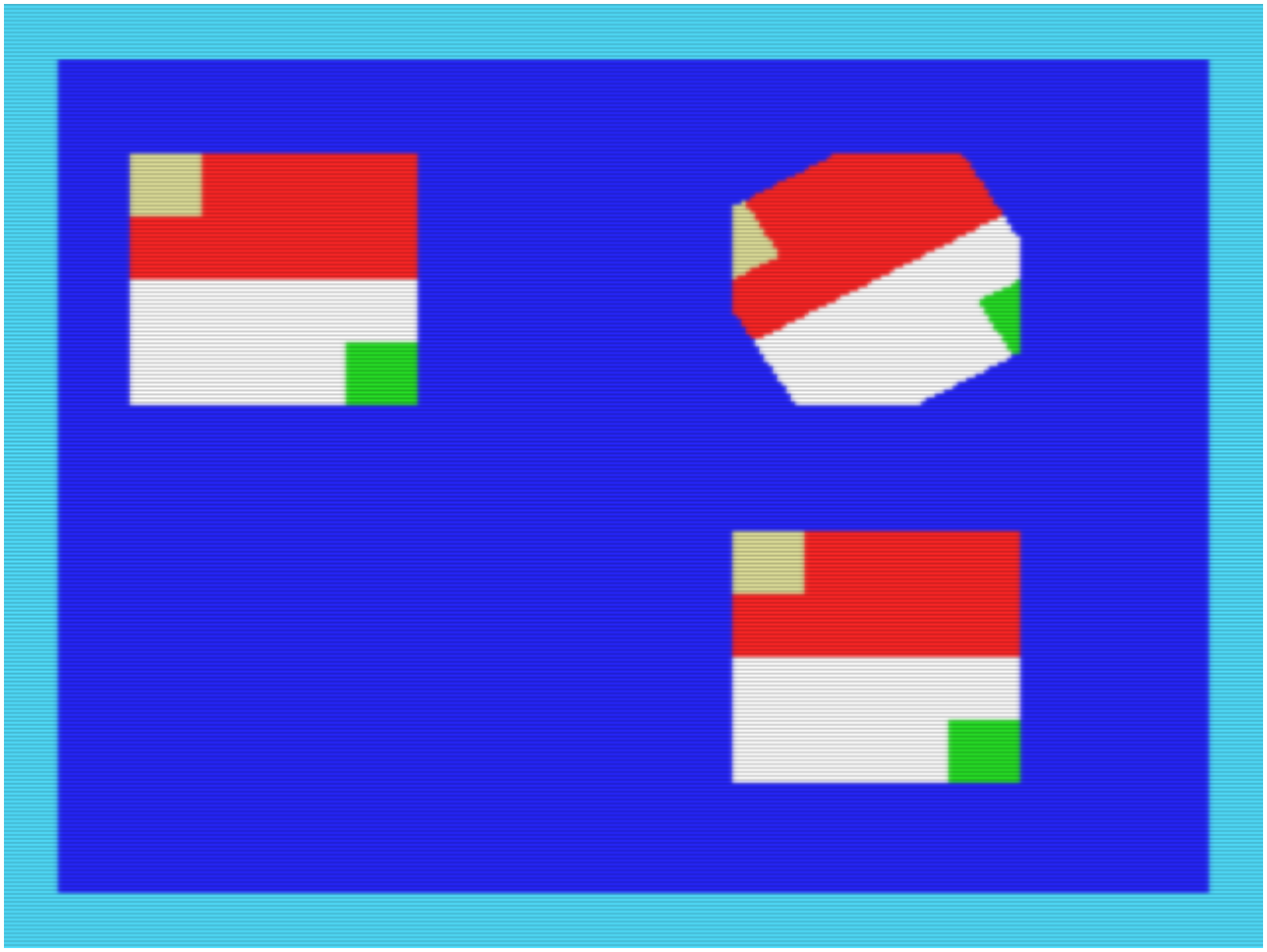


図 8.17 lrmm_demo.asm の実行結果（LRMM による 30 度回転）

LRMM は拡張コマンドなので、実行前に拡張モード（V58 = 0）に切り替えます。R#20 と R#21 はリセット時にロックされている（LockExtReg = 1）ため、先に Port#4 へ bit7 = 0 を書き込んでロックを解除します（→2.7）。

転送元図形の描画は HMMV の矩形塗りつぶしの組み合わせです（ソースファイル参照）。LRMM のパラメータ設定は次のとおりです。**(SX, SY) は回転中心ではなく参照開始点**で、転送先の左上ドットが転送元のどの位置を参照するかを示します。この例では、回転した図形が転送元と同程度の位置に収まるよう、読み取り平行四辺形の中心が転送元矩形の中心 (47.5, 55.5) に一致するよう逆算して (36, 13) としました（→8.14）。

```
        ; ---- LRMM: copy the pattern to (150,24), rotated by 30 degrees
        ld      hl, 36          ; SX = 36 (reference start point)
```

```
        ld      b, 32
        call        vdp_reg16
        ld      hl, 13          ; SY = 13
        ld      b, 34
        call        vdp_reg16
        ld      hl, 150         ; DX = 150
        ld      b, 36
        call        vdp_reg16
        ld      hl, 24          ; DY = 24
        ld      b, 38
        call        vdp_reg16
        ld      hl, 64          ; NX = 64
        ld      b, 40
        call        vdp_reg16
        ld      hl, 64          ; NY = 64
        ld      b, 42
        call        vdp_reg16
        ld      a, 4            ; CLR = 4: color used outside the window
        ld      b, 44
        call        vdp_reg
        ld      a, 0            ; ARG = 0
        ld      b, 45
        call        vdp_reg
```

回転ベクトルとウィンドウの設定が LRMM 独自の部分です。θ = 30 度のとき VX = INT(256 × cosθ) = 221、VY = INT(256 × sinθ) = 128 です（8.8 固定小数点、256 が 1.0）。ウィンドウは転送元矩形そのもの (16, 24)〜(79, 87)（両端含む）に設定し、回転によって読み取り位置が矩形の外に出る転送先の四隅は、CLR で指定した背景色 4 が読まれたものとして描かれます。そのため図 8.17 の右上の図形は四隅が背景色で切り取られた形になっています。

```
        ld      hl, 221         ; VX = INT(256 * cos 30 deg) = 221
        ld      b, 47
        call        vdp_reg16
        ld      hl, 128         ; VY = INT(256 * sin 30 deg) = 128
        ld      b, 49
        call        vdp_reg16
        ld      hl, 16          ; WSX = 16
        ld      b, 51
```

```
        call        vdp_reg16
        ld      hl, 24          ; WSY = 24
        ld      b, 53
        call        vdp_reg16
        ld      hl, 79          ; WEX = 79
        ld      b, 55
        call        vdp_reg16
        ld      hl, 87          ; WEY = 87
        ld      b, 57
        call        vdp_reg16
        ld      a, 0x30         ; LRMM, LOP = IMP
        ld      b, 46
        call        vdp_reg
        call        vdp_wait
```

## 8.20 サンプルプログラム：LFMM による文字列描画

サンプルプログラム `lfmm_demo.asm` は、SCREEN 5（Graphic 4）で、VRAM に置いた 8×16 ドットのフォントデータから文字列 "V9968" を LFMM で描く例です。実行結果を図 8.18 に示します。下段は同じ文字列を DIY = 1 で描いたもので、上下が反転しています。

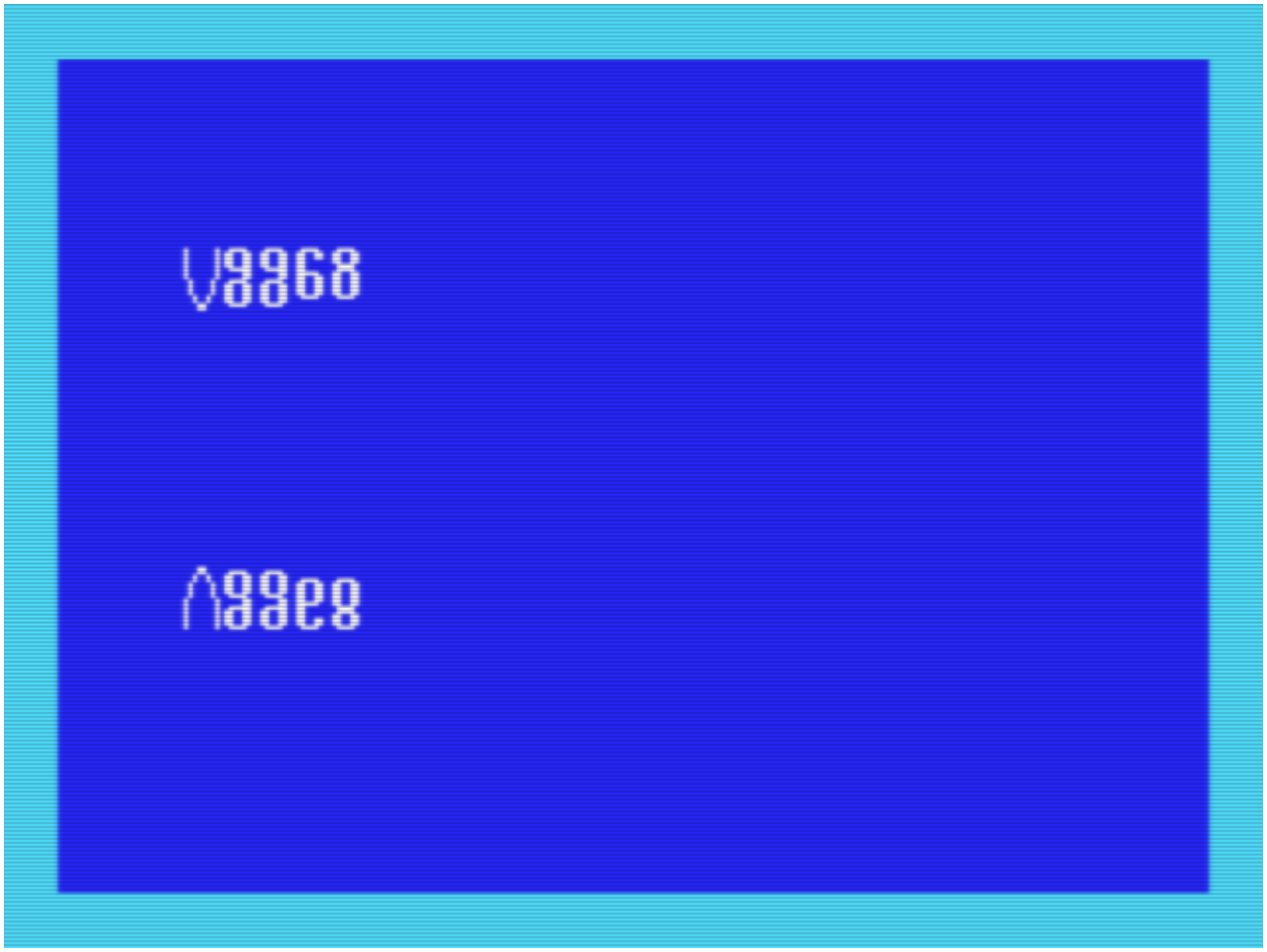


図 8.18 lfmm_demo.asm の実行結果（LFMM によるフォント描画と上下反転）

フォントデータは VRAM の 8000h（表示ページの外）に置きます。1 文字分は 1 行 1 バイト（MSB が左端）× 16 行で、文字列の順に連続して並べます。先頭の "V" は次のようなデータです（残りの文字はソースファイル参照）。

```
; "V"
db      0x81, 0x81, 0x81, 0x81, 0x81, 0x81, 0x81, 0x81
db      0x42, 0x42, 0x42, 0x42, 0x24, 0x24, 0x18, 0x18
```

LFMM のパラメータ設定は次のとおりです。**R#32〜R#34 は座標ではなくフォントデータのアドレス**（3 バイト、下位から順）で、R#35 は未使用のため 0 を書きます。**NX はドット数ではなく文字数**です。また、前景色は R#44、背景色は R#12 にコマンド実行前に設定しておきます（→8.16）。

```
; ---- LFMM #1: draw "V9968" at (28, 48), upright
ld      hl, 0x8000      ; font address -> R#32 (low), R#33
ld      b, 32
call        vdp_reg16
ld      hl, 0           ; R#34 = address bits 17..16 = 0
ld      b, 34           ; (R#35 = 0 as well, it is unused)
```

```
        call        vdp_reg16
        ld      hl, 28          ; DX = 28
        ld      b, 36
        call        vdp_reg16
        ld      hl, 48          ; DY = 48
        ld      b, 38
        call        vdp_reg16
        ld      hl, 5           ; NX = 5 (characters!)
        ld      b, 40
        call        vdp_reg16
        ld      hl, 16          ; NY = 16 (font height)
        ld      b, 42
        call        vdp_reg16
        ld      a, 15           ; foreground color -> R#44
        ld      b, 44
        call        vdp_reg
        ld      a, 4            ; background color -> R#12
        ld      b, 12
        call        vdp_reg
        ld      a, 0            ; ARG = 0
        ld      b, 45
        call        vdp_reg
        ld      a, 0x10         ; LFMM, LOP = IMP
        ld      b, 46
        call        vdp_reg
        call        vdp_wait
```

2 回目は ARG の DIY を 1 にして (28, 144) へ描きます。DIY = 1 では描画が DY から上方向へ進むため、DY は反転した文字列の下端になります（パラメータはコマンドごとに設定し直す点は他のコマンドと同じです）。

# 第 9 章 Geo3D

Geo3D は、V9968 カートリッジの FPGA に実装された 3D グラフィックス支援プロセッサです。3D 座標の透視投影計算と、ワイヤーフレーム／面塗り図形の描画そのものをハードウェアで実行し、Z80 側のプログラムは「モデルデータとカメラを与える」だけで 3D 描画を行えます。

本章は、Geo3D の仕様書（alexmoncks 氏によるフォークのドキュメント）と RTL をもとに、**Geo3D v2.0** の仕様を説明します。V9968 に付属する Geo3D の正式版は、この v2.0 です。前版である v1.0.0 との違いは 9.7 節にまとめました。

## 9.1 概要

Geo3D には 2 つの動作モードがあります。

- **即時モード（immediate mode）**：頂点 1 個ずつを送ると、回転・平行移動・透視投影を行った 2D 座標が返ってきます。描画そのものは行いません。ワイヤーフレームの頂点計算だけを任せたい場合に使います。
- **フェーズ 2 エンジン（phase-2 engine）**：頂点・辺・面のデータをまとめて渡すと、投影から描画までを一括で実行します。V9968 の VDP コマンドエンジン（第 8 章）を内部から駆動して、LINE によるワイヤーフレーム描画、面塗り、テクスチャ貼り付けを行います。

どちらのモードも、座標系や描画先は V9968 の VRAM 座標系（第 8 章の座標系）そのものです。Geo3D は独立した表示装置ではなく、V9968 の描画を肩代わりする存在だと考えてください。

## 9.2 ポートとレジスタアクセス

Geo3D のレジスタは、インデックス／データ方式でアクセスします。カートリッジの DIP スイッチで、ポートのベースを 88h 系と 98h 系から選びます（他のハードウェアとの I/O 競合を避けるためです）。

| ベース | インデックス／ステータスポート | データポート |
|---|---|---|
| 88h 系 | 8Dh | 8Fh |
| 98h 系 | 9Dh | 9Fh |

アクセス手順は次のとおりです。

- 書き込み：インデックスポートにレジスタ番号を書き、データポートに値を書きます。16 ビット値のレジスタは下位バイト、上位バイトの順に、連続したインデックスへ書きます。
- 読み出し：インデックスポートにレジスタ番号を書くと、データポートから値を読めます。

本章ではレジスタを「インデックス 0x40」のように表記します。

## 9.3 即時モード

### 9.3.1 レジスタ

| インデックス | 名称 | 内容 |
|---|---|---|
| 0x00～0x11 | M00～M22 | 回転行列 3×3（各 16 ビット、Q2.14 固定小数点、リトルエンディアン） |
| 0x12～0x17 | TX、TY、TZ | 平行移動（各 16 ビット符号付き） |
| 0x18～0x19 | F | 焦点距離（16 ビット符号なし） |
| 0x1A～0x1B | CX | 画面中心 X 座標 |
| 0x1C～0x1D | CY | 画面中心 Y 座標 |
| 0x1E～0x1F | ZNEAR | ニアクリップ面の Z 値 |
| 0x20～0x21 | W | 画面幅（オーバーフロー判定用） |
| 0x22～0x23 | H | 画面高さ（オーバーフロー判定用） |
| 0x24～0x29 | VX、VY、VZ | 頂点入力ストリーム（各 16 ビット符号付き）。0x29 への書き込みで計算開始 |
| 0x30～0x31 | SX | 投影結果の X 座標（読み出し） |
| 0x32～0x33 | SY | 投影結果の Y 座標（読み出し） |
| 0x34～0x35 | Z | カメラ座標系の Z 値（読み出し） |
| 0x36 | FLAGS | フラグ（読み出し） |

### 9.3.2 変換と投影

頂点 (VX, VY, VZ) は、まず回転行列 M と平行移動 T でカメラ座標系へ変換されます。

```
X = M00 · VX + M01 · VY + M02 · VZ + TX
Y = M10 · VX + M11 · VY + M12 · VZ + TY
Z = M20 · VX + M21 · VY + M22 · VZ + TZ
```

次に透視投影で画面座標へ変換されます。

```
SX = CX + X · F / Z
SY = CY − Y · F / Z
```

インデックス 0x24 から VX、VY、VZ の順に 6 バイトを書き込むと、最後の 1 バイト（0x29）の書き込みをきっかけに計算が始まります。一度に処理できる頂点は 1 個だけです。結果は 0x30 から読み出します。**0x30 を最初に読むことで結果がラッチされる**ので、0x30〜0x36 の 7 バイトを続けて読んでください（Z80 なら INIR 命令が使えます。入力側は OTIR で 6 バイト送ります）。

Z が ZNEAR 未満（または 0）の頂点では、FLAGS の NEAR ビットが立ちます。この場合も SX/SY は計算されますが値は信用できないので、その頂点を使う描画はスキップしてください。

FLAGS のビット構成は次のとおりです。

| ビット | 名称 | 意味 |
|---|---|---|
| 0 | NEAR | Z < ZNEAR（カメラに近すぎる） |
| 1 | OVFX | SX が画面幅の範囲外 |
| 2 | OVFY | SY が画面高さの範囲外 |
| 3 | BUSY | 計算中 |
| 4 | SX<0 | SX が負 |
| 5 | SX>=W | SX が画面幅以上 |
| 6 | SY<0 | SY が負 |

| ビット | 名称 | 意味 |
|---|---|---|
| 7 | SY>=H | SY が高さ以上 |

計算にかかる時間は、頂点あたり最大 49 クロック（21.477MHz 動作時で約 2.3µs）です。理論上は毎秒約 42 万頂点まで処理できますが、実際のプログラムでは Z80 側の OTIR/INIR による転送時間が支配的になります。

## 9.4 フェーズ 2 エンジン

フェーズ 2 エンジンは、シーン全体のデータをレジスタ経由で受け取り、投影と描画を一括実行します。

### 9.4.1 レジスタ

| インデックス | 名称 | 内容 |
|---|---|---|
| 0x40 | VADDR | 頂点データの格納先（内部 RAM のアドレス） |
| 0x41 | EADDR | 辺データの格納先 |
| 0x42 | NVERT | 頂点数 |
| 0x43 | NEDGE | 辺の数 |
| 0x44 | COLOR | ワイヤーフレームの色 |
| 0x45 | LOP | 描画時のロジカルオペレーション（第 8 章の指定と同じ） |
| 0x46〜0x47 | YPAGE | 描画先ページの Y オフセット（16 ビット） |
| 0x48 | CTRL／STATUS | 書き込み：bit0=RUN 開始、bit1=面塗り有効、bit2=テクスチャ有効。読み出し：bit0=busy |
| 0x49 | ID | バージョン識別。v1.0.0 では 0xFF、v2.0 では 0x20 を返す |

| インデックス | 名称 | 内容 |
|---|---|---|
| 0x4A〜0x4B | SKIPPED | スキップされた辺の数（16ビット、読み出し） |
| 0x4C〜0x4D | DRAWN | 描画された辺の数（16ビット、読み出し） |
| 0x4E〜0x4F | CULLED | カリングされた面の数（16ビット、読み出し） |
| 0x50 | 頂点ストリーム | 書き込み専用（9.4.2） |
| 0x51 | 辺ストリーム | 書き込み専用（9.4.2） |
| 0x52 | 面ストリーム | 書き込み専用（9.5） |
| 0x53 | テクスチャ座標ストリーム | 書き込み専用（9.6） |
| 0x58 | FADDR | 面データの格納先 |
| 0x59 | NFACE | 面の数 |
| 0x5A〜0x5F | LX、LY、LZ | 光源方向ベクトル（各 16ビット符号付き） |

v2.0 で追加されたレジスタ（VFIRST など）は 9.7 節にまとめました。

### 9.4.2 データの投入

シーンは 3 種類のストリームで投入します。いずれも、まず VADDR／EADDR／FADDR で格納先を決めてから、対応するストリームポートへ連続して書き込みます。書き込むたびに内部アドレスが自動で進みます。

- **頂点ストリーム（0x50）**：1 頂点 5 バイト。X、Y は 16ビット（リトルエンディアン）、Z は上位バイトのみ（Z の下位 8ビットは 0 として扱われます）
- **辺ストリーム（0x51）**：1 辺 2 バイト。始点と終点の頂点インデックス

頂点数・辺の数は NVERT、NEDGE に設定します（v1.0.0 では各 255 個まで。v2.0 の拡張は 9.7 節）。

頂点と辺だけのシーンはワイヤーフレームとして描かれます。描画色は COLOR レジスタで指定し、YPAGE で描画先ページを選びます。

### 9.4.3 実行と完了待ち

CTRL（0x48）に RUN ビット（bit0）を書くと実行が始まります。実行中は busy（読み出し時の bit0）が 1 になります。

**完了待ちには注意が必要です。** Geo3D は VDP コマンドエンジンを使って描画しますが、コマンドの合間には S#2 の CE ビットが 0 になる瞬間があります。「CE=0」だけでは Geo3D の処理完了を意味しません。必ず **Geo3D の busy が 0 で、かつ CE が 0** であることを確認してください。

また、**busy が 1 の間、Z80 側から VDP コマンドレジスタ（R#32〜R#58）や Geo3D の設定レジスタへ書き込んではいけません。**Geo3D 自身がそれらを使っているため、競合します。

処理結果の内訳は、SKIPPED（画面外などで描かれなかった辺）、DRAWN（描画された辺）、CULLED（裏向きでカリングされた面）の各カウンタで確認できます。

### 9.4.4 VDP コマンドエンジンとの関係

Geo3D は、VDP コマンドのパラメータレジスタ（R#36〜R#46 相当）に直接コマンドを発行して描画します。ワイヤーフレームの各辺は LINE コマンド、テクスチャの貼り付けは LRMM コマンド（→8.14）で描かれます。エンジンは、VDP コマンドが空いている（CE=0）タイミングを見て次のコマンドを投入します。この仕組みのため、Geo3D の実行中は Z80 が VDP コマンドを直接使うこともできません（→9.4.3）。

## 9.5 面塗りと陰影

CTRL の bit1 を立てると、面データを使った面塗り描画が有効になります。

面ストリーム（0x52）には、1 面あたり 11 バイトを書きます。

| バイト | 内容 |
|---|---|
| 0〜2 | 頂点インデックス A、B、C |
| 3 | BASE 色（bit7 はテクスチャ面のフラグ、9.6） |

| バイト | 内容 |
|---|---|
| | |
| 4〜9 | 面法線ベクトル（NX、NY、NZ、各 16 ビット符号付き） |
| 10 | strict フラグ |

陰影は、面法線 N と光源方向 L（LX、LY、LZ レジスタ）の内積から決まります。明るさの段階 level は 0〜6 の 7 段階で、描画色は BASE + level となります。パレット上に BASE から始まる 7 色のグラデーション（ランプ）を用意しておくのが使い方です。内積が負の面（光が当たらない面）は最も暗い段階になります。

描画順はペインタ方式です。面は Z の平均値が大きい（奥にある）ものから順に描かれるので、重なった面は手前のものが上に描かれます。ワイヤーフレームの辺にはこの並べ替えはなく、シーンに登録した順に描かれます。

## 9.6 テクスチャ

CTRL の bit2 を立てると、面へのテクスチャ貼り付けが有効になります。面ストリームの BASE 色の bit7 が立っている面が、テクスチャ面として扱われます。

テクスチャ座標は、テクスチャ座標ストリーム（0x53）へ、面の順序に合わせて 1 コーナーあたり 2 バイト（U、V）で投入します。テクスチャ画像の配置は次のレジスタで指定します。

| インデックス | 名称 | 内容 |
|---|---|---|
| 0x60〜0x61 | TEXX | テクスチャ原点の X 座標 |
| 0x62〜0x63 | TEXY | テクスチャ原点の Y 座標 |
| 0x64 | TSTRIDE | テクスチャの 1 行あたりのバイト数 |
| 0x65 | TADDR | テクスチャのベース（ページオフセット。テクスチャの Y 座標に加算される） |

テクスチャはアフィン変換で貼られます。エンジンは面を行単位のストリップに分解し、各ストリップを LRMM コマンドで VRAM から VRAM へ転送して描きます。

## 9.7 v2.0 の拡張

v2.0 では、扱えるシーンの規模と、シーンの再利用方法が拡張されています。プログラムからは、ID レジスタ（0x49）を読み、0x20 が返れば v2.0 と判定できます（v1.0.0 では 0xFF が返ります）。

### 9.7.1 1024 エントリ対応と拡張モード

XCTRL レジスタ（0x6F）の bit0 を立てると拡張モードになり、頂点・辺・面を最大 **1023 個**まで扱えます（v1.0.0 互換の形式では 255 個まで）。数を数えるレジスタには上位ビット用のレジスタが追加されました。

| インデックス | 名称 | 内容 |
|---|---|---|
| 0x54〜0x55 | VFIRST | 頂点リングの開始位置（16 ビット） |
| 0x56〜0x57 | FFIRST | 面リングの開始位置（16 ビット） |
| 0x66〜0x67 | EFIRST | 辺リングの開始位置（16 ビット） |
| 0x68 | VADDR_H | VADDR の上位ビット |
| 0x69 | EADDR_H | EADDR の上位ビット |
| 0x6A | NVERT_H | NVERT の上位ビット |
| 0x6B | NEDGE_H | NEDGE の上位ビット |
| 0x6C | FADDR_H | FADDR の上位ビット |
| 0x6D | NFACE_H | NFACE の上位ビット |
| 0x6E | TADDR_H | TADDR の上位ビット |
| 0x6F | XCTRL | bit0：拡張モード。bit7〜4：アドレス幅（AW）。10 で 1024 エントリ |

拡張モードではストリームの形式も変わります。

- 頂点ストリーム：1 頂点 6 バイト（Z も 16 ビットフルで指定可能）
- 辺ストリーム：1 辺 3 バイト（A 下位、A/B の上位ビットを詰めたバイト、B 下位）

- 面ストリーム：1 面 12 バイト（従来の 11 バイトに、インデックスと BASE 色の上位ビットを詰めたバイトが加わる）

### 9.7.2 常駐シーンとリングバッファ

VFIRST、FFIRST、EFIRST は、内部 RAM 上でのデータの開始位置を指定します。各データ列はリングとして扱われ、RAM の終端に達すると先頭へ回り込みます。これにより、シーンを毎回送り直さなくても、データを内部 RAM に常駐させたまま描画できます。

さらに、投影結果は RUN をまたいで保持されます。頂点を再計算せずに同じシーンを描き直すだけなら、NVERT=0 で RUN を実行すれば、前回の投影結果を使って再描画（リペイント）できます。カメラだけを動かす場合は頂点の再投入が必要です。

## 9.8 BASIC API（G3BASIC）

Geo3D には、BASIC から使うための拡張ライブラリ「G3BASIC」が用意されています。MSX-BASIC の CALL 文で呼び出す拡張命令群で、レジスタを直接叩かなくても、モデル定義・配置・カメラ・描画のループを BASIC のプログラムとして書けます。主な命令は次のとおりです。

| 分類 | 命令 | 役割 |
|---|---|---|
| セッション | G3INIT、G3END | Geo3D の初期化と終了 |
| データ | G3DATA | モデルデータの読み込み（DATA 文から） |
| データ | G3PAL、G3RAMP | パレット設定、陰影用カラーランプ生成 |
| 配置 | G3OBJ | オブジェクトをスロットへ割り当て |
| 配置 | G3POS | 位置の設定 |
| 配置 | G3ROT、G3SPIN | 回転角の設定、継続回転 |
| 配置 | G3SIZE | 拡大・縮小 |
| 配置 | G3COLOR、G3STYLE | 色と描画スタイル（ワイヤーフレーム／面塗りなど） |

| 分類 | 命令 | 役割 |
|---|---|---|
| カメラ | G3CAM | カメラ位置と向きの設定 |
| 描画 | G3FRAME | 1 フレーム分の描画実行 |

このほか、重心計算（G3CENT）や慣性（G3INERT）、注視（G3LOOK／G3AIM）、光源（G3LIGHT）、フォグ（G3FOG）、テクスチャ指定（G3TEX）、表示制御（G3SHOW／G3HIDE ほか）などの拡張命令があります。

モデルは、多面体や英字などの組み込みモデル（番号 0〜15）が用意されているほか、DATA 文で頂点・辺・面のデータを与えることで自作モデル（番号 16〜31）も定義できます。

性能の目安として、3.58MHz の Z80 上で、255 面のオブジェクトを約 20fps で描画できるとされています（G3BASICドキュメントによる）。

## 9.9 本章の情報源について

本章は、alexmoncks 氏による Geo3Dドキュメント（README、BASIC API 仕様、v2.0 仕様メモ）と、V9968 カートリッジ側の RTL を照合して記述しました。ドキュメントと RTL で食い違う点があれば RTL を正としています。なお、V9968 に付属する Geo3D の正式版は v2.0 であり、本章は v2.0 を正式仕様として記述しています。

## 9.10 サンプルプログラム：公式デモ ROM

Geo3D の公式デモ ROM が、alexmoncks 氏の V9968_Cartridge フォーク（ブランチ `geo3d-v2.0`）の `geo3d/rom/` に収められています。6 種類のデモをループ再生する ROM で、ビルドスクリプト（`build_rom.py --base 0x98`）で V9968 をポート 98h に置いた版（GEO3D_98.ROM）を生成し、Geo3D エミュレーション付き openMSX（マシン C-BIOS_V9968_JP、拡張 geo3d）で実行した結果を図 9.1 と図 9.2 に示します。起動直後の言語メニューで「1」（英語）を選ぶとデモが始まり、スペースキーで次のデモへ進みます。

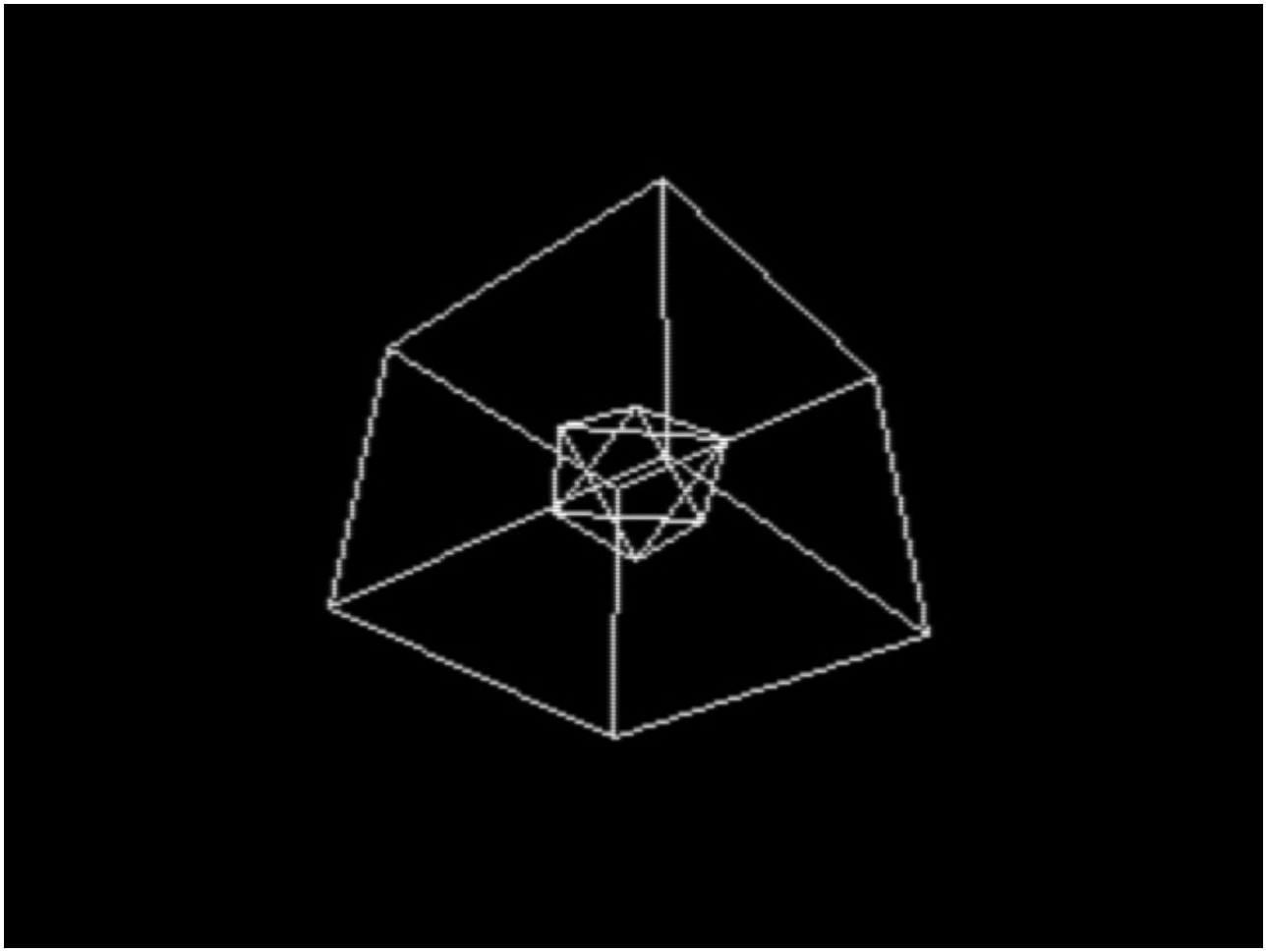


図 9.1 公式デモ ROM の実行結果（ワイヤーフレーム：立方体と八面体）

図 9.1 はワイヤーフレームデモで、立方体と八面体（合わせて頂点 14、辺 24）が回転します。ホスト（Z80）側の仕事は、モデルの投入が最初の一度だけで、あとは 1 フレームあたり約 30 バイトを送るだけです。裏ページの消去（HMMV）と表示ページの切り替え（R#2）は通常の VDP 操作で、Geo3D へは描画先ページ（YPAGE）、あらかじめ計算された行列 24 バイト、RUN を送ります。このデモの原典である MSX-DOS 用プログラム（geo3d/z80/geo3d_demo.asm、デモ ROM にはこのプログラム群を実行して得たポート出力ストリームが収められています）のフレームループは次のとおりです（ポート番号はカートリッジの DIP スイッチ 88h 設定の場合で、98h 版では各ポートに 10h を加えます）。

```
; ---- matrix for this frame + RUN ----
xor a
out (GEO_IDX), a            ; index 0x00 = M00
ld bc, 24 * 256 + GEO_DAT   ; B = 24 bytes, C = port
otir
ld a, 0x48
out (GEO_IDX), a
ld a, 1
out (GEO_DAT), a            ; RUN
```

```
geo_wait:
        in a, (GEO_IDX)
        rrca
        jr c, geo_wait              ; bit0 = RUN busy
```

行列は 1 回転分のテーブルとして用意されており、Z80 は座標計算を一切しません。頂点の変換、辺のクリッピング、LINE コマンド列の生成はすべて Geo3D が行います。Geo3D がなければ、Z80 は頂点計算に加えて 1 フレームあたり約 312 個の VDP コマンドレジスタ値を書き込む必要があります（デモのドキュメントによる）。

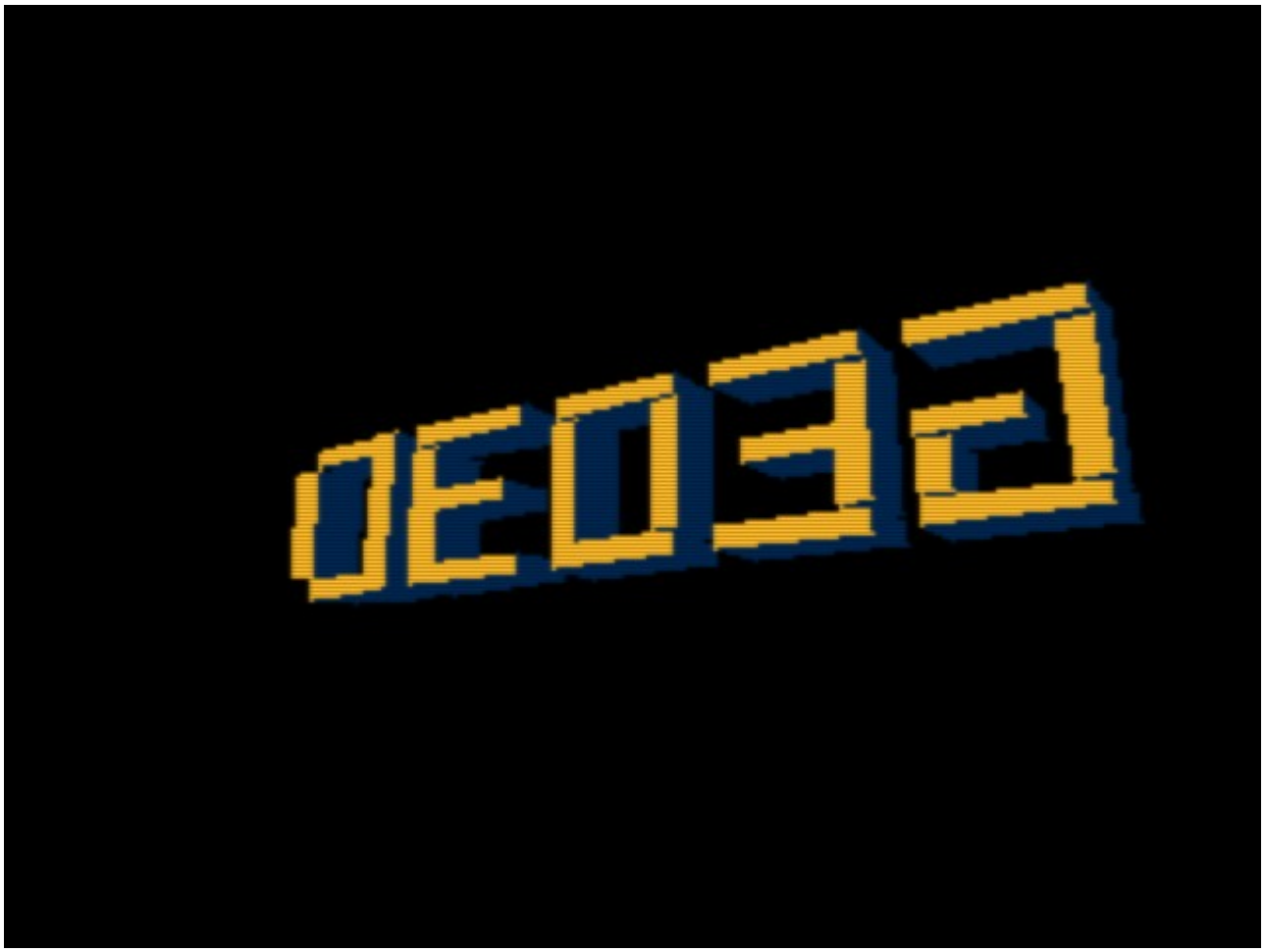


図 9.2 公式デモ ROM の実行結果（面塗り：「GEO3D」の立体文字）

図 9.2 は面塗りデモです。背面の除去、光源方向による陰影、遠い面からの描画順ソート、水平 LINE による面の塗りつぶしを Geo3D が担当し、ホスト側の送信量はワイヤーフレームと同じく 1 フレームあたり約 30 バイトで変わりません。このあとテクスチャ付き面塗り、パン／ズーム、フライインのデモが続きます。

# 第 10 章 割り込み

V9968 には、プログラムの実行と並行して画面表示や VDP コマンドの処理が進むことを CPU に知らせるための割り込み機能があります。本章では、割り込みの要因、各要因の有効化とクリアの方法、および割り込み制御ポート（Port#4）の使い方を説明します。

割り込みに関するレジスタとフラグのビット構成そのものは第 3 章・第 4 章で説明しました。本章では「割り込みをどう使うか」に絞って記述します。

## 10.1 割り込み要因の概要

V9968 の割り込み要因は次の 3 つです。

| 要因 | 発生条件 | フラグ | 許可ビット | Port#4 の表示 |
|---|---|---|---|---|
| フレーム割り込み | 垂直同期のタイミング（1 フレームごと） | S#0 の F | R#1 の IE0 | FI |
| ライン割り込み | 表示ラインが R#19 の指定ラインに達したとき | S#1 の FH | R#0 の IE1 | LI |
| コマンド完了割り込み | VDP コマンドの実行が完了したとき | （Port#4 の CI） | R#20 の CEIE | CI |

割り込み出力（INT）は、これら 3 要因について「フラグが立っている かつ 対応する許可ビットが 1」という条件の OR を取ったもの（負論理）です。どれか 1 つの条件が成立している間、割り込み出力はアクティブになります。したがって、割り込みハンドラ内では、どの要因が残っているかを Port#4 の読み出しで確認し、処理した要因をクリアしてください。要因が残ったままだと、割り込みから復帰した直後に再度割り込みがかかります。

なお、スプライトの衝突（S#0 の C）や表示上限の超過（S#0 の 5S）は、**割り込み要因ではありません**。これらはステータスレジスタをポーリングして検出するものです（第 4 章・第 6 章参照）。

## 10.2 フレーム割り込み

フレーム割り込みは、垂直同期のタイミングで 1 フレームに 1 回発生します。画面の書き換えを帰線期間に合わせたい場合や、一定周期の処理の基準として使う、最も基本的な割り込みです。

- 有効化: R#1 の IE0（bit5）を 1 にします。
- フラグ: S#0 の F（bit7）。割り込みが許可されていなくても、垂直同期のタイミングで F は立ちます。
- クリア: 次のいずれかで行います。
- S#0 を読み出す（このとき C と 5S も同時にクリアされます。第 4 章参照）。
- Port#4 に FI（bit0）= 1 を書く。

F は読み出すまで立ったままです。IE0 = 0 の間に発生したフレーム割り込みのフラグも残るため、あとから IE0 を 1 にすると、その時点で割り込みがかかることがあります。許可する前にフラグをクリアしておくことを勧めます。

## 10.3 ライン割り込み

ライン割り込みは、表示ラインが指定したライン番号に達したタイミングで発生します。画面の途中でのパレット切り替えやスクロール設定の変更など、いわゆるラスタ処理に使います。

- ライン番号の設定: R#19（Interrupt Line）に、割り込みを発生させたい表示ライン番号を設定します。
- 有効化: R#0 の IE1（bit4）を 1 にします。
- フラグ: S#1 の FH（bit0）。割り込みが許可されていなくても、指定ラインに達すると FH は立ちます。
- クリア: 次のいずれかで行います。
- S#1 を読み出す。
- Port#4 に LI（bit1）= 1 を書く。
- R#0 または R#19 に書き込む（副作用として FH がクリアされます）。

R#19 を書き換えると FH がクリアされるため、1 フレームの中で複数のライン割り込みを使いたい場合は、ハンドラ内で「次のライン番号を R#19 に書き込む」ことで、フラグのクリアと次回設定を同時に行えます。

ライン番号と垂直スクロールの関係にも注意してください。通常、R#19 のライン番号の比較は、垂直スクロール（R#23）のオフセットの影響を受けます。R#20 の ILNS（bit2）を 1 にすると、ライン番号は R#23 に影響されず、画面の一番上を 0 とする絶対位置として解釈されます（R#19 = 0 が画面の先頭ラインです）。垂直スクロールを動かしながら、画面上の決まった位置で割り込みたい場合は ILNS を使ってください。

## 10.4 コマンド完了割り込み

コマンド完了割り込みは、VDP コマンドの実行が完了したタイミングで発生します。S#2 の CE ビットをポーリングする代わりに、コマンドの終了を割り込みで待ちたい場合に使います（コマンドの実行モデルは第 8 章参照）。

- 有効化: R#20 の CEIE（bit6）を 1 にします。
- 要因の確認: Port#4 の CI（bit2）が、コマンド完了の要因が発生中であることを示します。
- クリア: Port#4 に CI（bit2）= 1 を書きます。

注意点が 2 つあります。

1. **CEIE は拡張レジスタ R#20 の中にあります。** R#20 は LockExtReg（Port#4 の bit7）が 1 の間は書き込みが無視されるため、CEIE を設定する前に Port#4 でロックを解除する必要があります（手順は 10.5 節および第 2 章参照）。なお、CEIE の使用そのものは拡張モード（V58 = 0）でなくても可能です。ロック解除とモード切り替えは別の操作です。
2. **S#2 の CE はフラグではなく実行中表示です。** CE はコマンド実行中に 1 になるビットで、読み出してもクリアされません。完了割り込みの要因のクリアは、必ず Port#4 への書き込みで行ってください。

## 10.5 Port#4 による要因確認とクリア

Port#4 は、割り込み要因の状態確認とクリア、および拡張レジスタのロック（LockExtReg）を扱うポートです（第 2 章参照）。

### 読み出し

| bit | 名称 | 意味 |
|---|---|---|
| 7 | LockExtReg | 拡張レジスタのロック状態（リセット時 1） |
| 6〜3 | — | 0 |
| 2 | CI | コマンド完了割り込みの要因が発生中なら 1 |
| 1 | LI | ライン割り込みの要因が発生中なら 1 |
| 0 | FI | フレーム割り込みの要因が発生中なら 1 |

読み出しでは、許可ビット（IE0 / IE1 / CEIE）の状態にかかわらず、要因の発生状態そのものが得られます。割り込みハンドラでの要因判別のほか、割り込みを使わないプログラムでも、フレーム同期の検出などに利用できます。

### 書き込み

| bit | 意味 |
|---|---|
| 7 | LockExtReg への書き込み（0 でロック解除、1 でロック） |
| 2 | 1 を書くとコマンド完了割り込みのフラグをクリア |
| 1 | 1 を書くとライン割り込みフラグをクリア |
| 0 | 1 を書くとフレーム割り込みフラグをクリア |

フラグのクリアは「1 を書いたビットだけ」が対象で、0 を書いても何も起こりません。複数の要因を同時にクリアすることもできます。ただし、bit7 は LockExtReg そのものへの書き込みです。要因クリアだけを行いたい場合は、現在のロック状態と同じ値を bit7 に書いてください（ロック状態は読み出しで確認できます）。不用意に bit7 を書き換えると、拡張レジスタの保護状態が変わってしまいます。

### 拡張レジスタのロック解除手順

R#20 / R#21 を設定するには、LockExtReg を解除する必要があります。手順は次の通りです（第 2 章も参照）。

1. Port#4 に bit7 = 0 の値を書く（LockExtReg が 0 になり、ロックが解除される）。
2. R#20 / R#21 へ必要な設定を書き込む（拡張モードへ切り替える場合は R#21 の V58 を 0 にします）。
3. 必要に応じて、Port#4 に bit7 = 1 の値を書き、再びロックする。

ロックは、リセット直後の意図しない書き込みからモード設定を守るための仕組みです。拡張機能を使い終わったあとは、再ロックしておくことを勧めます。

## 10.6 割り込みの使い方の流れ

フレーム割り込みとコマンド完了割り込みを併用する例で、全体の流れを示します。

1. **初期化**: Port#4 を読み、残っている要因フラグがあれば Port#4 への書き込みでクリアします。コマンド完了割り込みを使う場合は、LockExtReg を解除して R#20 の CEIE を 1 にし、再ロックします。
2. **許可**: R#1 の IE0 を 1 にしてフレーム割り込みを許可します（ライン割り込みを使う場合は R#19 を設定してから R#0 の IE1 を 1 にします）。
3. **ハンドラ**: 割り込みがかかったら Port#4 を読み、立っている要因を判別します。要因ごとに処理を行い、処理し

た要因は Port#4 への書き込み（または対応するステータスレジスタの読み出し）でクリアします。

4. **待ち合わせ**: メイン処理側でコマンドの完了を待ちたい場合は、コマンドを起動してから割り込み待ちに入ります。ポーリングで待つ場合は S#2 の CE が 0 になることを確認します（第 8 章参照）。どちらの方法でも、コマンド完了割り込みの要因（CI）が立ったまま残らないよう、確認後はクリアしてください。

割り込みを使わない設計も可能です。フレーム同期は S#0 の F のポーリング、表示期間の判別は S#2 の VR / HR ビット、コマンドの完了は S#2 の CE のポーリングで、それぞれ代替できます（第 4 章参照）。

## 10.7 サンプルプログラム：ライン割り込みによる画面分割

サンプルプログラム `line_demo.asm` は、ライン割り込みのフラグ（FH）をポーリングして、画面の上下でパレットを切り替える例です。SCREEN 5（Graphic 4）でパレットの 16 色を縦帯として描き、R#19 には画面中央の 106 ライン目を設定します。実行結果を図 10.1 に示します。上半分と下半分で各帯の色が変わっていることが分かります（周辺色もパレット経由のため、上下で色が変わります）。

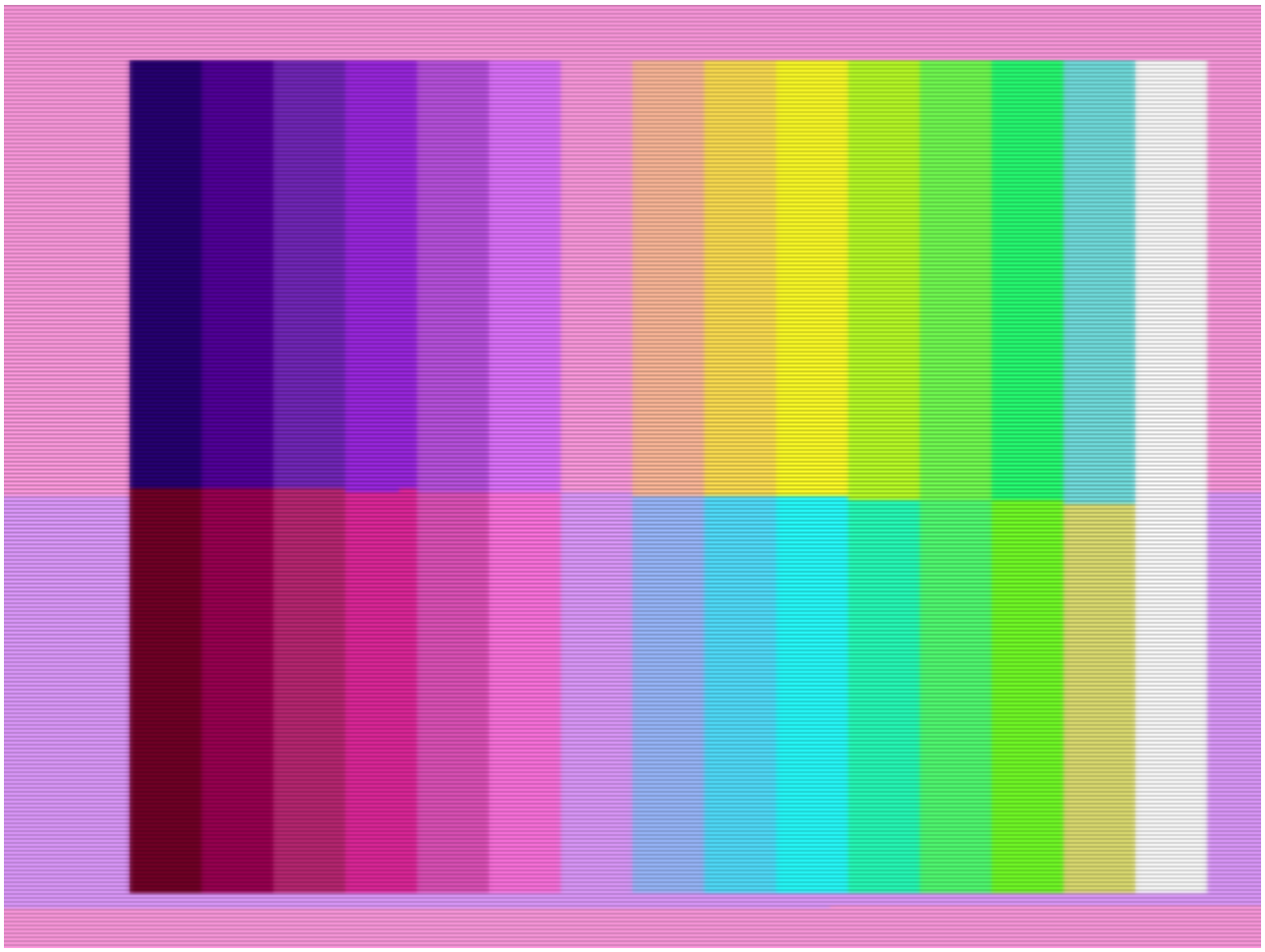


図 10.1 line_demo.asm の実行結果（ライン 106 でのパレット切り替え）

メインループは次のとおりです。フレームの開始を S#0 の F ビットで待ち、パレット A を書き込みます。次にステータス

選択を S#1 に切り替え、FH ビットが立つ（ラスターが R#19 のラインに達する）のを待って、パレット B（パレット A の R 成分と B 成分を入れ替えたもの）を書き込みます。

```
split_loop::
        ; wait for the V-blank flag F (S#0 bit7)
wait_f:
        in      a, [0x99]
        bit     7, a
        jr      z, wait_f
        ; top half: palette A
        ld      hl, palette_a
        call        apply_palette
        ; select S#1 and wait for the line-interrupt flag FH (bit0)
        ld      a, 1
        ld      b, 15
        call        vdp_reg
wait_fh:
        in      a, [0x99]
        bit     0, a
        jr      z, wait_fh
        ; bottom half: palette B
        ld      hl, palette_b
        call        apply_palette
        ; restore the S#0 selection
        xor     a, a
        ld      b, 15
        call        vdp_reg
        jr      split_loop
```

FH フラグは、IE1 でライン割り込みを許可していなくても、ラスターが指定ラインに達した時点で立ちます（→10.3）。そのため、割り込みハンドラを用意しなくても、ポーリングだけで画面分割が実現できます。カートリッジ ROM では 0038h のベクタのあるページ 0 が BIOS の領域で置き換えられないことも、ポーリング方式を選ぶ理由のひとつです。

なお、FH が立ってからパレット 16 エントリ分の書き込みが終わるまでに数ラインかかるため、実際の切り替え位置は R#19 の指定より数ライン下になります。パレットデータと帯の描画はソースファイル参照。

## 10.8 2 台の VDP がある環境での注意

V9968 が本体に内蔵されている場合と、カートリッジなどで 2 台目の VDP（2nd VDP）として接続されている場合とでは、I/O ベースアドレスと垂直同期割り込みの受け方が異なります。

| 構成 | I/O ベースアドレス | 垂直同期割り込みを受けるフック |
|---|---|---|
| 内蔵 VDP | 98h | H.TIMI |
| 2nd VDP | 88h | H.KEYI でも処理が必要 |

内蔵 VDP の場合、ベースアドレスは 98h で、BIOS ワークエリアの 0006h / 0007h にも 98h が格納されています（第 2 章参照）。垂直同期割り込みは、BIOS の割り込み処理を通じて H.TIMI フックで通常どおり受けられます。

2nd VDP の場合、ベースアドレスは 88h です。ワークエリアの 0006h / 0007h は内蔵 VDP の値（98h）のままであるため、2nd VDP へアクセスするプログラムは 88h を直接指定してください。また、2nd VDP の垂直同期割り込みは H.TIMI では受けられないため、H.KEYI フックでも処理する必要があります。要因の確認とクリアは、2nd VDP 自身の Port#4（88h + 4 = 8Ch）やステータスレジスタで行ってください。

> **【注意】** 内蔵 VDP と 2nd VDP の垂直同期は、一致しているとは限りません。H.TIMI が呼ばれるタイミングは内蔵 VDP の垂直同期に合わせたものであり、その時点で 2nd VDP が垂直ブランキング期間に入っている保証はありません。2nd VDP 側の画面更新を帰線期間に合わせたい場合は、H.TIMI を目安にせず、2nd VDP 自身のステータス（S#0 の F、S#2 の VR など）をポーリングして、2nd VDP 自身のタイミングで処理してください。

# 第 11 章 VRAM アクセスとタイミング

本章では、VRAM の構成と、CPU から VRAM にアクセスする際のモデルや制限について、第 2 章（アクセス手順）と第 8 章（VDP コマンド）に分散していた内容を、タイミングの観点からまとめます。

なお、本章で述べるのは、プログラマから見えるアクセスモデルと、実装で確定している制限までです。画面モードごとの CPU アクセスウィンドウ（表示期間中のどのタイミングでアクセスが保証されるか）のような詳細なタイミング仕様は、本章の末尾に確認事項として挙げます。

## 11.1 VRAM の構成

V9968 の VRAM アドレス空間は 18 ビット（A17〜A0）で、最大 256KB です。実際に使える範囲は動作モードで変わります。

| モード | 使える VRAM | アドレスの扱い |
|---|---|---|
| V9958 互換モード（V58 = 1） | 128KB 相当 | A17 が各所でマスクされ 0 として扱われる |
| 拡張モード（V58 = 0） | 256KB | A17〜A0 の全範囲が有効 |

互換モードで A17 がマスクされる箇所は次の通りです（第 3 章参照）。

- テーブルベースアドレスレジスタ R#2 / R#4 / R#6 / R#10 / R#11 に書いた A17 は、0 として保持されます。
- CPU アクセス用アドレスの上位を指定する R#14 の A17 もマスクされます。
- さらに、R#21 に 1 を書いて互換モードへ戻した瞬間、5 つのテーブルベースアドレスの bit17 はすべて 0 にクリアされます。拡張モードで上位ページを使っていたプログラムが互換モードへ戻る場合は、テーブル配置を再設定してください。

スプライトやコマンド関係でも同様に、互換モードでは座標・アドレスの上位ビットの扱いが 128KB 前提になります（R#39 / R#43 の DY / NY の有効ビット数など。第 3 章参照）。

### VRAM の物理配置（DRAM マッピング）

VRAM は物理的には 2 つの DRAM（DRAM0 と DRAM1、各 128KB）で構成されており、論理アドレス（プログラムから見える VRAM アドレス）がどちらの DRAM に置かれるかは、画面モードによって切り替わります。

| マッピング | 適用条件 | 論理アドレス → DRAM |
|---|---|---|
| フラット | SCREEN 0〜6（および SCREEN 7 以上でスプライトモード 3 のとき） | 0000h〜FFFFh → DRAM0、10000h〜1FFFFh → DRAM1 |
| インタリーブ | SCREEN 7 以上（スプライトモード 3 でないとき） | 偶数アドレス（0000h、0002h、0004h、…、1FFFEh）→ DRAM0、奇数アドレス（0001h、0003h、0005h、…、1FFFFh）→ DRAM1 |

SCREEN 7 以上でインタリーブになるのは、従来チップとの互換性を保つためであると同時に、2 つの DRAM にアクセスを分散して帯域を確保するためです。

一方、スプライトモード 3（SP3、R#20 bit3）ではパターンデータを連続したアドレス範囲として読み出す必要があるため、SCREEN 7 以上でもフラットな配置に戻ります。

> **【注意】** SCREEN 7 以上の画面モードで SP3 を切り替えると、VRAM データの物理的な並びが変わります。プログラムが扱う論理アドレス自体は変わりませんが、切り替え前に一方のマッピングで書き込んだデータは、切り替え後は別の並びとして現れます。たとえば SCREEN 8 でビットマップやパターンを配置した後にスプライトモード 3 へ切り替える（あるいはその逆を行う）場合は、必要なデータを切り替え後の配置に合わせて置き直してください。この振る舞いは FPGA 版でも同様です。

## 11.2 アドレスモデルとラップアラウンド

CPU アクセス用のアドレスレジスタは読み出し・書き込みで共通の 18 ビットです。下位 14 ビット（A13〜A0）は Port#1 の 2 バイト組プロトコルで、上位の A17〜A14 は R#14 で設定します（第 2 章参照）。

アドレスをインクリメントした際の桁上がり（ラップアラウンド）の範囲は、動作条件によって変わります。

| 条件 | ラップ範囲 |
|---|---|
| 画面モード {M5, M4} が 00 の系統（Graphic1 / Text1 / Multicolor / Graphic2）、または R#8 の VR = 0 のとき | 14 ビット（A13〜A0 のみ進む） |
| 上記以外で、互換モード（V58 = 1）のとき | 17 ビット |

| 条件 | ラップ範囲 |
|---|---|
| 上記以外で、拡張モード（V58 = 0）のとき | 18 ビット |

14 ビットでラップする系統は、TMS9918 時代からの互換性のために意図的に設けられた仕様です。この系統では、R#14 で A17〜A14 を指定して 4000h 以降の領域にもアクセスできますが、bit13 から bit14 への桁上がりが遮断されているため、連続アクセスで 3FFFh の次は 0000h に、7FFFh の次は 4000h に戻ります。ページをまたぐ転送は、境界ごとにアドレスを設定し直してください。

また、R#14 に設定する A17 は、R#8 の VR ビットが 0 のときは 0 固定として扱われます。

## 11.3 CPU アクセスの手順上の注意

アドレス設定と最初のアクセスの関係は、従来の VDP と同じ規則です（第 2 章参照）。

- 書き込み用にアドレスを設定した場合、アドレス設定直後の最初のアクセスではインクリメントされません（設定したアドレスそのものに書かれます）。
- 読み出し用にアドレスを設定した場合、Port#0 の最初の読み出しで、設定したアドレスのデータが返ります。読み出しアドレスは、VRAM からの応答時にインクリメントされます。
- Port#1 の 2 バイト組は、途中で Port#0 へのアクセスやいずれかのポートの読み出しを行うと破棄されます。アドレス設定とレジスタ書き込みのシーケンスは、割り込みハンドラなどと競合しないよう注意してください（ハンドラ内で VDP にアクセスする場合は、メイン処理側の 2 バイト組が中断される可能性があります）。

## 11.4 VDP コマンド実行中のアクセス

VDP コマンドの実行中も、VRAM はコマンドエンジンが使用しています。コマンドと CPU アクセスの関係で確定している事項は次の通りです。

- コマンドの実行中は、別のコマンドを起動してはいけません（第 8 章参照）。完了は S#2 の CE ビット、またはコマンド完了割り込みで確認します（第 10 章参照）。
- R#45 の MXC ビットが 1 の間は、CPU からの VRAM アクセスはマスクされます。この間、VRAM への書き込みは実行されず、読み出しは FFh を返します。MXC は、コマンド実行中に CPU 側のアクセスが競合しないよう、ソフトウェアが明示的にアクセスを遮断するためのビットです（第 8 章参照）。

【要確認】MXC = 0 のままコマンド実行中に CPU が VRAM にアクセスした場合の調停（どちらが優先されるか、アクセスがどう扱われるか）については、現時点の資料では確定していません。コマンド実行中の CPU アクセスを避けるか、

MXC で遮断するのが安全です。

## 11.5 表示期間中のアクセス

従来の VDP では、表示期間中の CPU アクセスには画面モードごとの制約があり、高速な連続アクセスでは帰線期間を使うなどの配慮が必要でした。V9968 でも、表示状態を知るための手がかりとして、S#2 の VR ビット（垂直帰線期間中）と HR ビット（水平帰線期間中）が用意されており、ソフトウェアはこれらをポーリングして表示の合間を検出できます（第 4 章参照）。

【要確認】画面モードごとに「表示期間中の CPU アクセスがどのタイミングで保証されるか」「連続アクセス時の最小間隔」といったタイミング仕様は、現時点の資料では確定していません。従来機向けのソフトウェアが V9968 上でもそのままのタイミングで動作するかは、この仕様の確定を待つ必要があります。

## 11.6 高速モードについて

R#20 の HS ビット（bit0）を 1 にすると高速モードになります（第 3 章参照）。HS は基板側の入力信号（force_highspeed）と OR されるため、レジスタ値が 0 でも、搭載される基板の条件によって高速モードになることがあります。

【要確認】高速モードがコマンド実行時間や CPU アクセスのタイミングに与える具体的な影響（どの処理がどの程度速くなるか）は、現時点の資料では確定していません。

# 第 12 章 互換性と移行の注意

V9968 は、TMS9918 から V9938、V9958 へと発展してきた VDP の系譜を引き継ぎ、従来のソフトウェアがそのまま動作することを基本方針として設計されています。本章では、従来チップとの動作の違いと、「9958 Compatible モード」（V9958 互換モード）の範囲、そして既存ソフトウェアを V9968 で動かす場合・V9968 向けに拡張する場合の注意点をまとめます。

各項目の詳細は、該当する章を参照してください。本章は差異の一覧に絞ります。

## 12.1 互換モードと拡張モード

V9968 の動作モードは、R#21 の V58 ビット（bit0）で切り替わります（第 3 章参照）。

- **リセット直後は V58 = 1（互換モード）**です。電源投入直後の V9968 は V9958 相当として振る舞い、従来のソフトウェアはそのまま動作します。
- 拡張モード（V58 = 0）への切り替えは、LockExtReg を解除してから R#21 に書き込む、明示的な手順が必要です（第 10 章参照）。従来のソフトウェアが誤って拡張モードに入ることはありません。
- 拡張モードで有効になるのは、主に次の要素です。
- VRAM 256KB のアドレス空間（第 11 章）
- 新 VDP コマンド LRMM / LFMC / LFMM（第 8 章）
- R#45 の FG4 など、拡張モードでのみ格納されるビット（第 3 章）
- SPS（R#25 bit7）の書き込み（12.3 節参照）

モードの判別には S#1 の ID が使えます。ID は互換モードで 00010（V9958 と同じ値）、拡張モードで 00011 を返します（第 4 章参照）。チップの世代を ID で判別するソフトウェアは、互換モードの V9968 を V9958 として扱えます。

互換モードに戻すときの副作用として、R#21 に 1 を書いた瞬間、5 つのテーブルベースアドレス（パターンネーム／パターンジェネレータ／スプライトパターンジェネレータ／カラー／スプライトアトリビュート）の bit17 がすべて 0 にクリアされます（第 3 章参照）。

## 12.2 「9958 Compatible」として正式化された動作

従来の資料では未定義とされていたり、代表値しか示されていなかったりした挙動のうち、V9968 で実際に使えることが分かったものは、本書では「9958 Compatible」と付記して正式な機能として説明しています。代表例は次の通りで

す。

- **画面モード選択のドントケア**: Text1 と Multicolor では M3 がドントケアであり、M3 = 1 の組み合わせ（Text1 = 00101、Multicolor = 00110）でも同じモードとして動作します（第 5 章参照）。
- **予約ロジカルオペレーションの扱い**: VDP コマンドのロジカルオペレーションで、従来は予約とされていたコード 0101～0111 は IMP 相当、1101～1111 は TIMP 相当として動作します（第 8 章参照）。

## 12.3 従来チップとの既知の動作差異

次に挙げる項目は、互換モードであっても従来チップと動作が異なる（または、従来資料の記述と異なる）点です。既存ソフトウェアの移植や、動作の検証の際に注意してください。

### 表示・色に関する差異

- **Graphic7 のスプライト色（V9938 との差異）**: V9938 では Graphic7 のスプライトはパレットを使わない固定 16 色でしたが、V9968 ではスプライトの色も背景と同じ 256 色パレットから決まります。256 色は 16 色ずつの 16 グループ（パレットセット）に分けられ、スプライトはグループ内の 16 色で表示されます。ただし、各グループの先頭の色は透明扱いになるため、実質的に使えるのは 15 色です（第 7 章参照）。
- **パレットのリセット初期値**: リセット時に初期化されるのはエントリ 0～15 のみで、V9958 互換の初期色が入ります。**エントリ 16～255 は不定**です。従来のプログラムは 16 色しか使わないため影響を受けませんが、EPAL や Sprite mode3 など拡張側のパレットを使う場合は、必ずプログラム側で初期化してから使ってください（第 7 章参照）。

### レジスタ読み出し値に関する差異

- **S#2 の bit3 / bit2 は 1 固定**です。旧資料に記載のないビットですが、未使用ビットが 0 で返ることを前提にした判定はしないでください（第 4 章参照）。
- **S#1 の bit7 / bit6 は 0 固定**、S#4 / S#6 / S#9 の未使用ビットは **1 固定**です。ステータスレジスタの値は、必要なビットだけをマスクして判定してください（第 4 章参照）。
- **S#10～S#15 は FFh を返します**。実装されていない番号を指定した場合の読み出し値は FFh で確定しています（第 4 章参照）。
- **Port#2 / Port#3 の読み出しは FFh を返します**。これらのポートは書き込み専用です（第 2 章参照）。

### 未実装レジスタの扱い

- R#22、R#24、R#28～R#31、およびコマンドレジスタの R#59～R#63 は未実装で、書き込みは無視されます

（第 3 章参照）。未定義レジスタへの書き込みを前提にした初期化コードは、V9968 では何も起こりません。

### 拡張ビットの保護

- **SPS（R#25 bit7）は V9968 の新設ビットで、拡張モード（V58 = 0）のときにのみ書き込めます**。互換モードでは書き込みが無視されるため、R#25 の bit7 を含む値を書く従来のソフトウェアが、意図せずスプライトの優先順位循環（第 6 章参照）を有効にしてしまうことはありません。
- S16（R#20 bit7）や CEIE（R#20 bit6）など R#20 / R#21 の拡張設定は、LockExtReg によるロックで保護されています（第 10 章参照）。

### アドレス空間に関する互換仕様

- 画面モード {M5, M4} が 00 の系統（Graphic1 / Text1 / Multicolor / Graphic2）では、CPU アクセスのアドレスは 14 ビットでラップします。これは TMS9918 搭載機との互換性のために意図的に残された仕様であり、不具合ではありません（第 11 章参照）。
- 互換モードでは A17 がマスクされ、VRAM は 128KB 相当として扱われます。従来のアドレス配置のまま使えます（第 11 章参照）。
- SCREEN 7 以上では、VRAM の物理配置が従来互換のインタリーブ（偶数アドレスが DRAM0、奇数アドレスが DRAM1）になります。ただし、スプライトモード 3 を有効にするとフラットな配置に戻り、切り替えによってデータの物理的な並びが変わります（11.1 節参照）。

### VDP コマンドに関する差異

- 従来の 13 コマンドの動作は、従来仕様（『MSX2 テクニカルハンドブック』）と照合して同一であることを確認しています（第 8 章参照）。
- 新コマンド LRMM / LFMC / LFMM のコード（0001〜0011）は、V9958 では未使用のコマンドコードに割り当てられています。互換モードでこれらを起動すると STOP として扱われます。従来のソフトウェアが未使用コードを発行することは通常ありませんが、コード空間を共有している点は意識してください（第 8 章参照）。

## 12.4 移行・拡張時の注意まとめ

既存ソフトウェアを V9968 向けに拡張・移植する場合は、次の点を確認してください。

1. **拡張モードへの移行手順を守ること**: LockExtReg の解除 → R#21 で V58 = 0 → 必要な拡張設定、の順で行い、使い終わったら再ロックします（第 10 章参照）。
2. **パレットの 16〜255 を初期化すること**: リセット直後は不定です（12.3 節）。

3. **ステータスレジスタはマスクして判定すること**: 固定ビット（1 固定・0 固定）の存在を前提に、全ビット一致で比較しないでください（第 4 章参照）。
4. **互換モードへ戻すときはテーブルベースを再設定すること**: bit17 がクリアされる副作用があります（12.1 節）。

5. **チップ判別には S#1 の ID を使うこと**: 互換モードでは V9958 と同じ値（00010）が返るため、V9968 の存在そのものを検出したい場合は、拡張モードへ切り替えたうえで ID を確認するなどの手順が必要です。
6. **2 台目の VDP（2nd VDP）として使う場合は、ベースアドレスと割り込みフックの違いに注意すること**: ベースアドレスは 88h で、垂直同期割り込みは H.KEYI フックでも処理する必要があります。内蔵 VDP の垂直同期と一致しているとは限りません（10.8 節参照）。

なお、本書の各章に残されている【要確認】の項目（R#20 の FIL / ILNS / SVNS、R#45 の FG4 / XHR、ウィンドウレジスタの適用範囲、表示期間中の CPU アクセスタイミングなど）は、仕様が確定次第、将来の版で反映されます。未確定の項目に依存したソフトウェアは、現時点では作成しないでください。

# 付録

本付録には、第 3 章〜第 8 章で説明した内容のクイックリファレンスをまとめます。ビット構成の詳細や機能説明は各章を参照してください。本付録の内容が各章と食い違う場合は、各章の記述を正とします。

## 付録 A コントロールレジスター覧

### A.1 表示・アクセス関係（R#0〜R#27）

すべて書き込み専用です。R#20 / R#21 は、LockExtReg = 1 の間は書き込みが無視されます（第 10 章参照）。

「V9968 拡張」は、V9968 で追加・拡張されたレジスタ（ビット）を示します。

| R# | 名称 | リセット値 | 主な内容 |
|---|---|---|---|
| R#0 | Mode0 | IE1=0, M5M4M3=000 | bit4: IE1（ライン割り込み許可） / bit3〜1: M5, M4, M3 |
| R#1 | Mode1 | すべて 0 | bit6: BL（表示許可） / bit5: IE0（フレーム割り込み許可） / bit4: M1 / bit3: M2 / bit1: SI / bit0: MAG |
| R#2 | Pattern Name Table Address | 0 | A17〜A10（ベースアドレス [17:10]） |
| R#3 | Color Table Address (L) | 0 | A13〜A6（カラーテーブルベース [13:6]） |
| R#4 | Pattern Generator Table Address | 0 | A17〜A11（ベース [17:11]） |
| R#5 | Sprite Attribute Table Address (L) | 0 | A14〜A7（ベース [14:7]）。bit1 / bit0 はプレーン番号との AND マスクとしても機能 |
| R#6 | Sprite Pattern Generator Table Address | 0 | A17〜A11（ベース [17:11]） |

| R# | 名称 | リセット値 | 主な内容 |
|---|---|---|---|
| R#7 | Background Color | 0 | 周辺色（BD7〜BD0、全ビット格納） |
| R#8 | Mode2 | VR=1、他は 0 | bit5: TP / bit3: VR / bit1: SPD |
| R#9 | Mode3 | すべて 0 | bit7: LN / bit3: IL / bit2: EO / bit1: NT |
| R#10 | Color Table Address (H) | 0 | A17〜A14（カラーテーブルベース [17:14]） |
| R#11 | Sprite Attribute Table Address (H) | 0 | A17〜A15（ベース [17:15]） |
| R#12 | Text Color / Back Color | 0 | 上位ニブル: テキスト色 / 下位ニブル: バック色（ブリンク時は R#7 の代わりに使用） |
| R#13 | Blinking Period | 0 | bit7〜4: CN3〜CN0 / bit3〜0: CF3〜CF0 |
| R#14 | VRAM Access Base Address | アドレス 0 | A17〜A14（CPU アクセス用アドレスの上位） |
| R#15 | Status Register Pointer | 0 | 読み出したいステータスレジスタ番号（S3〜S0） |
| R#16 | Color Palette Pointer | ポインタ 0 | パレット書き込み先の番号。EPAL=0 時は 0〜15、EPAL=1 時は 0〜255 |
| R#17 | Control Register Pointer | ポインタ 0, AII=0 | bit7: AII（自動インクリメント） / bit5〜0: レジスタ番号 |
| R#18 | Display Adjust | 0 | bit7〜4: V3〜V0（垂直位置） / bit3〜0: H3〜H0（水平位置） |
| R#19 | Interrupt Line | 0 | ライン割り込みを発生させるライン番号（IL7〜IL0） |
| R#20 | Mode5 | すべて 0 | bit7: S16 / bit6: CEIE / bit5: FIL / bit4: EPAL / bit3: SP3 / bit2: ILNS / bit1: SVNS（注 1） / bit0: HS |
| R#21 | Mode6 | V58=1 | bit7: 0（固定） / bit6: 0（固定） / bit5: 1（固定） / bit4: 1（固定） / bit3: 1 |

| R# | 名称 | リセット値 | 主な内容 |
|---|---|---|---|
| | | | （固定）/ bit2: 0（固定）/ bit1: 1（固定）/ bit0: V58（1: 互換モード / 0: 拡張モード） |
| R#22 | — | — | 未実装（書き込みは無視される） |
| R#23 | Display Offset | 0 | 垂直スクロールオフセット（DO7〜DO0） |
| R#24 | — | — | 未実装（書き込みは無視される） |
| R#25 | Mode4 | すべて 0 | bit7: SPS（注 2）/ bit6: CMD / bit5: VDS（注 3）/ bit4: YAE / bit3: YJK / bit2: WTE（注 3）/ bit1: MSK / bit0: SP2 |
| R#26 | Horizontal Offset (By Character) | 0 | HO8〜HO3（水平スクロール、文字単位） |
| R#27 | Horizontal Offset (By Dot) | 0 | HO2〜HO0（水平スクロール、ドット単位） |
| R#28 〜 R#31 | — | — | 未実装（書き込みは無視される） |

- 注 1: SVNS は、スプライトが垂直スクロール（R#23）の影響を受けなくなるビットで、スプライトは Y=0 で画面の一番上に表示される画面固定の位置になります（第 3 章・第 6 章参照）。ILNS は、ライン割り込み（R#19）のライン番号を垂直スクロール（R#23）の影響を受けず、画面の一番上を基準とする絶対位置にするビットです（第 3 章・第 10 章参照）。FIL はフラットインターレースで、インターレース時に page0 と page1 を VRAM 上で上下につなげて表示するモードです（第 3 章参照）。
- 注 2: SPS は V9968 の新設ビットです。拡張モード（V58 = 0）のときにのみ書き込めます。
- 注 3: VDS / WTE は V9958 のレイアウトでの名称で、V9968 では格納されません。

## A.2 コマンド関係（R#32〜R#58）

すべて書き込み専用です。R#46 への書き込みでコマンドが起動します。

| R# | 名称 | リセット値 | 主な内容 |
|---|---|---|---|
| R#32 | Source X (L) | 0 | SX[7:0] |
| R#33 | Source X (H) | 0 | SX[11:8] |
| R#34 | Source Y (L) | 0 | SY[7:0] |
| R#35 | Source Y (H) | 0 | SY[12:8] |
| R#36 | Destination X (L) | 0 | DX[7:0] |
| R#37 | Destination X (H) | 0 | DX[8] |
| R#38 | Destination Y (L) | 0 | DY[7:0] |
| R#39 | Destination Y (H) | 0 | DY[10:8]（256K 時）／ DY[9:8]（128K 時、DY[10] は 0 固定） |
| R#40 | Number Of Dots X (L) | 0 | NX[7:0] |
| R#41 | Number Of Dots X (H) | 0 | NX[10:8] |
| R#42 | Number Of Dots Y (L) | 0 | NY[7:0] |
| R#43 | Number Of Dots Y (H) | 0 | NY[10:8]（R#39 と同じモード依存） |
| R#44 | Color | 0 | カラーコード。HMMC / LMMC / LFMC では CPU 転送データの受け口を兼ねる。LFMC / LFMM では前景色 |
| R#45 | Argument | すべて 0 | bit7: FG4（注 4） / bit6: MXC（互換モード）・XHR（拡張モード） / bit5: MXD / |

| R# | 名称 | リセット値 | 主な内容 |
|---|---|---|---|
| | | | bit4: MXS（注 4） / bit3: DIY / bit2: DIX / bit1: EQ / bit0: MAJ |
| R#46 | Command | STOP / 論理演算 0 | bit7〜4: コマンドコード（付録 D） / bit3〜0: ロジカルオペレーション |
| R#47 | Rotation Vector X (L) | 0 | VX[7:0]（LRMM 用、符号付き 8.8 固定小数点） |
| R#48 | Rotation Vector X (H) | 0 | VX[15:8] |
| R#49 | Rotation Vector Y (L) | 0 | VY[7:0] |
| R#50 | Rotation Vector Y (H) | 0 | VY[15:8] |
| R#51 | Window Start X (L) | 0 | WSX[7:0]（LRMM のみで使用） |
| R#52 | Window Start X (H) | 0 | WSX[8] |
| R#53 | Window Start Y (L) | 0 | WSY[7:0] |
| R#54 | Window Start Y (H) | 0 | WSY[10:8] |
| R#55 | Window End X (L) | WEX 全体で 1FFh | WEX[7:0] |
| R#56 | Window End X (H) | （同上） | WEX[8] |
| R#57 | Window End Y | WEY 全体 | WEY[7:0] |

| R# | 名称 | リセット値 | 主な内容 |
|---|---|---|---|
| | (L) | で 7FFh | |
| R#58 | Window End Y (H) | （同上） | WEY[10:8] |
| R#59 ～ R#63 | — | — | 未実装（書き込みは無視される） |

- 注 4: FG4 は、1 のとき画面モード指定にかかわらず VDP コマンドが Graphic4（SCREEN 5）として動作するビットです（SpriteMode3 のパターンデータへの描画などに使用します。表示自体が Graphic4 の場合は、FG4 の有無で動作は変わりません。第 3 章・第 8 章参照）。FG4 は拡張モードでないと 0 にマスクされて格納されます。XHR は拡張モードで LRMM の画素アスペクト補正（縦長画素として扱う）に使うビットです（第 8 章参照）。また、V9968 では拡張 RAM が廃止されているため、MXS / MXD の選択対象は VRAM のみです。

なお、R#32 / R#33 / R#34 への書き込みは、LFMC / LFMM 用フォントアドレス（18 ビット）の [7:0] / [15:8] / [17:16] にも同時に作用します（第 8 章参照）。

## A.3 レジスタ別ビット配置図（R#0～R#27）

各レジスタのビット配置を番号順に示します。図の見方は第 3 章と同じで、左が MSB、右が LSB です。各フィールドの下のブラケットがそのフィールドの説明で、フラグ類は 0 と 1 の意味を示します。「・」は未使用ビット、グレーの欄は固定値、または V9968 では格納されないビットを表します。未実装の R#22 と R#24 は、全ビット未使用として図を示します。R#28～R#31 は範囲として A.1 の表にまとめてあるため、個別の図はありません。

### R#0 Mode0

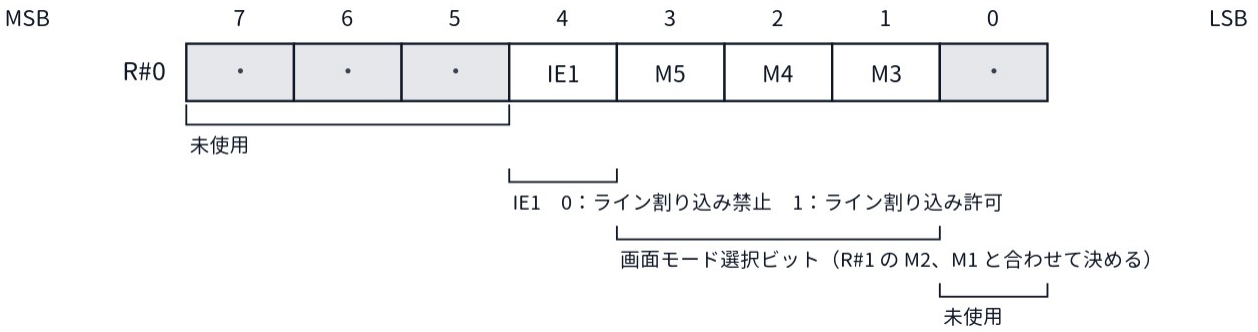



図 A.1 R#0 Mode0 のビット配置

### R#1 Mode1




図 A.2 R#1 Mode1 のビット配置

### R#2 Pattern Name Table Address

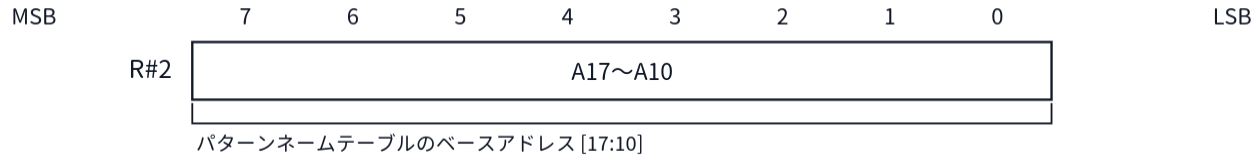



図 A.3 R#2 Pattern Name Table Address のビット配置

### R#3 Color Table Address (L)

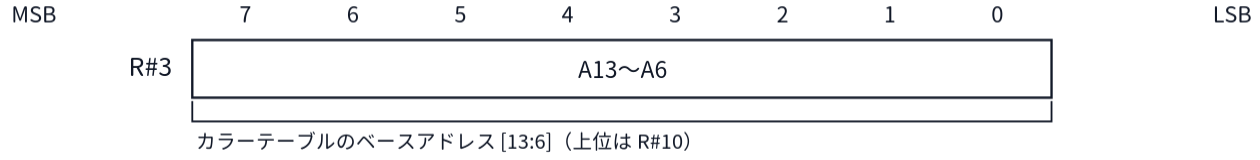



図 A.4 R#3 Color Table Address (L) のビット配置

### R#4 Pattern Generator Table Address

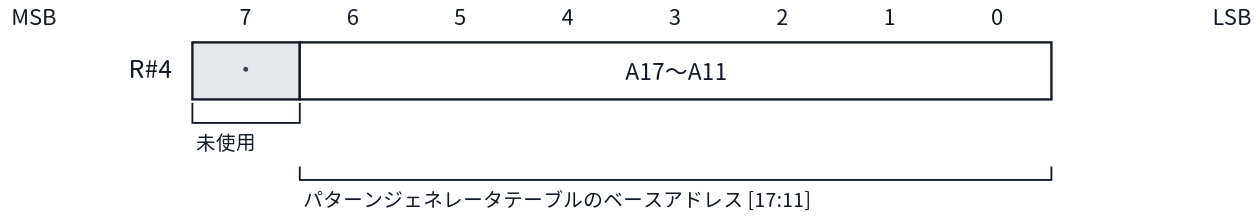


図 A.5 R#4 Pattern Generator Table Address のビット配置

### R#5 Sprite Attribute Table Address (L)

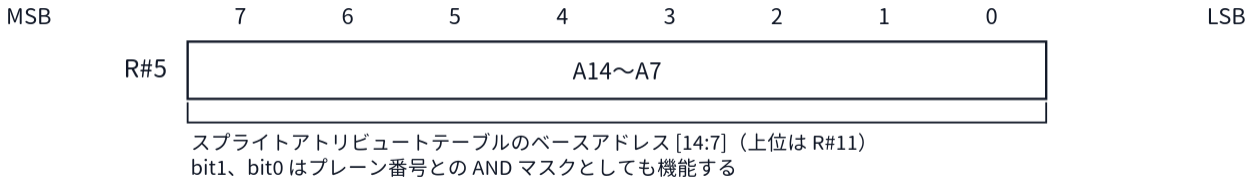



図 A.6 R#5 Sprite Attribute Table Address (L) のビット配置

### R#6 Sprite Pattern Generator Table Address

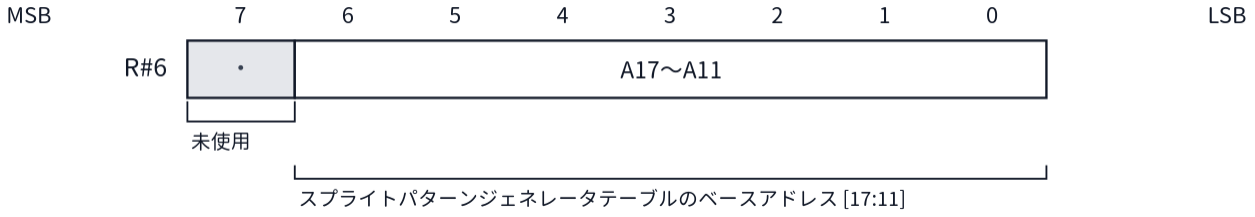



図 A.7 R#6 Sprite Pattern Generator Table Address のビット配置

### R#7 Background Color

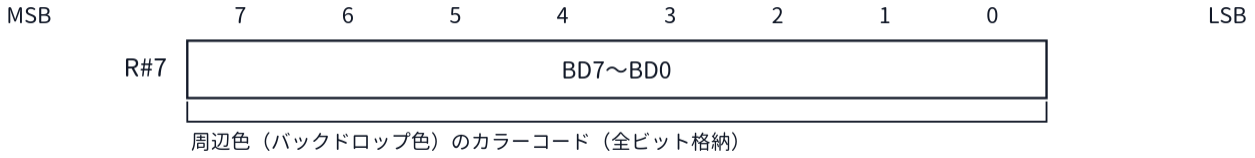



図 A.8 R#7 Background Color のビット配置

### R#8 Mode2

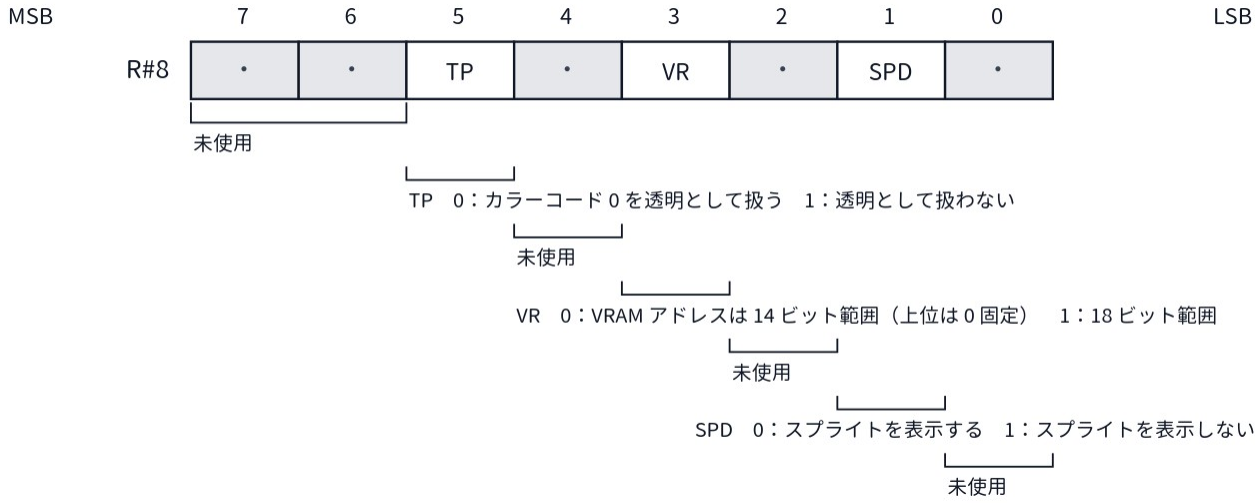



図 A.9 R#8 Mode2 のビット配置

### R#9 Mode3

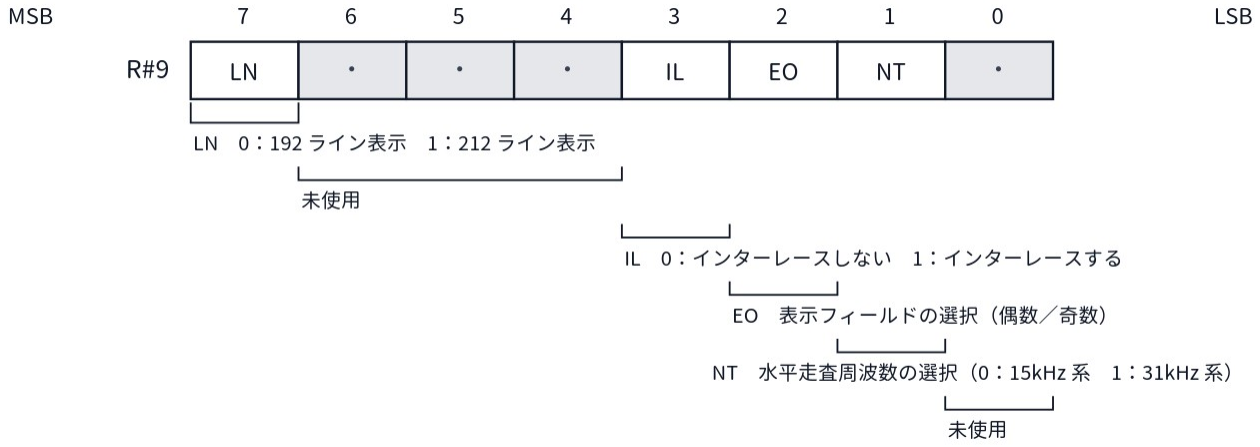



図 A.10 R#9 Mode3 のビット配置

### R#10 Color Table Address (H)

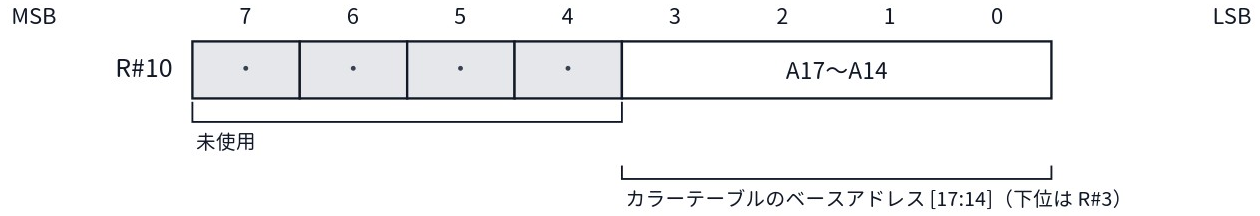



図 A.11 R#10 Color Table Address (H) のビット配置

### R#11 Sprite Attribute Table Address (H)

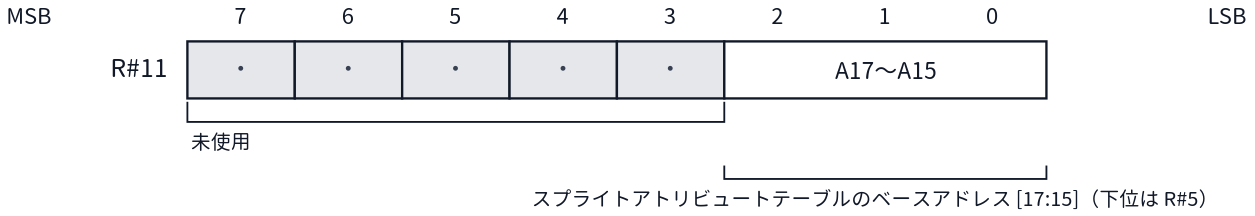



図 A.12 R#11 Sprite Attribute Table Address (H) のビット配置

### R#12 Text Color / Back Color

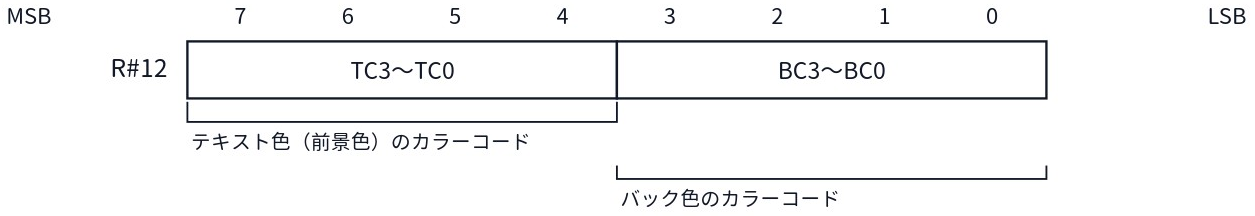



図 A.13 R#12 Text Color / Back Color のビット配置

### R#13 Blinking Period

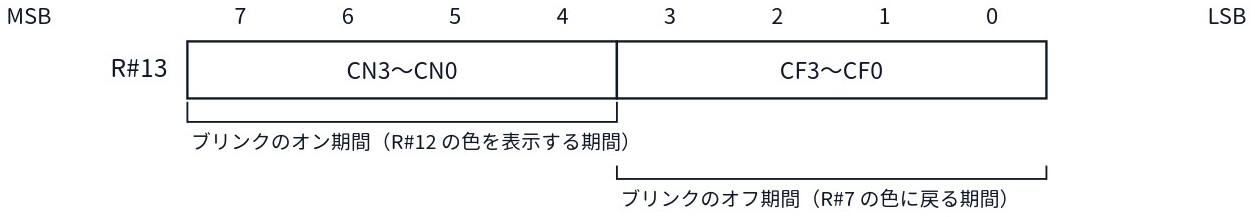



図 A.14 R#13 Blinking Period のビット配置

### R#14 VRAM Access Base Address

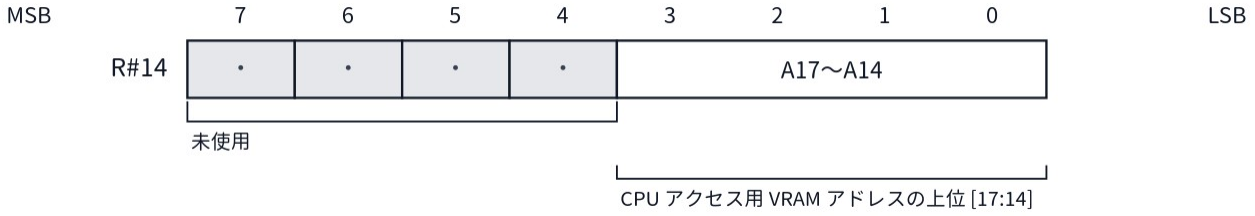



図 A.15 R#14 VRAM Access Base Address のビット配置

### R#15 Status Register Pointer

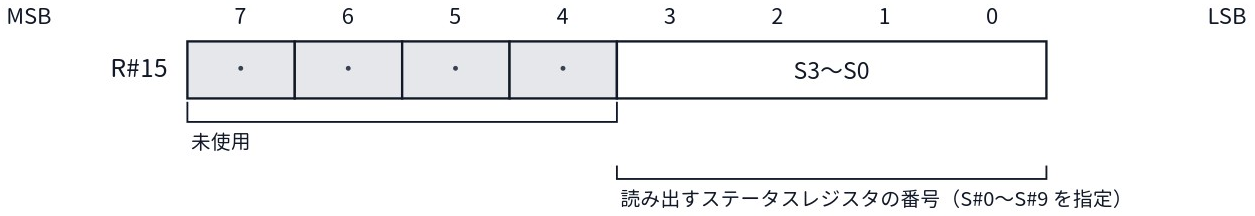



図 A.16 R#15 Status Register Pointer のビット配置

### R#16 Color Palette Pointer

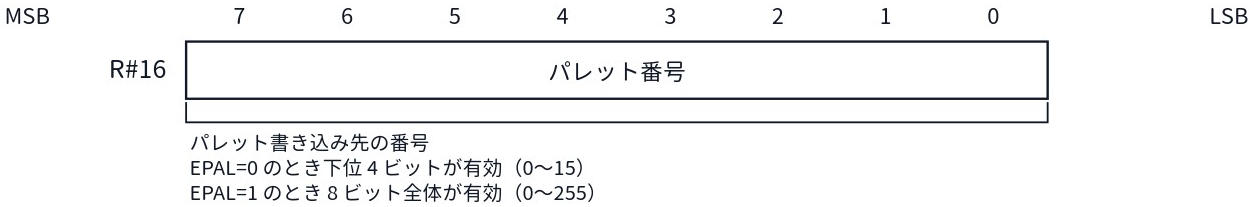



図 A.17 R#16 Color Palette Pointer のビット配置

### R#17 Control Register Pointer

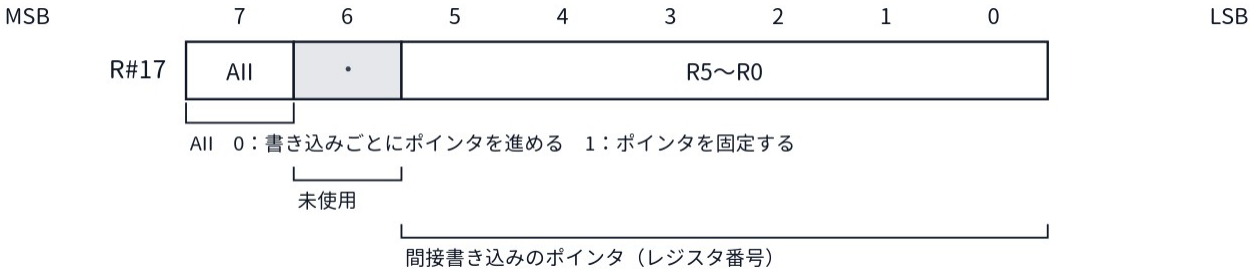



図 A.18 R#17 Control Register Pointer のビット配置

### R#18 Display Adjust

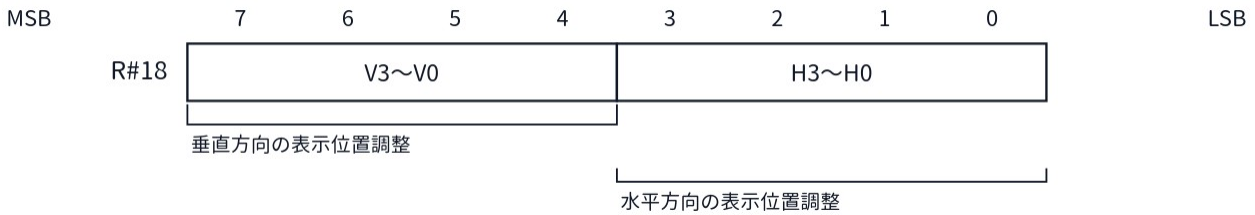



図 A.19 R#18 Display Adjust のビット配置

### R#19 Interrupt Line

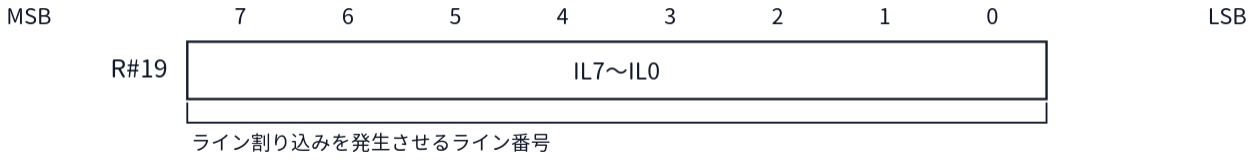



図 A.20 R#19 Interrupt Line のビット配置

### R#20 Mode5




図 A.21 R#20 Mode5 のビット配置

### R#21 Mode6

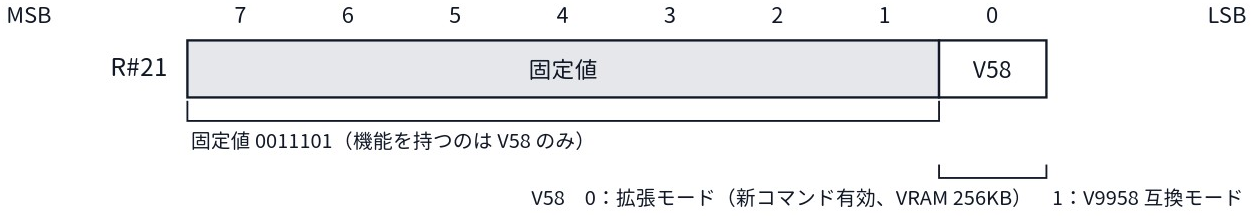



図 A.22 R#21 Mode6 のビット配置

### R#22 （未実装）

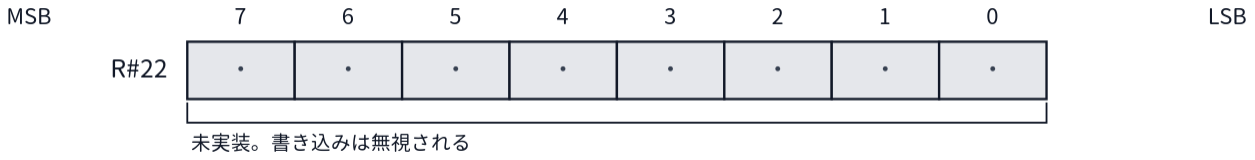


図 A.23 R#22（未実装）のビット配置

### R#23 Display Offset

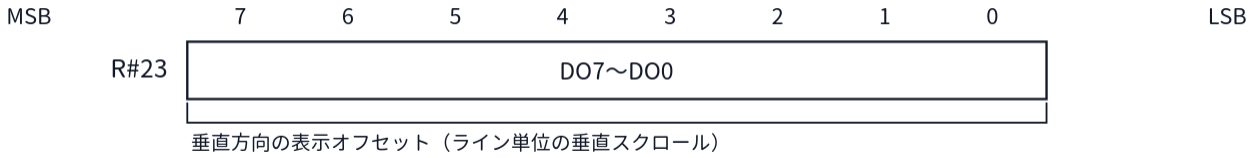



図 A.24 R#23 Display Offset のビット配置

### R#24 （未実装）

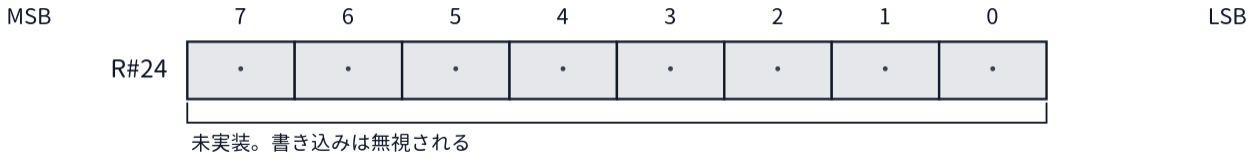



図 A.25 R#24（未実装）のビット配置

### R#25 Mode4




図 A.26 R#25 Mode4 のビット配置

### R#26 Horizontal Offset (By Character)

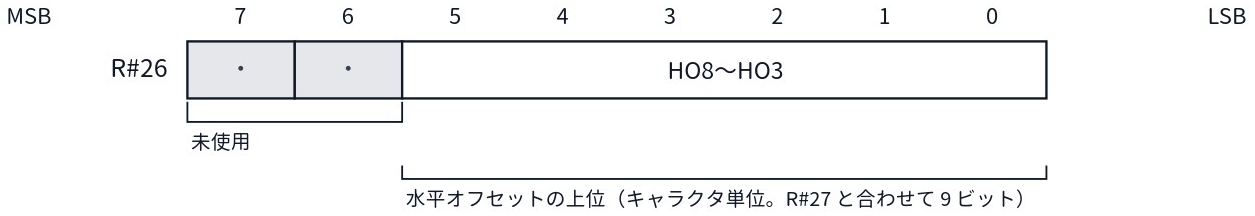



図 A.27 R#26 Horizontal Offset (By Character) のビット配置

### R#27 Horizontal Offset (By Dot)

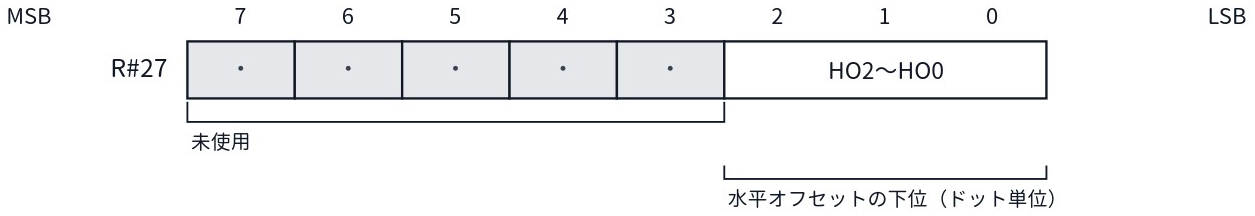



図 A.28 R#27 Horizontal Offset (By Dot) のビット配置

## A.4 レジスタ別ビット配置図（R#32〜R#58）

コマンド関係のレジスタのビット配置を番号順に示します。図の見方は A.3 と同じです。各コマンドでの使い方の詳細は第 8 章を参照してください。R#59〜R#63 は未実装のため図はありません。

### R#32 Source X (L)

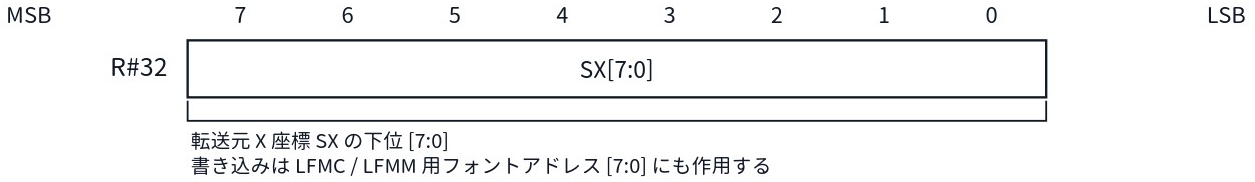



図 A.29 R#32 Source X (L) のビット配置

### R#33 Source X (H)

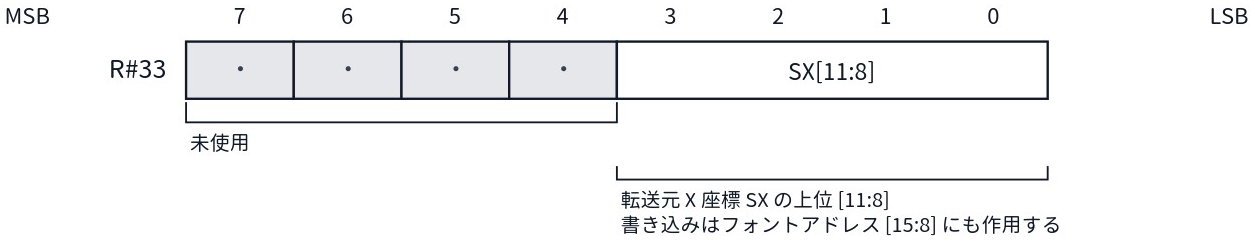



図 A.30 R#33 Source X (H) のビット配置

### R#34 Source Y (L)

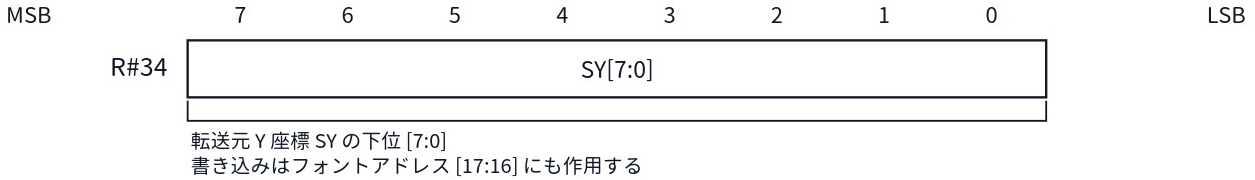



図 A.31 R#34 Source Y (L) のビット配置

### R#35 Source Y (H)

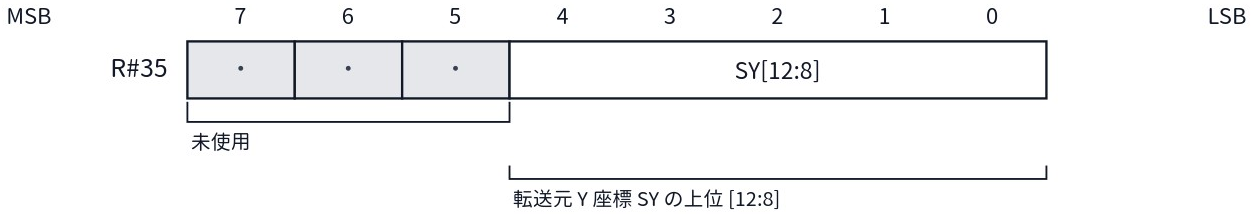



図 A.32 R#35 Source Y (H) のビット配置

### R#36 Destination X (L)

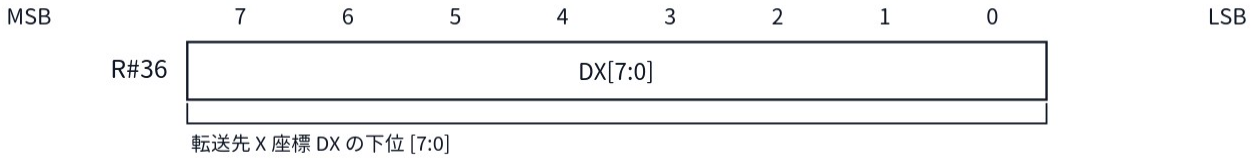



図 A.33 R#36 Destination X (L) のビット配置

### R#37 Destination X (H)

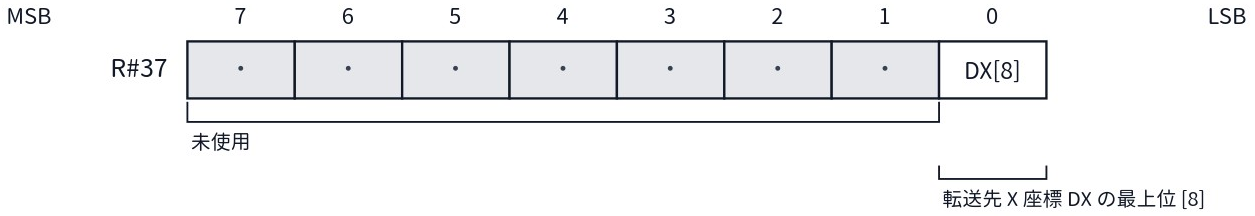



図 A.34 R#37 Destination X (H) のビット配置

### R#38 Destination Y (L)

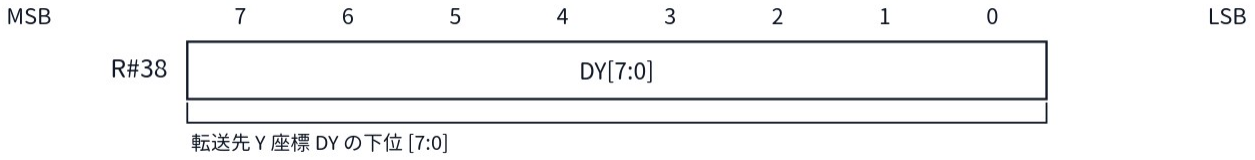



図 A.35 R#38 Destination Y (L) のビット配置

### R#39 Destination Y (H)

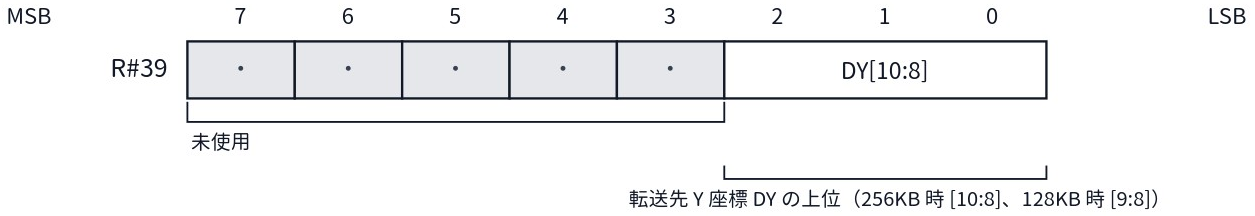



図 A.36 R#39 Destination Y (H) のビット配置

### R#40 Number Of Dots X (L)




図 A.37 R#40 Number Of Dots X (L) のビット配置

### R#41 Number Of Dots X (H)

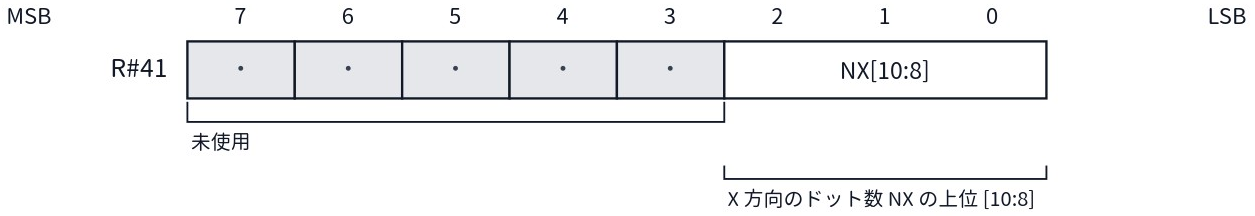



図 A.38 R#41 Number Of Dots X (H) のビット配置

### R#42 Number Of Dots Y (L)

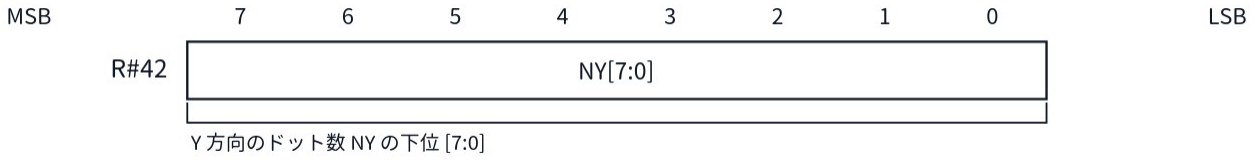



図 A.39 R#42 Number Of Dots Y (L) のビット配置

### R#43 Number Of Dots Y (H)

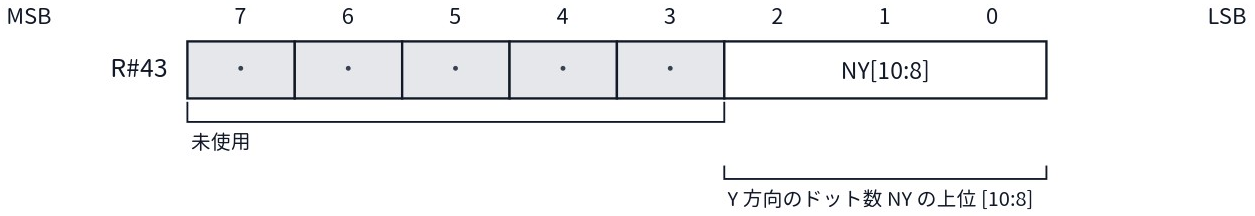



図 A.40 R#43 Number Of Dots Y (H) のビット配置

### R#44 Color




図 A.41 R#44 Color のビット配置

### R#45 Argument

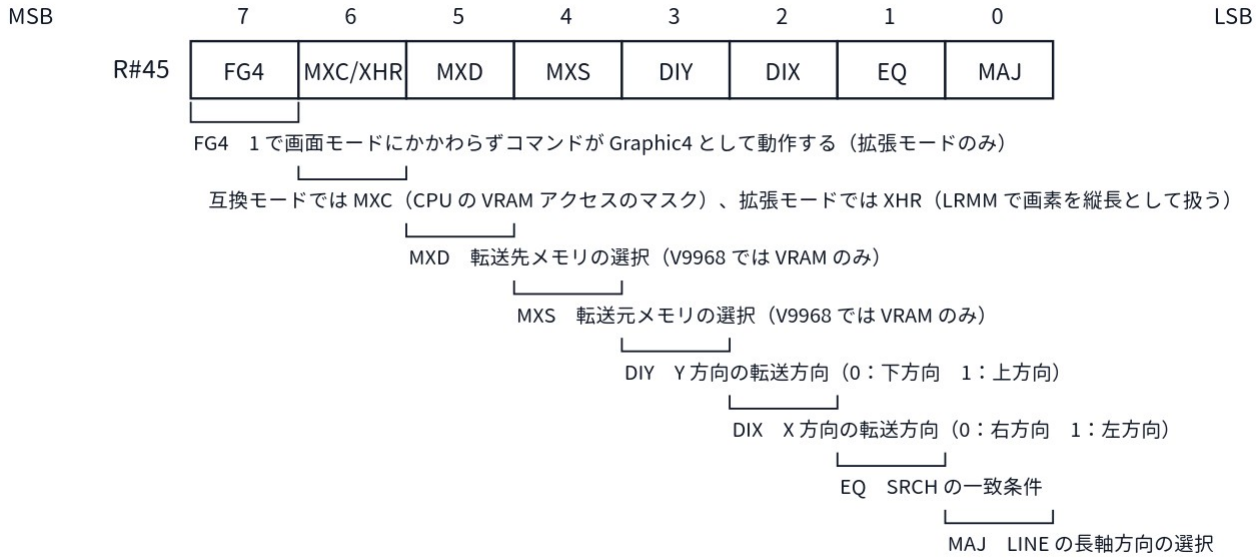



図 A.42 R#45 Argument のビット配置

### R#46 Command

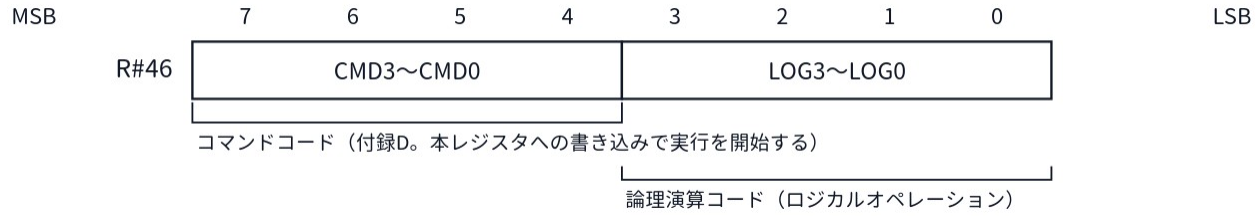



図 A.43 R#46 Command のビット配置

### R#47 Rotation Vector X (L)




図 A.44 R#47 Rotation Vector X (L) のビット配置

### R#48 Rotation Vector X (H)

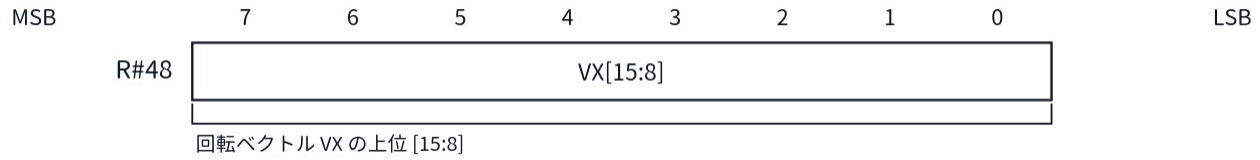


図 A.45 R#48 Rotation Vector X (H) のビット配置

### R#49 Rotation Vector Y (L)




図 A.46 R#49 Rotation Vector Y (L) のビット配置

### R#50 Rotation Vector Y (H)

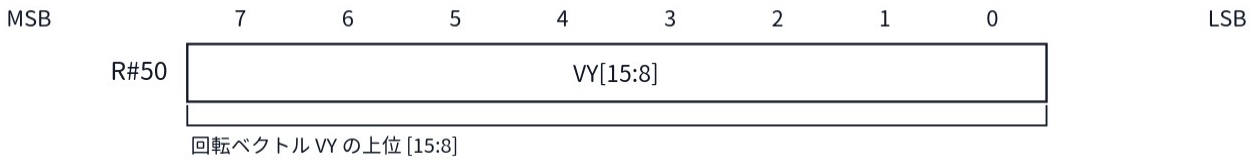



図 A.47 R#50 Rotation Vector Y (H) のビット配置

### R#51 Window Start X (L)




図 A.48 R#51 Window Start X (L) のビット配置

### R#52 Window Start X (H)

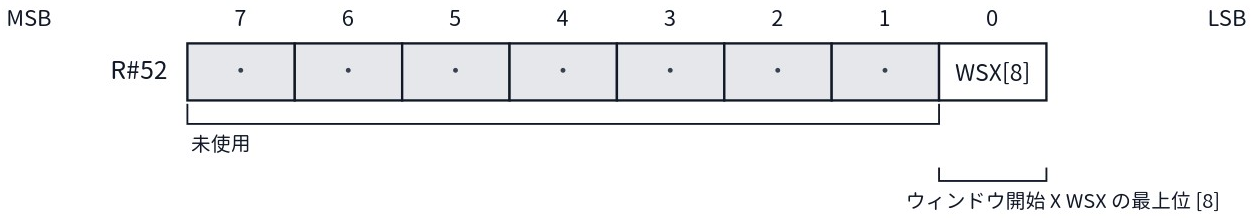



図 A.49 R#52 Window Start X (H) のビット配置

### R#53 Window Start Y (L)

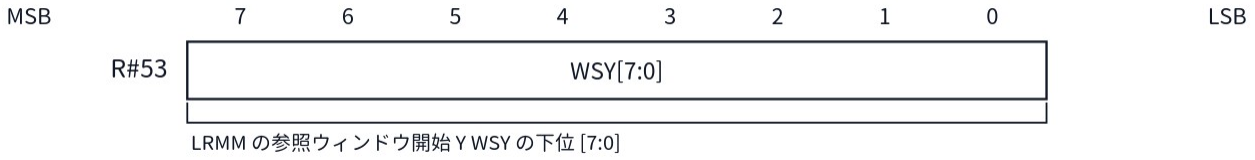



図 A.50 R#53 Window Start Y (L) のビット配置

### R#54 Window Start Y (H)

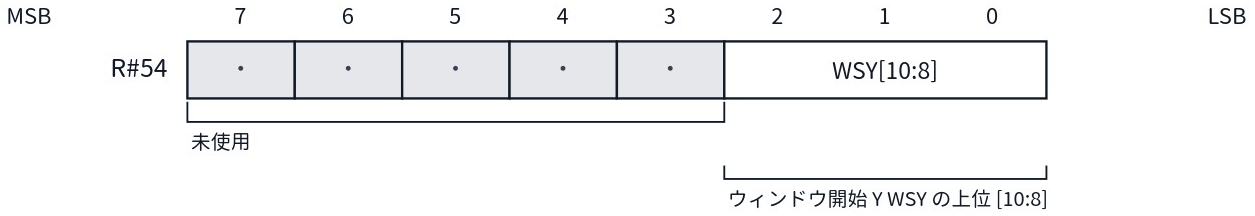



図 A.51 R#54 Window Start Y (H) のビット配置

### R#55 Window End X (L)

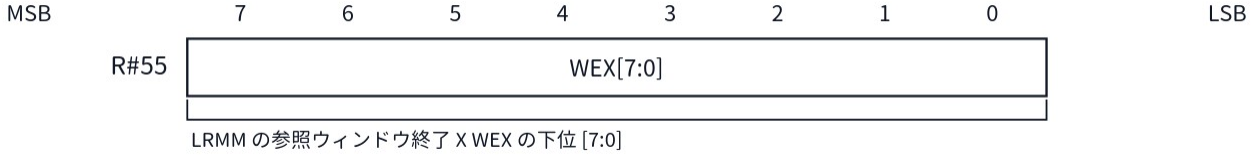



図 A.52 R#55 Window End X (L) のビット配置

### R#56 Window End X (H)

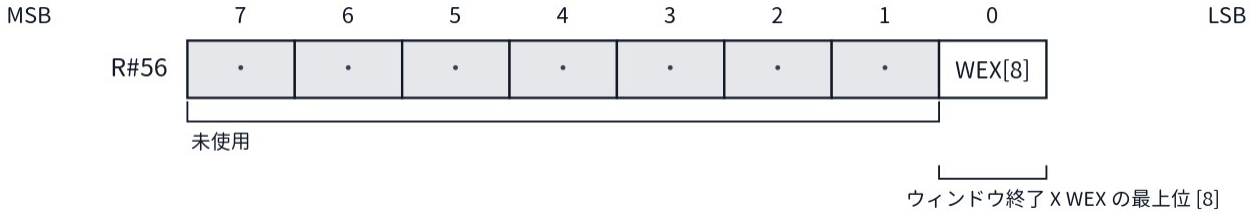



図 A.53 R#56 Window End X (H) のビット配置

### R#57 Window End Y (L)

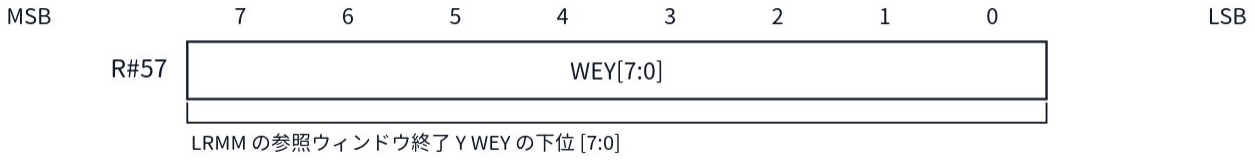



図 A.54 R#57 Window End Y (L) のビット配置

### R#58 Window End Y (H)

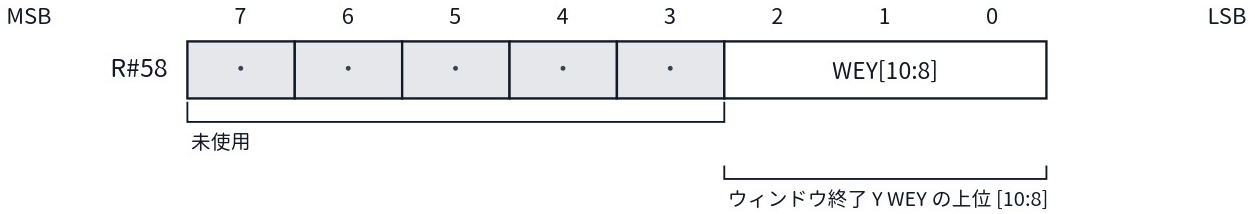



図 A.55 R#58 Window End Y (H) のビット配置

## 付録 B ステータスレジスター覧

すべて読み出し専用です。R#15 で番号を指定し、Port#1 を読みます（第 2 章・第 4 章参照）。

| S# | 主な内容 | 読み出し時の副作用 |
|---|---|---|
| S#0 | bit7: F（フレーム割り込み） / bit6: 5S（表示上限超過） / bit5: C（スプライト衝突） / bit4〜0: 5TH（上限を超えたプレーン番号） | F / C / 5S がクリアされる |
| S#1 | bit5〜1: ID（互換モード 00010 / 拡張モード 00011） / bit0: FH（ライン割り込み）。bit7〜6 は 0 固定 | FH がクリアされる |
| S#2 | bit7: TR / bit6: VR / bit5: HR / bit4: BD / bit3: 1 固定 / bit2: 1 固定 / bit1: EO / bit0: CE | なし |
| S#3 | スプライト衝突 X 座標 [7:0] | なし |
| S#4 | 衝突 X[8]（bit0）。bit7〜1 は 1 固定 | なし |
| S#5 | スプライト衝突 Y 座標 [7:0] | 衝突座標（X / Y）がク |

| S# | 主な内容 | 読み出し時の副作用 |
|---|---|---|
| | | リアされる |
| S#6 | 衝突 Y[9:8]（bit1〜0）。bit7〜2 は 1 固定 | なし |
| S#7 | POINT / LMCM で読み出された画素値 | LMCM の転送制御に使われる |
| S#8 | SRCH 境界 X 座標 [7:0] | なし |
| S#9 | 境界 X[8]（bit0）。bit7〜1 は 1 固定 | BD（S#2 bit4）がクリアされる |
| S#10〜S#15 | 未実装 | 常に FFh が返る |

## B.1 レジスタ別ビット配置図（S#0〜S#9）

各ステータスレジスタのビット配置を番号順に示します。図の見方は付録 A と同じです。読み出し時の副作用については本付録 B の表と第 4 章を参照してください。S#10〜S#15 は未実装で、読み出すと常に FFh が返ります。

### S#0

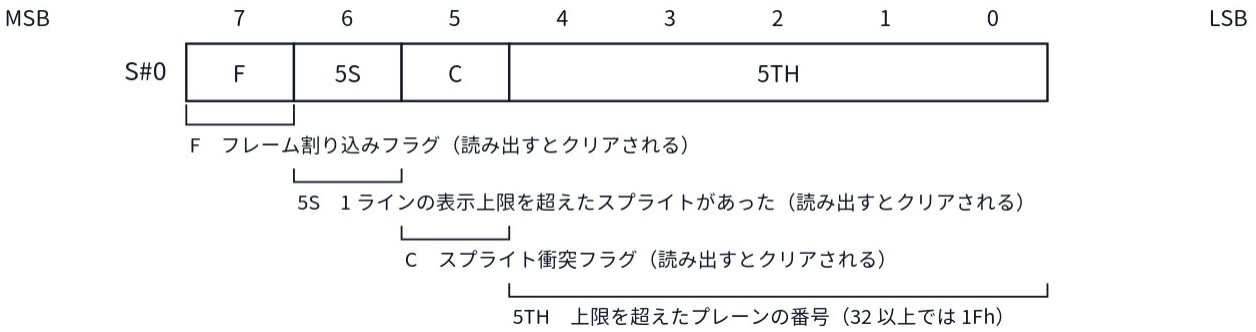



図 B.1 S#0 のビット配置

### S#1

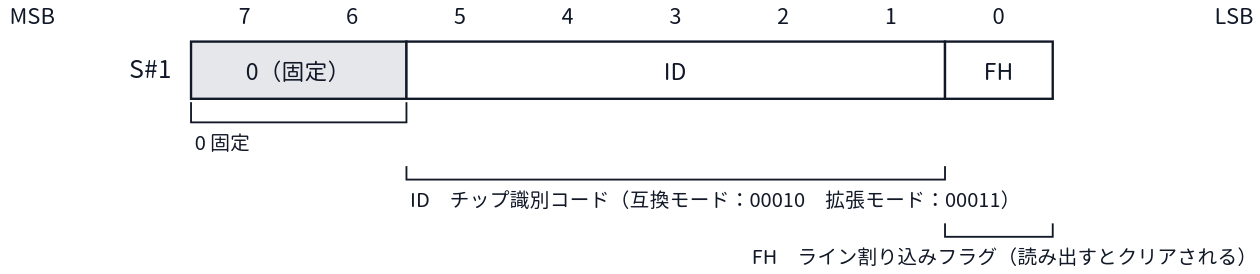



図 B.2 S#1 のビット配置

### S#2

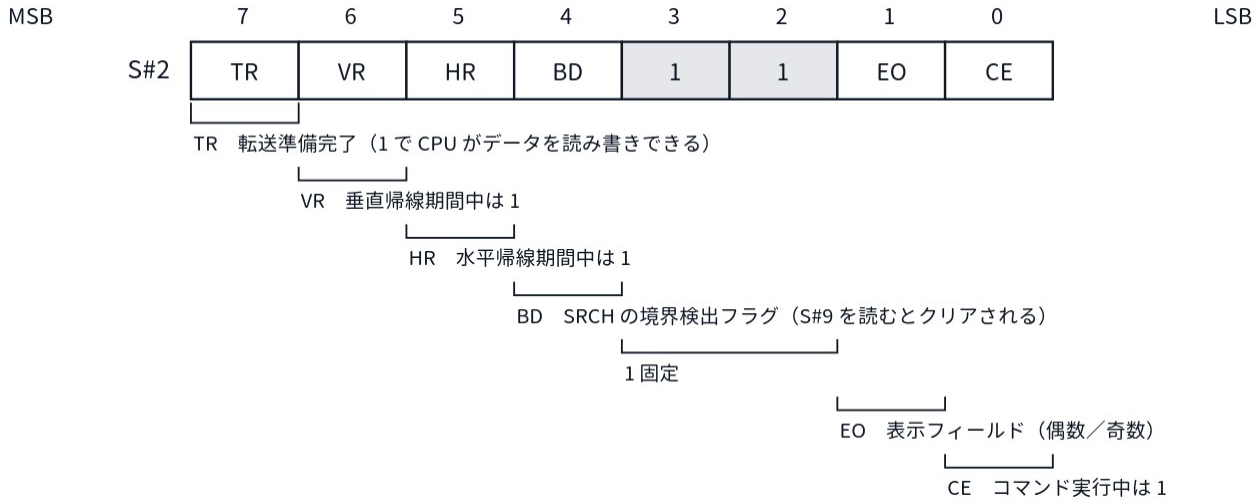



図 B.3 S#2 のビット配置

### S#3

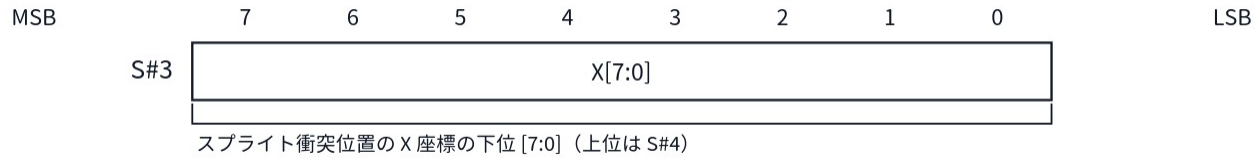



図 B.4 S#3 のビット配置

### S#4

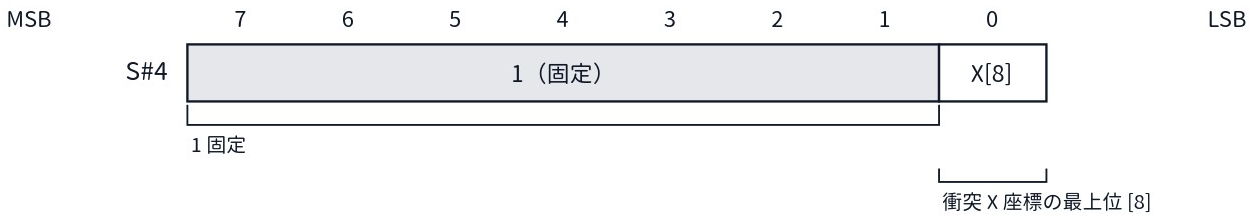



図 B.5 S#4 のビット配置

### S#5

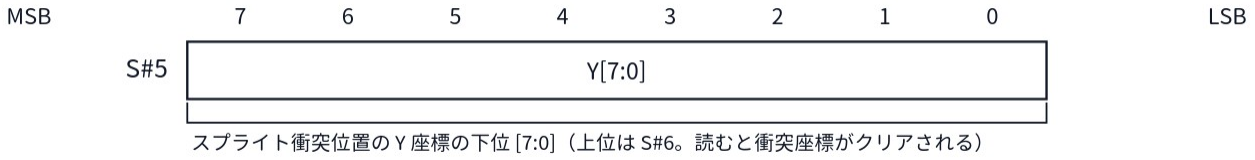



図 B.6 S#5 のビット配置

### S#6

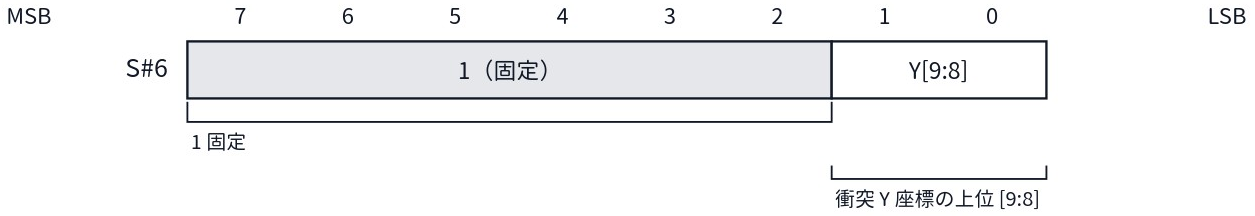



図 B.7 S#6 のビット配置

### S#7

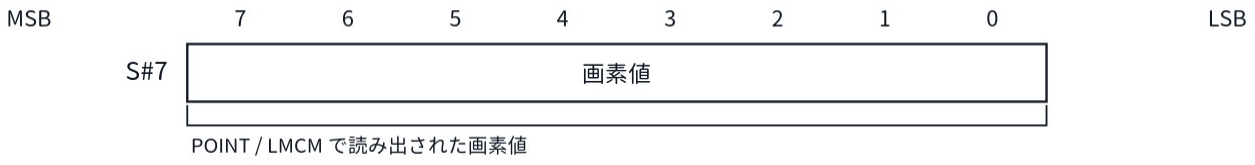



図 B.8 S#7 のビット配置

**S#8**

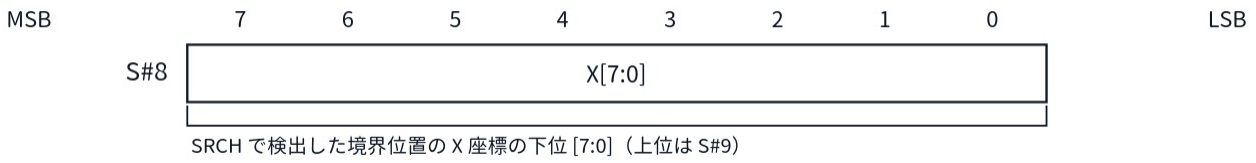



図 B.9 S#8 のビット配置

**S#9**

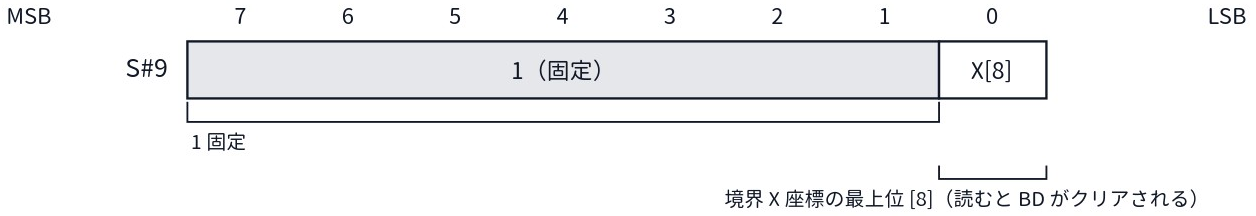



図 B.10 S#9 のビット配置

## 付録 C スクリーンモード一覧

モードは R#0 の M5 / M4 / M3 と R#1 の M1 / M2 の組み合わせで選択します（第 5 章参照）。Text1 と Multicolor では M3 はドントケアです（表の値は代表値。M3 = 1 でも同じモードになります）。

| モード | 略称 | M5 M4 M3 M2 M1 | BASIC の SCREEN | 画面 | 色 | スプライト |
|---|---|---|---|---|---|---|
| Graphic1 | G1 | 0 0 0 0 0 | SCREEN 1 | 32×24 パターン | パレット 16 色 | モード 1 |
| Text1 | T1 | 0 0 x 0 1 | SCREEN 0（WIDTH 40） | 40×24 文字（6×8ドット） | 2 色 | なし |
| Multicolor | MC | 0 0 x 1 0 | SCREEN 3 | 64×48 ブロック | パレット 16 色 | モード 1 |
| Graphic2 | G2 | 0 0 1 0 0 | SCREEN 2 | 256×192ドット | パレット 16 色 | モード 1 |

| モード | 略称 | M5 M4 M3 M2 M1 | BASIC の SCREEN | 画面 | 色 | スプライト |
|---|---|---|---|---|---|---|
| Graphic3 | G3 | 0 1 0 0 0 | SCREEN 4 | 256×192ドット | パレット 16 色 | モード 2 |
| Graphic4 | G4 | 0 1 1 0 0 | SCREEN 5 | 256×212ドット | パレット 16 色 | モード 2 |
| Text2 | T2 | 0 1 0 0 1 | SCREEN 0（WIDTH 80） | 80×24 文字（6×8ドット） | 2 色＋ブリンク色 | なし |
| Graphic5 | G5 | 1 0 0 0 0 | SCREEN 6 | 512×212ドット | パレット 4 色 | モード 2 |
| Graphic6 | G6 | 1 0 1 0 0 | SCREEN 7 | 512×212ドット | パレット 16 色 | モード 2 |
| Graphic7 | G7 | 1 1 1 0 0 | SCREEN 8 | 256×212ドット | 256 色（直接指定） | モード 2 |
| Graphic7-YAE | G7-YAE | 1 1 1 0 0（＋R#25 の YAE） | SCREEN 10 / 11 | 256×212ドット | YJK＋パレット 16 色 | モード 2 |
| Graphic7-YJK | G7-YJK | 1 1 1 0 0（＋R#25 の YJK） | SCREEN 12 | 256×212ドット | YJK（約 19,268 色） | モード 2 |

スプライトモードの列は、Sprite mode3 の選択時（R#20 の SP3、第 6 章参照）を除く従来モードでの対応です。

以下では、モードごとに標準的な VRAM 配置（VRAM マップ）と、その配置にするためのレジスタ設定を示します。ここでいう標準配置とは、MSX-BASIC が SCREEN 文で各モードを初期化したときの配置で、MSX2 テクニカルハンドブック付録の VRAM マップと一致します。レジスタの値は、各テーブルのベースアドレスから第 3 章のビット配置に従って求めたもので、MSX-BASIC 初期化後の実測値とも一致することを確認しています。

図の帯の高さは概略であり、面積比は表しません。アドレスはすべて論理アドレスです。なお、テクニカルハンドブックの VRAM マップにあるパレット退避エリアは、MSX-BASIC がパレット値を退避するための領域で、VDP のレジスタで位置を決めるテーブルではないため、本付録の図には含めていません。拡張モード（V58 = 0）では 20000h～3FFFFh の領

域も使えますが、標準配置はいずれのモードでも 128KB の範囲内に収まります。

各表の読み方についての注意をまとめます。

- R#1 の値には、MSX-BASIC が設定する表示許可（BL = 1）とフレーム割り込み許可（IE0 = 1）が含まれます。モードの選択自体に必要なのは M1／M2 のみです。SI／MAG はいずれも 0（スプライト 8×8 ドット、等倍）です。

- R#7 の値は、MSX-BASIC の初期値（COLOR 文の初期設定による値）です。テキストモードでは上位ニブルが前景色、下位ニブルが背景色を表します。その他のモードでは下位ニブルが周辺色です。
- ビットマップモードの表示ライン数は、特に断らない限り 212 ライン（R#9 の LN = 1）の場合です。

## C.1 Text1（SCREEN 0、WIDTH 40）

スプライトとカラーテーブルは使いません。パターンネームテーブルとパターンジェネレータテーブルだけの、最も単純な配置です。




図 C.1 Text1 の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | M5 = 0、M4 = 0、M3 = 0 | モード選択ビット（M3 はドントケア） |
| R#1 | 70h | BL = 1、IE0 = 1、M1 = 1、M2 = 0 |
| R#2 | 00h | パターンネームテーブル = 00000h |
| R#4 | 01h | パターンジェネレータテーブル = 00800h |

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#7 | F4h | 前景色 15／背景色 4 |

## C.2 Text2（SCREEN 0、WIDTH 80）

ブリンクテーブルが加わります。行数は 24 行（R#9 の LN = 0）が標準です。LN = 1（26.5 行）にすると、パターンネームテーブルは 2,160 バイト（00000h～0086Fh）、ブリンクテーブルは 270 バイト（00800h～0090Dh）まで広がります。




図 C.2 Text2 の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | M5 = 0、M4 = 1、M3 = 0 | モード選択ビット |
| R#1 | 70h | BL = 1、IE0 = 1、M1 = 1、M2 = 0 |
| R#2 | 03h | パターンネームテーブル = 00000h（ベースに効くのは A17～A12 で、下位 2 ビットは 1 に設定されます） |
| R#3 | 27h | ブリンクテーブル = 00800h（R#10 と組み合わせて A16～A9 を指定。下位ビットは 1 に設定されます） |
| R#10 | 00h | ブリンクテーブルのベースアドレス上位 |
| R#4 | 02h | パターンジェネレータテーブル = 01000h |
| R#7 | F4h | 前景色 15／背景色 4（ブリンク色は R#12、周期は R#13） |
| R#9 | LN = 0 | 24 行（LN = 1 で 26.5 行） |

## C.3 Multicolor（SCREEN 3）

パターンジェネレータテーブルが色データを兼ねます。カラーテーブルはありません。スプライトはモード 1 です。

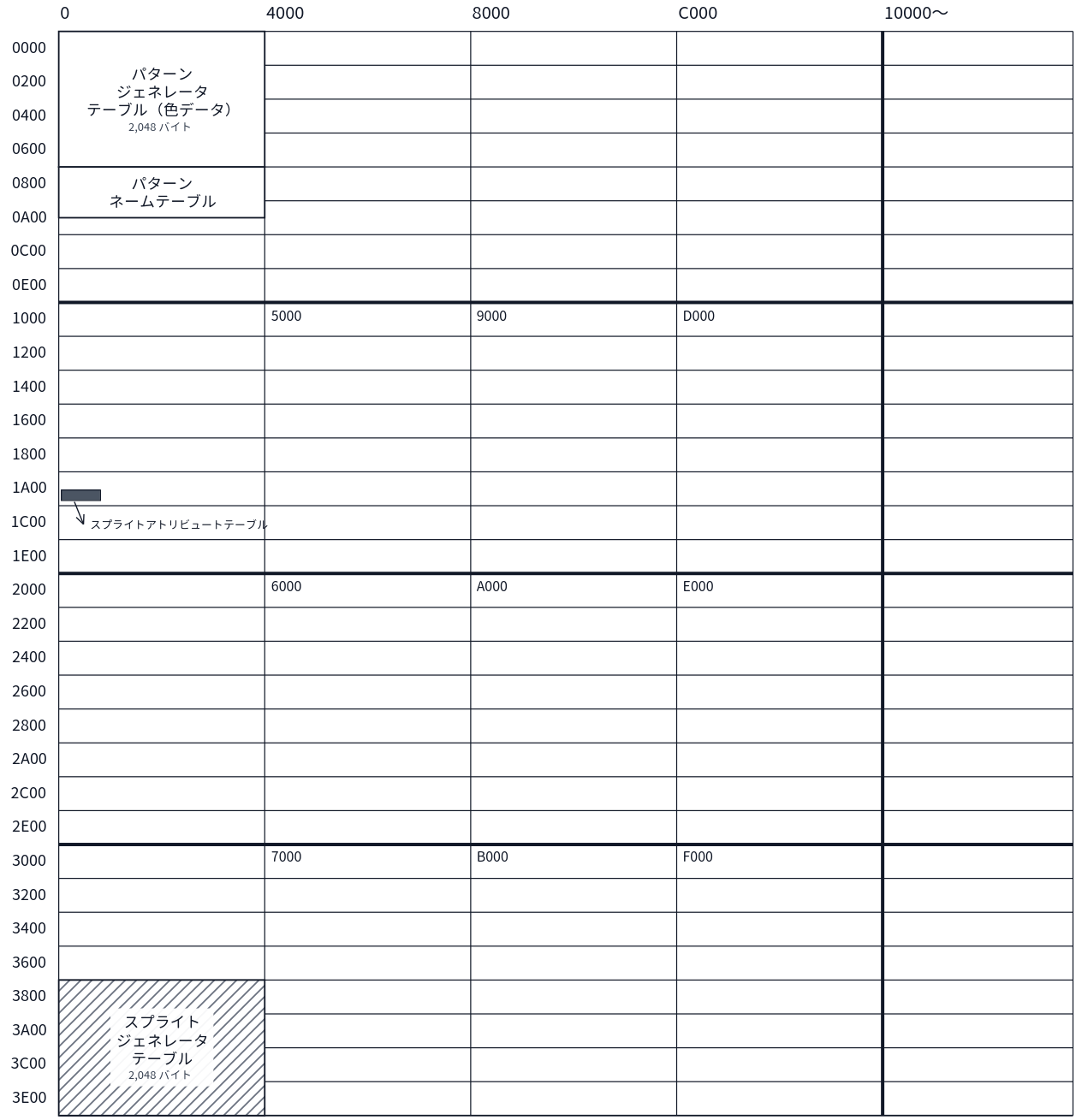



図 C.3 Multicolor の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | M5 = 0、M4 = 0、M3 = 0 | モード選択ビット（M3 はドントケア） |
| R#1 | 68h | BL = 1、IE0 = 1、M1 = 0、M2 = 1、SI = 0、MAG = 0 |
| R#2 | 02h | パターンネームテーブル = 00800h |
| R#4 | 00h | パターンジェネレータテーブル（色データ）= 00000h |

| レジスタ | 標準値 | 内容 |
| --- | --- | --- |
| R#5 | 36h | スプライトアトリビュートテーブル = 01B00h |
| R#11 | 00h | スプライトアトリビュートテーブルのベースアドレス上位 |
| R#6 | 07h | スプライトパターンジェネレータテーブル = 03800h |
| R#7 | 07h | 周辺色 7 |

## C.4 Graphic1（SCREEN 1）

スプライトはモード 1 です。カラーテーブルは 32 バイトと小さく、パターンネームテーブルの直後には大きな未使用領域があります。

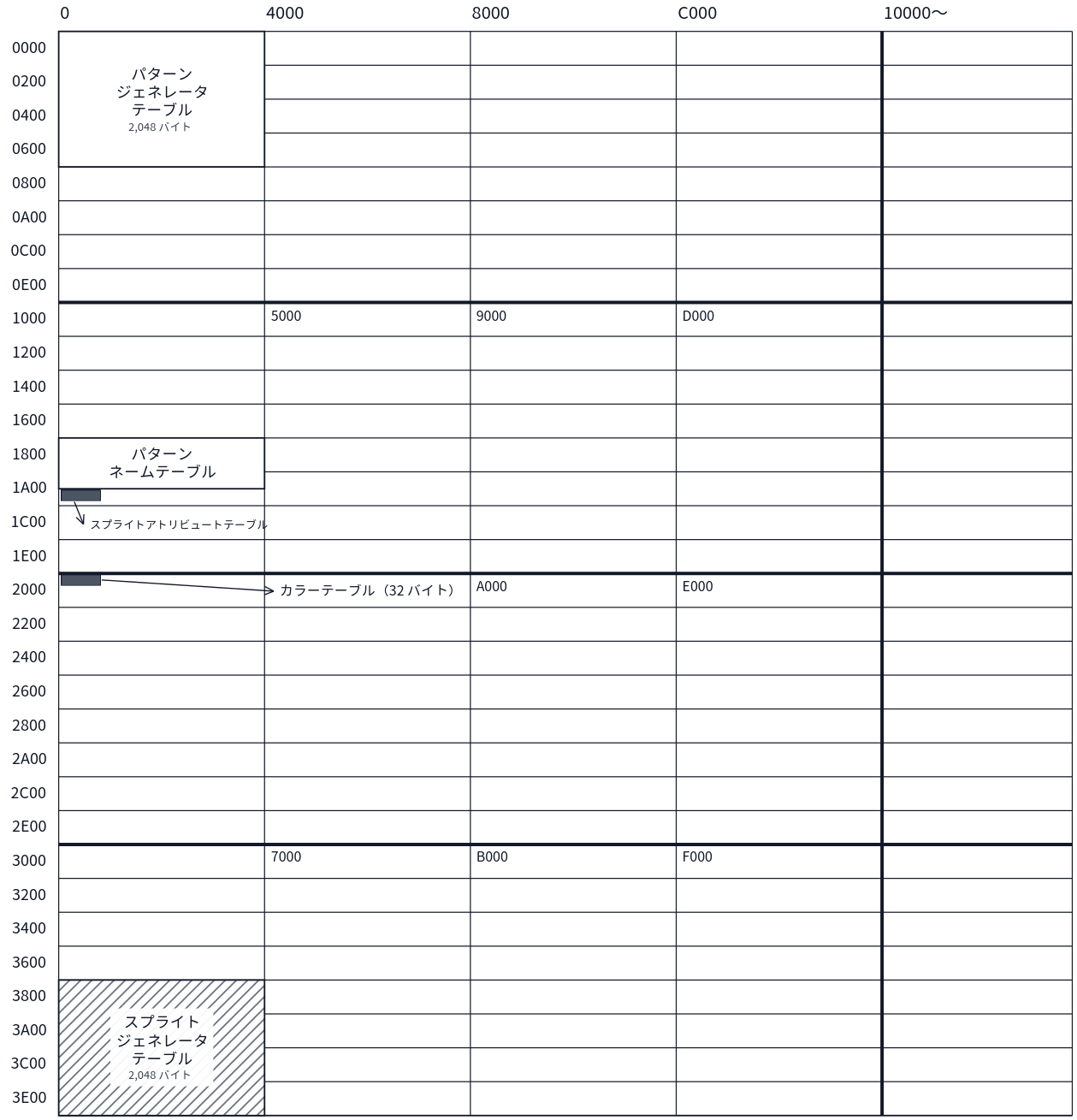



図 C.4 Graphic1 の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | M5 = 0、M4 = 0、M3 = 0 | モード選択ビット |
| R#1 | 60h | BL = 1、IE0 = 1、M1 = 0、M2 = 0、SI = 0、MAG = 0 |
| R#2 | 06h | パターンネームテーブル = 01800h |
| R#3 | 80h | カラーテーブル = 02000h |

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#10 | 00h | カラーテーブルのベースアドレス上位 |
| R#4 | 00h | パターンジェネレータテーブル = 00000h |
| R#5 | 36h | スプライトアトリビュートテーブル = 01B00h |
| R#11 | 00h | スプライトアトリビュートテーブルのベースアドレス上位 |
| R#6 | 07h | スプライトパターンジェネレータテーブル = 03800h |
| R#7 | 07h | 周辺色 7 |

## C.5 Graphic2（SCREEN 2）

パターンジェネレータテーブルとカラーテーブルは 3 バンク構成で、それぞれ 6,144 バイトを使います。スプライトはモード 1 です。

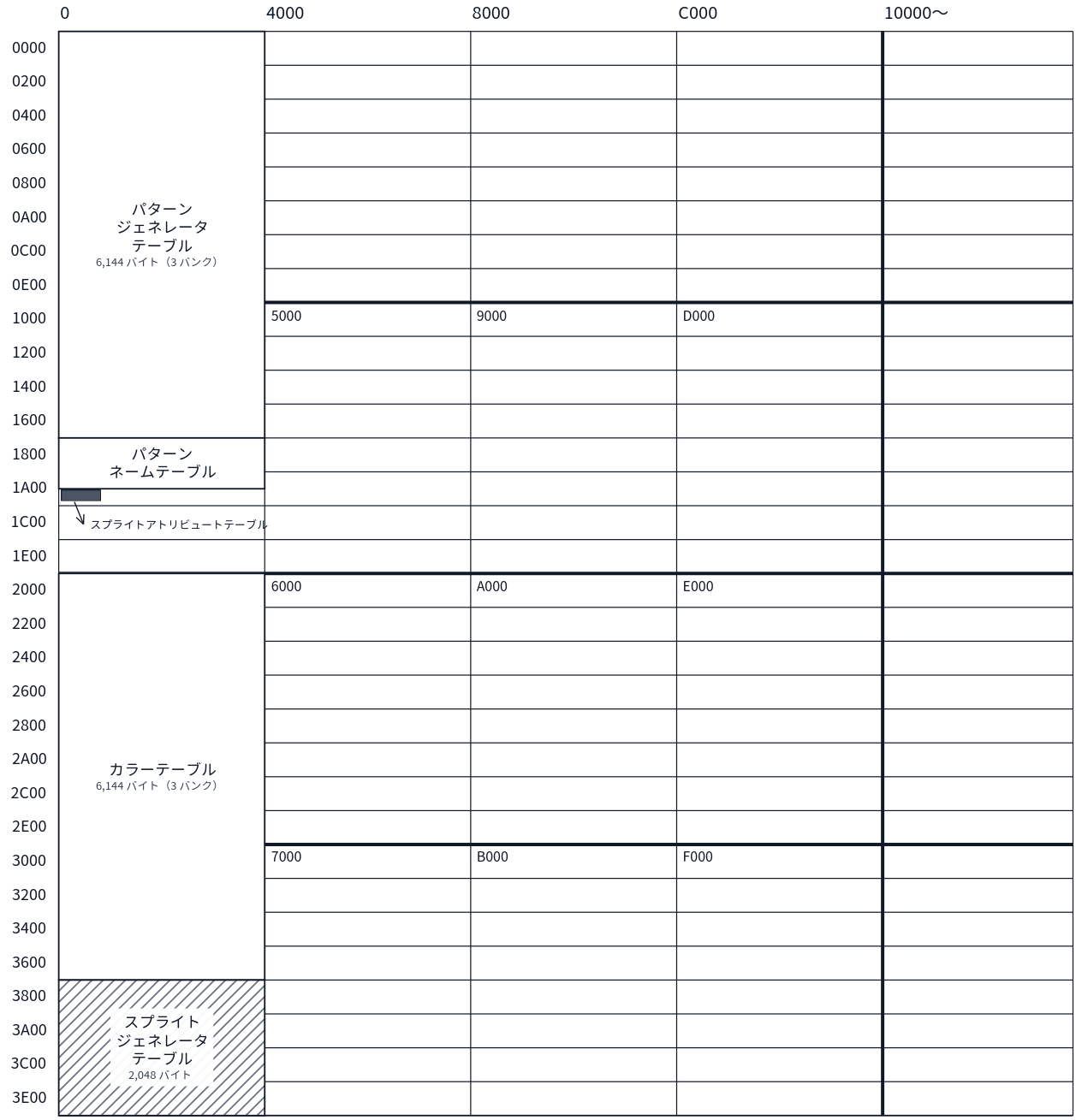



図 C.5 Graphic2 の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | M5 = 0、M4 = 0、M3 = 1 | モード選択ビット |
| R#1 | 60h | BL = 1、IE0 = 1、M1 = 0、M2 = 0、SI = 0、MAG = 0 |
| R#2 | 06h | パターンネームテーブル = 01800h |

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#3 | FFh | カラーテーブル = 02000h（ベースに効くのは C13 で、下位ビットはマスクとして 1 に設定されます。5.7 節参照） |
| R#10 | 00h | カラーテーブルのベースアドレス上位 |
| R#4 | 03h | パターンジェネレータテーブル = 00000h（ベースに効くのは F13 で、下位 2 ビットはマスクとして 1 に設定されます。5.7 節参照） |
| R#5 | 36h | スプライトアトリビュートテーブル = 01B00h |
| R#11 | 00h | スプライトアトリビュートテーブルのベースアドレス上位 |
| R#6 | 07h | スプライトパターンジェネレータテーブル = 03800h |
| R#7 | 07h | 周辺色 7 |

## C.6 Graphic3（SCREEN 4）

VRAM の配置は Graphic2 とほぼ同じで、違いはスプライトがモード 2 になる点と、それに伴ってスプライトアトリビュートテーブルとスプライトカラーテーブルの位置が変わる点だけです（5.8 節）。

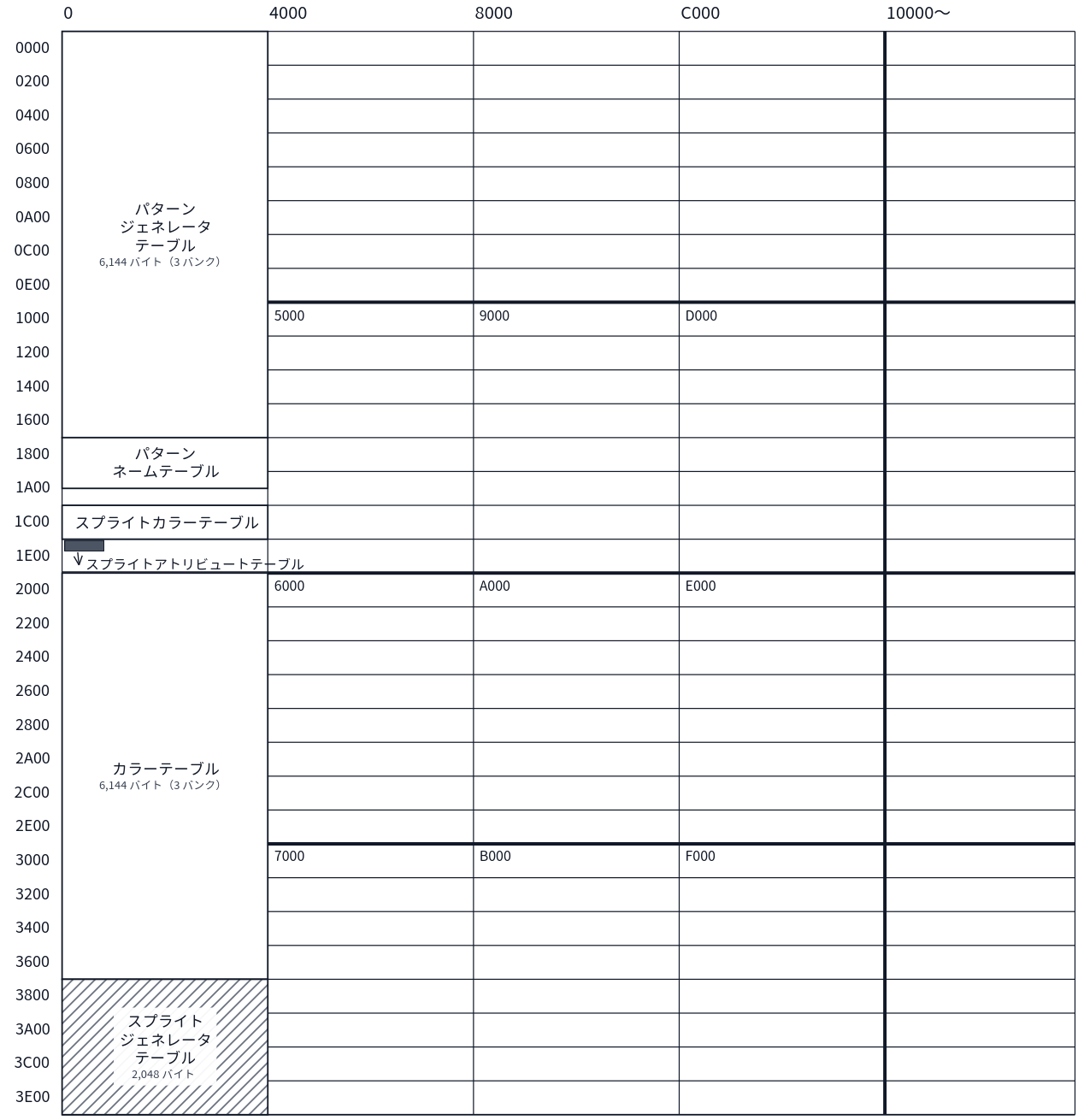



図 C.6 Graphic3 の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | M5 = 0、M4 = 1、M3 = 0 | モード選択ビット |
| R#1 | 60h | BL = 1、IE0 = 1、M1 = 0、M2 = 0、SI = 0、MAG = 0 |
| R#2 | 06h | パターンネームテーブル = 01800h |

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#3 | FFh | カラーテーブル = 02000h（Graphic2 と同じ） |
| R#10 | 00h | カラーテーブルのベースアドレス上位 |
| R#4 | 03h | パターンジェネレータテーブル = 00000h（Graphic2 と同じ） |
| R#5 | 3Fh | スプライトアトリビュートテーブル = 01E00h（スプライトカラーテーブルは 01C00h に自動配置されます。6.4.2 節参照） |
| R#11 | 00h | スプライトアトリビュートテーブルのベースアドレス上位 |
| R#6 | 07h | スプライトパターンジェネレータテーブル = 03800h |
| R#7 | 07h | 周辺色 7 |

## C.7 Graphic4／Graphic5（SCREEN 5／6）

ビットマップモードです。VRAM は 32KB 単位のページに分かれ、ページ 0 の後方にスプライト関係のテーブルが置かれます。スプライト関係のテーブルは表示ページを切り替えても共通で、ページ 1 以降はビットマップだけが置かれます。192 ライン時（LN = 0）はビットマップが 24,576 バイト（ページ内では 00000h～05FFFh）になります。Graphic4 と Graphic5 の配置とレジスタ値は、モード選択ビットを除いて同一です。

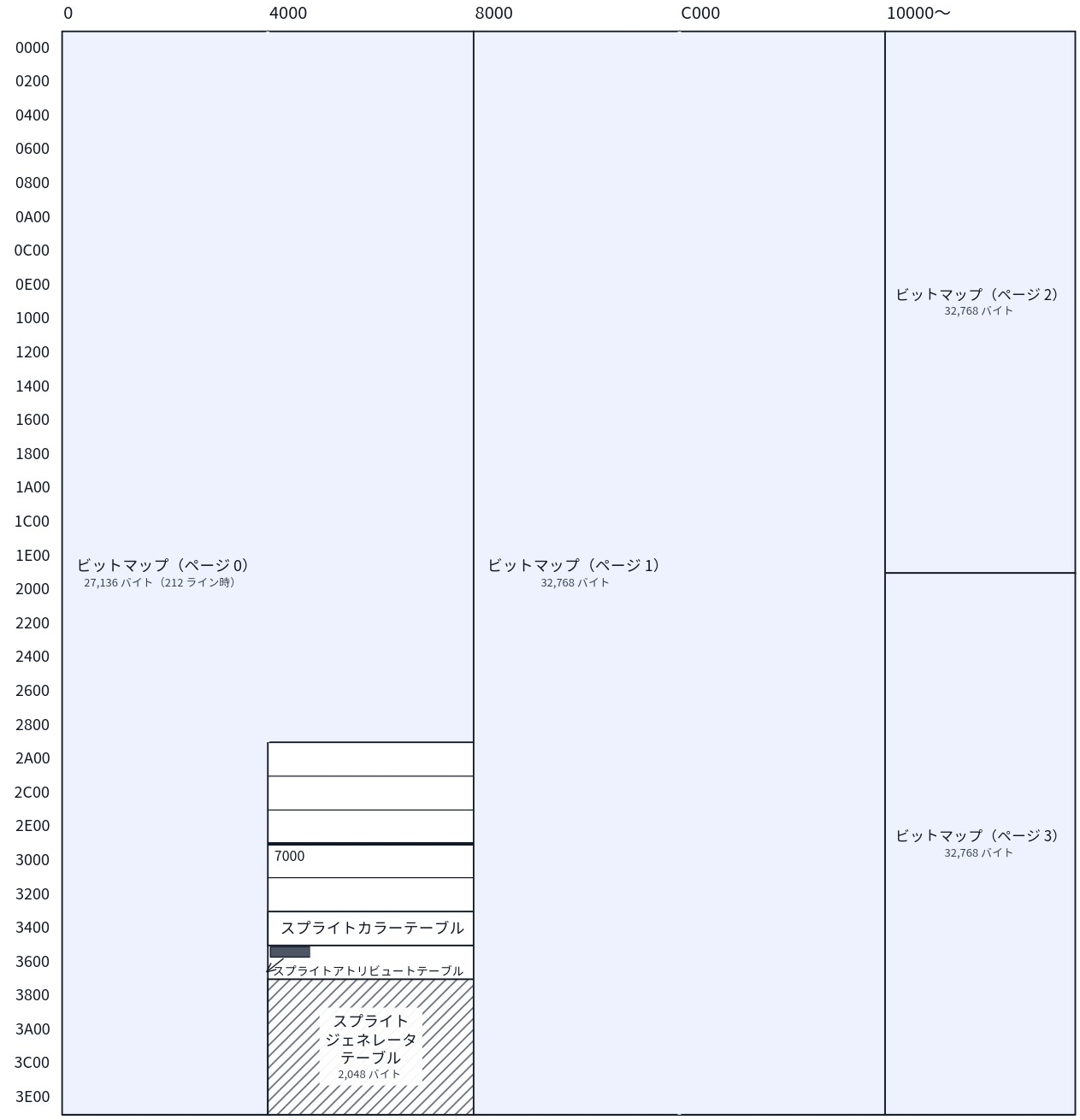



図 C.7 Graphic4／Graphic5 の標準 VRAM マップ

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#0 | Graphic4：M5 = 0、M4 = 1、M3 = 1／<br>Graphic5：M5 = 1、M4 = 0、M3 = 0 | モード選択ビット |
| R#1 | 60h | BL = 1、IE0 = 1、M1 = 0、M2 = 0、SI = 0、MAG = 0（スプライトモード 2） |
| R#2 | 1Fh | ビットマップのページ 0 = 00000h（ページ選択は A17〜A15 で、下位 5 |

| レジスタ | 標準値 | 内容 |
|---|---|---|
| | | ビットは 1 に設定されます） |
| R#5 | EFh | スプライトアトリビュートテーブル = 07600h（スプライトカラーテーブルは 07400h に自動配置されます。6.4.2 節参照） |
| R#11 | 00h | スプライトアトリビュートテーブルのベースアドレス上位 |
| R#6 | 0Fh | スプライトパターンジェネレータテーブル = 07800h |
| R#7 | 07h | 周辺色 7 |
| R#9 | LN = 1 | 212 ライン（LN = 0 で 192 ライン） |

## C.8 Graphic6／Graphic7／Graphic7-YAE／Graphic7-YJK（SCREEN 7／8／10／11／12）

ビットマップモードです。ページは 64KB 単位で、ページ 0 の後方にスプライト関係のテーブルが置かれます。配置とレジスタ値は 4 つのモードで共通し、違いはモード選択ビットと、YAE／YJK の選択（R#25）だけです。192 ライン時（LN = 0）はビットマップが 49,152 バイト（ページ内では 00000h～0BFFFh）になります。

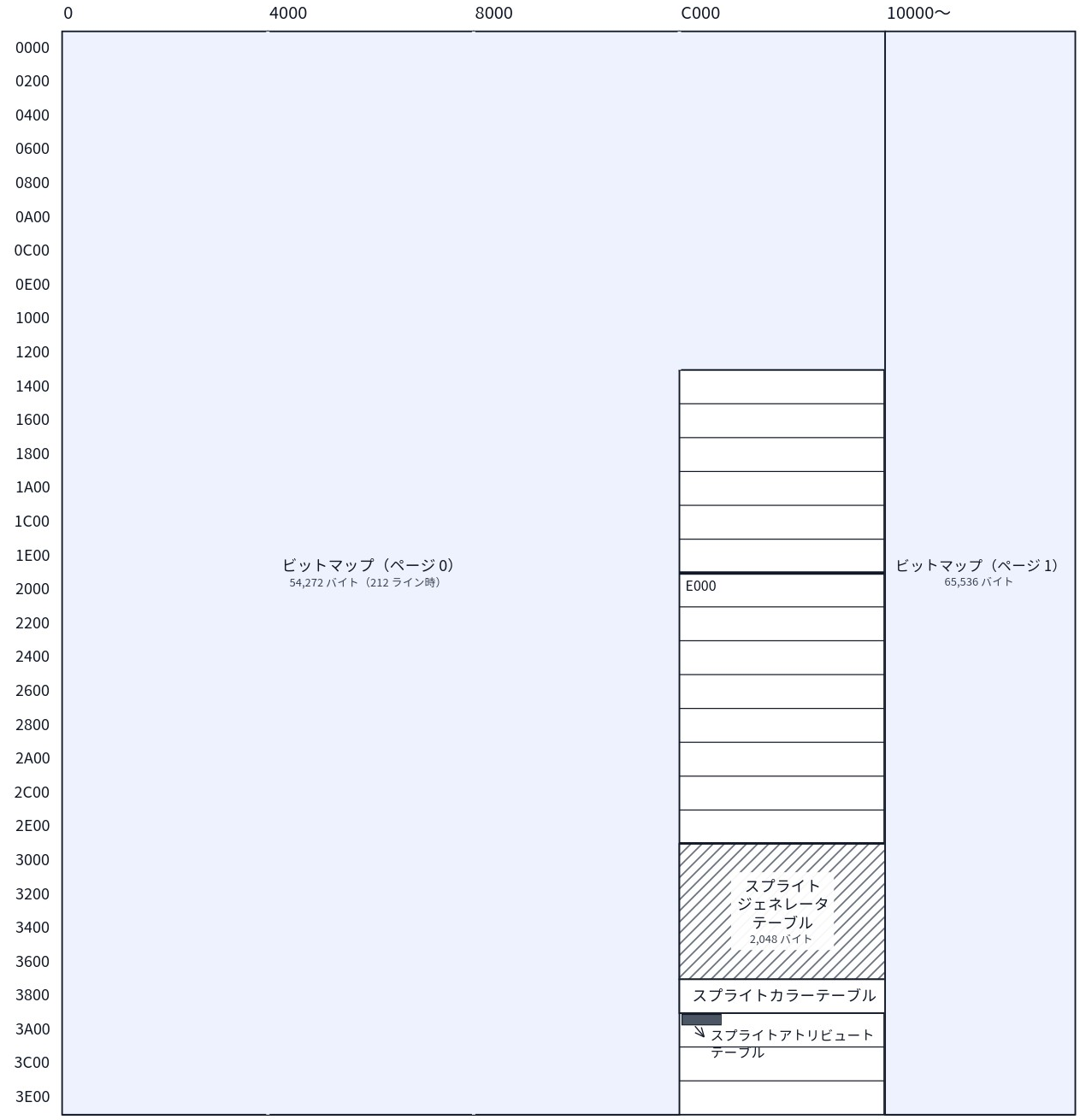



図 C.8 Graphic6／Graphic7 の標準 VRAM マップ（SCREEN 10〜12 も同じ配置）

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#<br>0 | Graphic6：M5 = 1、M4 = 0、M3 = 1／Graphic7 系：<br>M5 = 1、M4 = 1、M3 = 1 | モード選択ビット |

| レジスタ | 標準値 | 内容 |
|---|---|---|
| R#1 | 60h | BL = 1、IE0 = 1、M1 = 0、M2 = 0、SI = 0、MAG = 0（スプライトモード 2） |
| R#2 | 1Fh | ビットマップのページ 0 = 00000h（ページ選択は A17／A16 で、下位 5 ビットは 1 に設定されます） |
| R#5 | F7h | スプライトアトリビュートテーブル = 0FA00h（スプライトカラーテーブルは 0F800h に自動配置されます。6.4.2 節参照） |
| R#11 | 03h | スプライトアトリビュートテーブルのベースアドレス上位 |
| R#6 | 1Eh | スプライトパターンジェネレータテーブル = 0F000h |
| R#7 | 07h | 周辺色 7（Graphic7 では色の解釈が他モードと異なります。5.12 節参照） |
| R#9 | LN = 1 | 212 ライン（LN = 0 で 192 ライン） |
| R#25 | SCREEN 8：YAE = 0、YJK = 0／SCREEN 10／11：YAE = 1、YJK = 1／SCREEN 12：YAE = 0、YJK = 1 | 色表現形式の選択（Graphic7 の変種のみ） |

なお、このグループは SCREEN 7 以上のモードであるため、VRAM の物理配置はインタリーブ（偶数アドレスが DRAM0、奇数アドレスが DRAM1）になります。図 C.8 は論理アドレスの配置を示したものです。また、Sprite mode3（R#20 の SP3）を使うと物理配置はフラットに戻ります。物理配置の詳細と、切り替え時にデータの並びが変わる点への注意は 11.1 節を参照してください。

## 付録 D VDP コマンドコード一覧

コマンドコードは R#46 の bit7〜4 に指定します。bit3〜0 はロジカルオペレーションです（第 8 章参照）。「V9968 新

設」の 3 コマンドは、拡張モード（V58 = 0）でのみ有効で、互換モードでは STOP として扱われます。

| コード | コマンド | 区分 | コード | コマンド | 区分 |
|---|---|---|---|---|---|
| 1111 | HMMC | CPU→VRAM 高速転送 | 0111 | LINE | 直線描画 |
| 1110 | YMMM | VRAM→VRAM 高速転送（Y 方向） | 0110 | SRCH | 色サーチ |
| 1101 | HMMM | VRAM→VRAM 高速転送 | 0101 | PSET | ドット描画 |
| 1100 | HMMV | VRAM 高速塗りつぶし | 0100 | POINT | ドット読み出し |
| 1011 | LMMC | CPU→VRAM 論理転送 | 0011 | LRMM | 回転・拡大縮小付き論理転送 |
| 1010 | LMCM | VRAM→CPU 論理転送 | 0010 | LFMC | CPU→VRAM フォント描画 |
| 1001 | LMMM | VRAM→VRAM 論理転送 | 0001 | LFMM | VRAM→VRAM フォント描画 |
| 1000 | LMMV | VRAM 論理塗りつぶし | 0000 | STOP | 停止 |

## 付録 E I/O ポート一覧

ベースアドレスは BIOS の 0006h / 0007h に格納された値（通常 98h）です（第 2 章参照）。

| ポート | 書き込み | 読み出し |
|---|---|---|
| Port#0（base+0） | VRAM データ書き込み（書き込み後、アドレスはインクリメント） | VRAM データ読み出し |
| Port#1（base+1） | 2 バイト組プロトコル（レジスタ書き込み／VRAM アドレス設定） | R#15 が指すステータスレジスタ |
| Port#2（base+2） | パレットデータ書き込み | FFh（機能なし） |

| ポート | 書き込み | 読み出し |
|---|---|---|
| Port#3（base+3） | R#17 ポインタ経由の間接レジスタ書き込み | FFh（機能なし） |
| Port#4（base+4） | 割り込みフラグクリア＋LockExtReg 設定（1 を書いた要因のみクリア） | 割り込み状態 [LockExtReg][0][0][0][0][CI][LI][FI] |

## 改訂履歴

| 版 | 日付 | 内容 |
|---|---|---|
| 第 1 版 | — | 初版 |

# 索引

本書の重要な用語と、その用語を主に説明しているページの一覧である。ページ番号はリンクになっており、クリックすると該当箇所へ移動する。


| 用語 | ページ |
|---|---|
| 5S / 5TH | 69 |
| 9958 Compatible | 131 |
| CEIE | 16, 121 |
| EPAL | 16, 72 |
| FG4 | 28 |
| FIL | 16 |
| G3BASIC | 116 |
| Geo3D | 108 |
| Graphic7 | 49 |
| HMMC | 84 |
| HMMM | 86 |
| HMMV | 87 |
| HS | 16, 130 |
| ILNS | 16 |
| LFMC | 97 |

| 用語 | ページ |
| --- | --- |
| LFMM | 98 |
| LINE | 91 |
| LMCM | 88 |
| LMMC | 87 |
| LMMM | 89 |
| LMMV | 90 |
| LockExtReg | 11 |
| LRMM | 95 |
| MXC | 28 |
| MXD | 28 |
| MXS | 28 |
| POINT | 94 |
| Port#4 | 10 |
| PSET | 93 |
| R#20 | 16 |
| R#21 | 17 |
| R#25 | 18 |
| R#45 | 28 |

| 用語 | ページ |
| --- | --- |
| R#46 | 29 |
| S16 | 16, 70 |
| SP3 | 16, 56 |
| SPS | 18, 70 |
| SRCH | 92 |
| STOP | 99 |
| SVNS | 16 |
| V58 | 17 |
| VRAM | 127 |
| XHR | 28 |
| YAE | 52, 75 |
| YJK | 53, 75 |
| YMMM | 85 |
| ウィンドウレジスタ（R#51～R#58） | 31 |
| 回転ベクトル（R#47～R#50） | 30 |
| 拡大・縮小 | 67 |
| 拡張モード | 131 |
| コマンド完了割り込み | 121 |

| 用語 | ページ |
| --- | --- |
| 互換モード | 131 |
| 衝突検出 | 69 |
| ステータスレジスタ | 34 |
| スプライトモード 1 | 58 |
| スプライトモード 2 | 59 |
| スプライトモード 3 | 63 |
| 即時モード | 109 |
| テクスチャ | 114 |
| パレット | 71 |
| パレットセット | 61 |
| パレット RAM | 71 |
| フェーズ 2 エンジン | 111 |
| フレーム割り込み | 120 |
| ライン割り込み | 121 |
| リングバッファ | 116 |
| 割り込み | 120 |

# V9968 MSX-VIDEO TECHNICAL DATA BOOK

2026 年 10 月 31 日 発行

著作 原 貴幸 (HRA!)